HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

_____ o0o _____



ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Tên đề tài: Xây dựng nền tảng quản lý, giao bài và chấm bài tự động cho môn Lập trình Web

Sinh viên thực hiện: Vũ Công Tuấn Dương – B22DCKH024
Hoàng Văn Khuyến – B22DCKH070

Giảng viên hướng dẫn: TS Phạm Hoàng Việt

Niên khóa: 2022–2027

Hệ đào tạo: Đại học chính quy

HÀ NỘI, 12/2026

LỜI CẢM ƠN

NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .

Điểm: ..... (bằng chữ: .....)

Hà Nội, ngày ... tháng ... năm ...

CÁN BỘ - GIẢNG VIÊN HƯỚNG DẪN

(Ký, họ tên)

NHẬN XÉT CỦA GIẢNG VIÊN PHẢN BIỆN

. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .

Điểm: ..... (bằng chữ: .....)

Hà Nội, ngày ... tháng ... năm ...

CÁN BỘ - GIẢNG VIÊN PHẢN BIỆN

(Ký, họ tên)

MỤC LỤC

DANH MỤC HÌNH VẼ ..... **vi**

DANH MỤC BẢNG BIỂU ..... **vii**

LỜI MỞ ĐẦU **1**

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG **2**

1.1 Mở đầu 2

1.2 Kiến trúc vi dịch vụ 2

1.2.1 Khái niệm kiến trúc vi dịch vụ 2

1.2.2 So sánh với kiến trúc đơn khối 2

1.3 Ngôn ngữ lập trình và nền tảng ứng dụng 3

1.3.1 Ngôn ngữ Java 3

1.3.2 Framework Spring Boot 4

1.3.3 Spring Cloud OpenFeign 4

1.3.4 Xác thực với Spring Security và OAuth2 4

1.3.5 RESTful API 4

1.4 Cơ sở dữ liệu và công nghệ ORM 4

1.4.1 Hệ quản trị cơ sở dữ liệu quan hệ 4

1.4.2 ORM và JPA 5

1.4.3 Quản lý di chuyển schema 5

1.5 Giao tiếp bất đồng bộ qua hệ thống trao đổi tin nhắn 5

1.5.1 Khái niệm hệ thống trao đổi tin nhắn 5

1.5.2 Apache Kafka 5

1.6 Lưu trữ đối tượng 6

1.6.1 Khái niệm lưu trữ đối tượng 6

1.6.2 RustFS và presigned URL 6

1.7 Đóng gói và điều phối ứng dụng 7
1.7.1 Container và Docker 7
1.7.2 Kubernetes (k3s) 7
1.7.3 Helm và ArgoCD 7
1.8 Quản lý danh tính và truy cập 8
1.8.1 Khái niệm IAM 8
1.8.2 OAuth 2.0 — Khái niệm chung về ủy quyền 8
1.8.3 OpenID Connect và JWT 9
1.8.4 Xác thực trên trình duyệt — Authorization Code Flow và PKCE 9
1.9 Giao diện người dùng phía trước 11
1.9.1 React và TypeScript 11
1.9.2 Các thư viện hỗ trợ 11
1.10 Kết luận chương 11
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 12
2.1 Phân tích hệ thống 12
2.1.1 Xác định các tác nhân trong hệ thống 12
2.1.2 Biểu đồ ca chức năng (usecase) tổng quan của hệ thống 12
2.1.3 Kịch bản các chức năng 14
a Kịch bản các chức năng chung 14
b Kịch bản các chức năng của quản trị viên 16
c Kịch bản các chức năng của giảng viên 17
d Kịch bản các chức năng của sinh viên 22
2.1.4 Biểu đồ lớp toàn hệ thống 27
a Xác định các lớp của hệ thống 27
b Bước 4: Xét quan hệ số lượng giữa các lớp 27
c Bước 5: Bổ sung quan hệ đối tượng giữa các lớp . 29
d Biểu đồ lớp thực thể pha phân tích 31
2.1.5 Phân rã hệ thống theo miền chức năng 32

2.1.6 Biểu đồ lớp chi tiết từng dịch vụ 32
a Biểu đồ lớp pha phân tích miền Xác thực và phân quyền 33
b Biểu đồ lớp pha phân tích dịch vụ quản lý lớp học và điểm 33
c Biểu đồ lớp pha phân tích dịch vụ nộp bài 34
d Biểu đồ lớp pha phân tích dịch vụ thực thi chấm . 34
e Biểu đồ lớp pha phân tích dịch vụ kết quả chấm . 35
f Biểu đồ lớp pha phân tích dịch vụ trợ lý AI 36

2.2 Thiết kế hệ thống 36

2.2.1 Biểu đồ lớp thiết kế chi tiết từng dịch vụ 36
a Dịch vụ Xác thực và phân quyền 37
b Dịch vụ Quản lý lớp học và điểm 37
c Dịch vụ Nộp bài 37
d Dịch vụ Thực thi chấm 38
e Dịch vụ Kết quả chấm 39
f Dịch vụ Trợ lý AI 41

2.2.2 Thiết kế cơ sở dữ liệu theo dịch vụ 41

2.2.3 Sơ đồ tuần tự các chức năng 43
a Đăng nhập 43
b Quản lý lớp học 44
c Quản lý bài tập 45
d Xem kết quả chấm của cả lớp 46
e Nộp bài tập 47
f Xem kết quả chấm bài tập của bản thân 48
g Xem điểm và kết quả cá nhân 48

2.2.4 Kiến trúc hệ thống 49

CHƯƠNG 3: TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG 51

3.1 Triển khai hệ thống 51

3.1.1 Môi trường và kiến trúc triển khai 51

3.1.2 Đóng gói và cấu hình triển khai 52

3.1.3 Tích hợp liên tục với GitHub Actions 53

3.1.4 Triển khai liên tục với ArgoCD 54
3.1.5 Hạ tầng dùng chung 55
3.1.6 Nhất quán dữ liệu trong quy trình chấm bài với Grading Saga 57
3.1.7 Tổng hợp kết quả triển khai 59
3.2 Đánh giá hệ thống..... 59
3.2.1 Tiêu chí và phương pháp đánh giá..... 59
3.2.2 Kịch bản kiểm thử chức năng 60
3.2.3 Kịch bản kiểm thử hiệu năng và độ tin cậy 61
3.2.4 Ưu điểm của kiến trúc triển khai..... 62
3.2.5 Hạn chế và rủi ro..... 63
3.3 Hướng phát triển..... 64
3.4 Kết luận chương 65
TÀI LIỆU THAM KHẢO..... 68

DANH MỤC HÌNH VẼ

Hình 1.1 Luồng xác thực JWT qua API Gateway. 9
Hình 1.2 Luồng xác thực trên trình duyệt với Authorization Code Flow và PKCE. 10

Hình 2.1 Biểu đồ usecase tổng quan của hệ thống. 13
Hình 2.2 Biểu đồ usecase chi tiết Quản lý lớp học của giảng viên. 18
Hình 2.3 Biểu đồ usecase chi tiết Quản lý bài tập của giảng viên. 20
Hình 2.4 Biểu đồ usecase chi tiết Xem kết quả chấm của cả lớp dành cho giảng viên. 21
Hình 2.5 Biểu đồ usecase chi tiết Nộp bài tập của sinh viên. 23
Hình 2.6 Biểu đồ usecase chi tiết Xem điểm và kết quả cá nhân của sinh viên. 24
Hình 2.7 Biểu đồ usecase chi tiết Xem kết quả chấm bài tập của bản thân dành cho sinh viên. 26
Hình 2.8 Biểu đồ lớp thực thể pha phân tích. 31
Hình 2.9 Biểu đồ lớp pha phân tích miền Xác thực và phân quyền. . . . 33
Hình 2.10 Biểu đồ lớp pha phân tích dịch vụ quản lý lớp học và điểm. . . 34
Hình 2.11 Biểu đồ lớp pha phân tích dịch vụ nộp bài. 34
Hình 2.12 Biểu đồ lớp pha phân tích dịch vụ thực thi chấm. 35
Hình 2.13 Biểu đồ lớp pha phân tích dịch vụ kết quả chấm. 35
Hình 2.14 Biểu đồ lớp pha phân tích dịch vụ trợ lý AI. 36
Hình 2.15 Biểu đồ lớp thiết kế toàn hệ thống. 36
Hình 2.16 Biểu đồ lớp thiết kế dịch vụ Xác thực và phân quyền. 37
Hình 2.17 Biểu đồ lớp thiết kế dịch vụ Quản lý lớp học và điểm. 37
Hình 2.18 Biểu đồ lớp thiết kế dịch vụ Nộp bài. 38
Hình 2.19 Biểu đồ lớp thiết kế dịch vụ Thực thi chấm. 39
Hình 2.20 Biểu đồ lớp thiết kế dịch vụ Kết quả chấm. 40
Hình 2.21 Biểu đồ lớp thiết kế dịch vụ Trợ lý AI. 41
Hình 2.22 Biểu đồ quan hệ thực thể (ERD) của hệ thống. 43
Hình 2.23 Sơ đồ tuần tự chức năng Đăng nhập. 44
Hình 2.24 Sơ đồ tuần tự chức năng Xem lớp. 45
Hình 2.25 Sơ đồ tuần tự chức năng Xem chi tiết lớp. 45
Hình 2.26 Sơ đồ tuần tự chức năng Quản lý bài tập. 46
Hình 2.27 Sơ đồ tuần tự chức năng Xem kết quả chấm của cả lớp. 47
Hình 2.28 Sơ đồ tuần tự chức năng Nộp bài tập. 47

Hình 2.29 Sơ đồ tuần tự chức năng Xem kết quả chấm bài tập của bản thân. 48
Hình 2.30 Sơ đồ tuần tự chức năng Xem điểm và kết quả cá nhân. 49
Hình 2.31 Kiến trúc tổng thể của hệ thống. 49

Hình 3.1 Kiến trúc triển khai của hệ thống trên cụm k3s. 51
Hình 3.2 Quy trình CI/CD từ mã nguồn đến cụm triển khai. 54

DANH MỤC BẢNG BIỂU

Bảng 1.1 So sánh kiến trúc đơn khối và kiến trúc vi dịch vụ. 3

Bảng 2.1 Kịch bản chức năng Đăng nhập. 15
Bảng 2.2 Kịch bản chức năng Đổi mật khẩu. 16
Bảng 2.3 Kịch bản chức năng Nhập danh sách tài khoản. 17
Bảng 2.4 Kịch bản chức năng Quản lý lớp học. 18
Bảng 2.5 Kịch bản chức năng Quản lý lớp học (tiếp). 19
Bảng 2.6 Kịch bản chức năng Quản lý bài tập. 20
Bảng 2.7 Kịch bản chức năng Quản lý bài tập (tiếp). 21
Bảng 2.8 Kịch bản chức năng Xem kết quả chấm của cả lớp. 22
Bảng 2.9 Kịch bản chức năng Nộp bài tập. 24
Bảng 2.10 Kịch bản chức năng Xem điểm và kết quả cá nhân. 25
Bảng 2.11 Kịch bản chức năng Xem kết quả chấm bài tập của bản thân. . 26
Bảng 2.12 Phân rã hệ thống thành các miền chức năng. 32

Bảng 3.1 Các thành phần chính của môi trường triển khai. 52
Bảng 3.2 Các giai đoạn của quy trình CI/CD. 54
Bảng 3.3 Các Secret cấu hình cho cụm triển khai. 56
Bảng 3.4 Các trạng thái của một công việc chấm bài. 57
Bảng 3.5 Kịch bản kiểm thử chức năng. 61

Danh mục thuật ngữ và từ viết tắt

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
ACID	Atomicity, Consistency, Isolation, Durability	Các tính chất đảm bảo của giao dịch cơ sở dữ liệu
Ad	Advertisement	Quảng cáo
AI Assistant Service	AI Assistant Service	Dịch vụ sinh giải thích và gợi ý học tập
API	Application Programming Interface	Giao diện lập trình ứng dụng
API Gateway	API Gateway	Cổng API tiếp nhận và định tuyến yêu cầu từ client
ArgoCD	ArgoCD	Công cụ triển khai theo mô hình GitOps
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp liên tục và triển khai liên tục
CNTT		Công nghệ thông tin
Container	Container	Đơn vị thực thi ứng dụng được cô lập
Course Service	Course Service	Dịch vụ quản lý lớp học, bài tập và điểm
CSS	Cascading Style Sheets	Ngôn ngữ định dạng giao diện
ĐATN		Đồ án tốt nghiệp
Docker	Docker	Nền tảng đóng gói và chạy ứng dụng dưới dạng container
EUD	End-User Development	Phát triển ứng dụng người dùng cuối
Executor Service	Executor Service	Dịch vụ thực thi chấm tự động
GWT	Google Web Toolkit	Công cụ lập trình JavaScript bằng Java

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
GitOps	GitOps	Phương pháp quản lý hạ tầng và triển khai dựa trên Git
Grafana	Grafana	Nền tảng trực quan hóa và phân tích số liệu
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HTTP Secure	Giao thức truyền tải siêu văn bản an toàn
IdP	Identity Provider	Nhà cung cấp danh tính
IAM	Identity and Access Management	Quản lý danh tính và quyền truy cập
JPA	Java Persistence API	API chuẩn ánh xạ đối tượng-hệ quan hệ cho Java
JSON	JavaScript Object Notation	Ký hiệu đối tượng JavaScript
JWT	JSON Web Token	Token trao đổi thông tin định dạng JSON, có ký số
k3s	k3s	Phiên bản Kubernetes nhẹ, tối ưu cho môi trường edge
k6	k6	Công cụ kiểm tra hiệu năng (load testing)
Keycloak	Keycloak	Nền tảng quản lý danh tính và truy cập mã nguồn mở
Loki	Loki	Hệ thống tập hợp và truy vấn nhật ký
Microservices	Microservices	Kiến trúc ứng dụng chia thành các dịch vụ nhỏ độc lập
Monolith	Monolith	Kiến trúc đơn khối, toàn bộ ứng dụng trong một tiến trình
OAuth2	OAuth 2.0	Giao thức ủy quyền mở
Access Token	Access Token	Token ngắn hạn dùng để gọi API truy cập tài nguyên

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
Authorization Code Grant	Authorization Code Grant	Phương thức cấp token gồm 2 bước: xin mã ngắn hạn rồi đổi thành token
Authorization Server	Authorization Server	Server xác thực người dùng và cấp phát token
Code Challenge	Code Challenge	Giá trị băm của code_verifier, gửi kèm yêu cầu xin mã xác thực
Code Verifier	Code Verifier	Chuỗi ngẫu nhiên entropy cao, dùng để chứng minh quyền sở hữu authorization code
Client Credentials Grant	Client Credentials Grant	Phương thức cấp token cho giao tiếp máy-máy, không qua người dùng
ID Token	ID Token	JWT chứa thông tin danh tính người dùng, được ký số bởi IdP
Public Client	Public Client	Client không thể giữ bí mật (ví dụ: SPA)
Refresh Token	Refresh Token	Token dài hạn dùng để làm mới access token
Resource Owner Password Credentials Grant	Resource Owner Password Credentials Grant	Phương thức cho phép client nhận trực tiếp username/password; chỉ dùng cho client được tin cậy
Resource Server	Resource Server	Server chứa tài nguyên được bảo vệ
Result Service	Result Service	Dịch vụ lưu và cung cấp kết quả chấm
SPA	Single Page Application	Ứng dụng web chạy một trang, tải lại nội dung động thay vì chuyển trang
PKCE	Proof Key for Code Exchange	Cơ chế bảo vệ authorization code cho client công cộng, dùng cặp code_challenge/code_verifier (S256)
Pod	Pod	Đơn vị nhỏ nhất chạy container trong Kubernetes
Object Storage	Object Storage	Phương thức lưu trữ dữ liệu dưới dạng đối tượng
OIDC	OpenID Connect	Lớp xác thực dựa trên OAuth2

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
ORM	Object-Relational Mapping	Kỹ thuật ánh xạ đối tượng-hệ quan hệ
Presigned URL	Presigned URL	Đường dẫn truy cập tạm thời có ký, cho phép tải lên/xuống trực tiếp
Prometheus	Prometheus	Hệ thống lưu trữ và truy vấn số liệu (metrics)
Pyroscope	Pyroscope	Nền tảng profiling liên tục
PVC	Persistent Volume Claim	Yêu cầu cấp phát lưu trữ bền vững trong Kubernetes
React	React	Thư viện JavaScript xây dựng giao diện người dùng
React Router	React Router	Thư viện định tuyến cho ứng dụng React
REST	Representational State Transfer	Kiến trúc giao tiếp ứng dụng phân tán
RDBMS	Relational Database Management System	Hệ quản trị cơ sở dữ liệu quan hệ
PostgreSQL	PostgreSQL	Hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở
RustFS	RustFS	Hệ thống lưu trữ đối tượng tương thích S3, viết bằng Rust
Saga	Saga	Mẫu phối hợp các bước của một giao dịch phân tán, có thể bù trừ khi lỗi
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
SV		Sinh viên
TanStack Query	TanStack Query	Thư viện quản lý dữ liệu từ server
Tempo	Tempo	Hệ thống lưu trữ và truy vấn traces
TypeScript	TypeScript	Ngôn ngữ lập trình mở rộng JavaScript có kiểu tĩnh
Usecase	Use Case	Ca chức năng
Vite	Vite	Công cụ xây dựng (build tool) cho ứng dụng web
Alloy	Alloy	Bộ thu thập telemetry OTLP của Grafana

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
Ant Design	Ant Design	Bộ thành phần giao diện cho React
Docker Compose	Docker Compose	Công cụ định nghĩa và chạy ứng dụng đa container
DinD	Docker-in-Docker	Kỹ thuật chạy Docker daemon bên trong một container
Flyway	Flyway	Công cụ quản lý di chuyển schema cơ sở dữ liệu
Helm	Helm	Công cụ quản lý gói cho Kubernetes
Java	Java	Ngôn ngữ lập trình hướng đối tượng
keycloak-js	keycloak-js	Thư viện tương tác với Keycloak dành cho JavaScript
kafka	Apache Kafka	Hệ thống trao đổi tin nhắn phân tán
Observability	Observability	Khả năng quan sát trạng thái nội bộ hệ thống
OpenFeign	Spring Cloud OpenFeign	Thư viện client HTTP khai báo cho Spring
Spring Boot	Spring Boot	Framework ứng dụng dựa trên Java
Spring Security	Spring Security	Module bảo mật của Spring
Submission Service	Submission Service	Dịch vụ nhận và lưu bài nộp
i18next	i18next	Thư viện quốc tế hóa ứng dụng

LỜI MỞ ĐẦU

0.1 Vấn đề

Trong các môn lập trình ứng dụng, đặc biệt là môn Lập trình Web, việc chấm bài thủ công bộc lộ nhiều hạn chế: giảng viên mất nhiều thời gian chạy thử từng bài làm, kết quả thiếu nhất quán giữa các lần chấm, sinh viên phải chờ lâu mới nhận được phản hồi. Khó khăn tăng lên khi sĩ số lớp đông và mỗi sinh viên nộp nhiều bài tập trong kỳ.

Yêu cầu đặc thù của môn học khiến bài toán phức tạp hơn: bài làm của sinh viên là mã nguồn phía máy chủ (backend), nên chấm tĩnh dựa trên mã nguồn là không khả thi — hệ thống bắt buộc phải dựng và chạy được server từ bài nộp để kiểm tra hành vi thực tế qua API, đồng thời kiểm tra các điều kiện của cơ sở dữ liệu mà bài tập sử dụng. Việc thực thi mã nguồn không tin cậy đòi hỏi một cơ chế cách ly an toàn, và hệ thống phải chịu được tải khi nhiều sinh viên nộp bài cùng lúc.

0.2 Giải pháp

Đồ án xây dựng hệ thống chấm bài tự động môn Lập trình Web theo kiến trúc vi dịch vụ, gồm cổng API (API Gateway), các dịch vụ quản lý lớp học và bài tập, dịch vụ nộp bài, dịch vụ thực thi chấm bài và dịch vụ quản lý kết quả. Sinh viên đóng gói bài làm thành file zip và tải lên hệ thống lưu trữ đối tượng; sự kiện nộp bài được đưa qua hàng đợi tin nhắn để dịch vụ chấm bài xử lý bất đồng bộ mà không bắt sinh viên chờ đợi. Mỗi bài làm được dựng và chạy trong container Docker cách ly, sau đó hệ thống thực hiện các kịch bản kiểm tra đã được giảng viên cấu hình — từ các phương thức HTTP cơ bản đến kiểm tra cấu trúc bảng, ràng buộc dữ liệu và cách xử lý ngoại lệ. Xác thực và phân quyền tập trung qua Keycloak với vai trò giảng viên và sinh viên; toàn bộ công nghệ sử dụng là mã nguồn mở.

0.3 Cấu trúc báo cáo

Ngoài phần Lời mở đầu, báo cáo gồm ba chương:

- Chương 1 trình bày cơ sở lý thuyết và công nghệ sử dụng trong hệ thống.
- Chương 2 phân tích yêu cầu và thiết kế hệ thống.
- Chương 3 trình bày triển khai, đánh giá hệ thống và hướng phát triển.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

1.1 Mở đầu

Chương này trình bày các cơ sở lý thuyết và công nghệ được sử dụng trong việc xây dựng hệ thống chấm bài tự động môn Lập trình Web. Việc lựa chọn các công nghệ được thực hiện dựa trên những yêu cầu cụ thể của đề tài, bao gồm khả năng cách ly môi trường thực thi mã nguồn của sinh viên, khả năng xử lý bất đồng bộ khi chấm bài hàng loạt, cơ chế xác thực và phân quyền chặt chẽ. Tiêu chí lựa chọn gồm:

- Tính mã nguồn mở (giúp giảm chi phí licence và dễ dàng tùy biến theo nghiệp vụ).
- Khả năng mở rộng độc lập cho từng thành phần.
- Tính an toàn của luồng xác thực trên trình duyệt.
- Việc tách biệt lưu trữ bài nộp sang hệ thống object storage riêng nhằm bảo đảm hiệu năng và độ tin cậy khi số lượng bài nộp tăng lớn.

Các công nghệ được giới thiệu trong chương này đều là các giải pháp mã nguồn mở, hiện đại và được áp dụng rộng rãi trong thực tiễn công nghiệp. Mỗi công nghệ sẽ được trình bày về khái niệm cơ bản, nguyên lý hoạt động, cùng với lý do lựa chọn công nghệ.

1.2 Kiến trúc vi dịch vụ

1.2.1 Khái niệm kiến trúc vi dịch vụ

Kiến trúc vi dịch vụ (microservices architecture) là một phương pháp thiết kế phần mềm, trong đó một ứng dụng được cấu trúc thành một tập hợp các dịch vụ độc lập, nhẹ, mỗi dịch vụ thực hiện một chức năng nghiệp vụ riêng biệt và được triển khai, mở rộng riêng rẽ [1]. Các dịch vụ giao tiếp với nhau thông qua các cơ chế giao tiếp nhẹ, phổ biến nhất là các giao thức HTTP với RESTful API (REST) hoặc qua các hệ thống trao đổi tin nhắn.

Mỗi vi dịch vụ (microservice) trong hệ thống có cơ sở dữ liệu quan hệ (RDBMS) riêng, không chia sẻ trực tiếp dữ liệu với các dịch vụ khác (pattern DB-per-Service) [1]. Việc này đảm bảo tính độc lập về triển khai và khả năng lựa chọn công nghệ phù hợp cho từng nghiệp vụ.

1.2.2 So sánh với kiến trúc đơn khối

Kiến trúc đơn khối (monolithic architecture) - mô hình truyền thống - đóng gói toàn bộ các chức năng của ứng dụng vào một tiến trình duy nhất. Trong mô hình

này, mọi thành phần đều chia sẻ cùng một cơ sở dữ liệu và cùng một không gian tên. Khi hệ thống phát triển, monolith trở nên cồng kềnh, việc sửa đổi một chức năng nhỏ có thể ảnh hưởng đến toàn hệ thống, và khả năng mở rộng bị giới hạn bởi việc bắt buộc phải nhân bản toàn bộ ứng dụng [1].

Bảng 1.1 so sánh hai kiến trúc này:

Tiêu chí	Đơn khối (Monolith)	Vi dịch vụ (Microservices)
Cấu trúc	Toàn bộ ứng dụng trong một tiến trình duy nhất	Nhiều dịch vụ độc lập, mỗi dịch vụ một tiến trình
Cơ sở dữ liệu	Chia sẻ chung một nguồn dữ liệu	Mỗi dịch vụ có cơ sở dữ liệu riêng
Triển khai	Build và deploy toàn bộ cùng lúc	Mỗi dịch vụ được deploy độc lập
Mở rộng	Phải nhân bản toàn bộ ứng dụng	Chỉ nhân bản dịch vụ cần thiết
Sai phạm	Lỗi ở một module có thể làm sập toàn hệ thống	Lỗi bị giới hạn trong phạm vi dịch vụ

Bảng 1.1: So sánh kiến trúc đơn khối và kiến trúc vi dịch vụ.

So với kiến trúc đơn khối, kiến trúc vi dịch vụ mang lại các lợi ích phù hợp với yêu cầu của đề tài:

- Cô lập sai phạm: khi một dịch vụ gặp sự cố, các dịch vụ khác vẫn hoạt động bình thường.
- Tách biệt trách nhiệm: mỗi dịch vụ có một mục tiêu nghiệp vụ duy nhất, giúp việc phát triển và bảo trì đơn giản hơn.
- Mở rộng có chọn lọc: những dịch vụ tốn tài nguyên có thể được nhân bản độc lập mà không cần mở rộng toàn bộ ứng dụng.
- Độc lập về triển khai: mỗi dịch vụ có thể được build, deploy và cập nhật mà không ảnh hưởng đến các dịch vụ khác.

1.3 Ngôn ngữ lập trình và nền tảng ứng dụng

1.3.1 Ngôn ngữ Java

Java là ngôn ngữ lập trình hướng đối tượng, mạnh và phổ biến trong phát triển ứng dụng doanh nghiệp, với hiệu năng cao cùng hệ sinh thái phụ trợ phong phú. Phiên bản Java 21 - phiên bản phát hành dài hạn (LTS) - cung cấp nhiều tính năng hiện đại như records (định dạng dữ liệu gọn nhẹ), pattern matching (khớp mẫu) và virtual threads (luồng ảo, hỗ trợ xử lý I/O với chi phí tài nguyên thấp) [2]. Những tính năng này giúp giảm đáng kể lượng code không cần thiết (boilerplate) và nâng

cao hiệu năng cho các tác vụ I/O nặng như giao tiếp mạng và truy vấn cơ sở dữ liệu quan hệ.

1.3.2 Framework Spring Boot

Spring Boot là một framework ứng dụng dựa trên nền tảng Java, cung cấp cơ chế tự động cấu hình (auto-configuration), nhúng server web (Tomcat) và các starter - tập hợp các phụ thuộc được cấu hình sẵn cho từng tính năng [3]. Spring Boot giúp giảm tối đa khối lượng cấu hình thủ công, cho phép nhà phát triển tập trung vào nghiệp vụ kinh doanh. Trong hệ thống, mỗi vi dịch vụ được xây dựng bằng Spring Boot, đảm bảo tính nhất quán và dễ bảo trì cho toàn bộ hệ thống.

1.3.3 Spring Cloud OpenFeign

Spring Cloud OpenFeign là một thư viện cho phép định nghĩa các client HTTP theo cách khai báo (declarative). Thay vì viết code HTTP thủ công, nhà phát triển chỉ cần định nghĩa một interface với các phương thức tương ứng với các endpoint cần gọi, và OpenFeign tự động tạo phần triển khai để thực hiện gọi HTTP [4]. Trong hệ thống, OpenFeign được sử dụng để các vi dịch vụ nội bộ gọi nhau một cách đơn giản và nhất quán.

1.3.4 Xác thực với Spring Security và OAuth2

Spring Security là module bảo mật của Spring, cung cấp cơ chế xác thực (authentication) và phân quyền (authorization) mạnh mẽ. Kết hợp với OAuth2 - giao thức ủy quyền mở - Spring Boot cung cấp cơ chế OAuth2 Resource Server, cho phép xác thực JWT (JSON Web Token) nhận được từ máy chủ xác thực (Keycloak) mà không cần duy trì trạng thái session phía server [5].

1.3.5 RESTful API

REST (Representational State Transfer) là một kiểu kiến trúc cho các hệ thống phân tán, dựa trên các nguyên tắc: tài nguyên (resource) được xác định bằng URI, các thao tác thực hiện qua các phương thức HTTP (GET, POST, PUT, DELETE), và truyền thông không trạng thái (stateless) [6]. Toàn bộ giao diện của hệ thống đều được xây dựng theo REST, giúp client dễ dàng tương tác và tích hợp.

1.4 Cơ sở dữ liệu và công nghệ ORM

1.4.1 Hệ quản trị cơ sở dữ liệu quan hệ

Cơ sở dữ liệu quan hệ (RDBMS - Relational Database Management System) lưu trữ dữ liệu dưới dạng các bảng gồm các hàng và cột, tuân theo mô hình quan hệ và các tính chất ACID (tính nguyên tử, tính nhất quán, tính cô lập, tính bền vững). PostgreSQL - hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở hàng đầu [7] - được chọn làm hệ thống lưu trữ cho hệ thống nhờ tính chuẩn SQL cao, độ tin cậy, khả

năng mở rộng và các tiện ích mở rộng phong phú. Mỗi vi dịch vụ trong hệ thống có một schema CSDL riêng, đảm bảo tính độc lập.

1.4.2 ORM và JPA

ORM (Object-Relational Mapping) là kỹ thuật ánh xạ giữa các đối tượng trong code và các bảng trong cơ sở dữ liệu quan hệ, giúp nhà phát triển thao tác dữ liệu thông qua đối tượng mà không cần viết SQL thủ công. JPA (Java Persistence API) là chuẩn ORM cho Java, được thực thi bởi Hibernate [8]. Việc sử dụng JPA giúp code sạch hơn, dễ bảo trì và dễ dàng thay đổi hệ quản trị CSDL ở mức cấu hình.

1.4.3 Quản lý di chuyển schema

Flyway là công cụ quản lý các thay đổi schema cơ sở dữ liệu (migration) - mỗi lần thay đổi cấu trúc bảng (tạo bảng, thêm cột) được ghi dưới dạng một phiên bản migration có thể kiểm soát và tái lập [9]. Điều này đảm bảo schema của cơ sở dữ liệu luôn đồng bộ với code của ứng dụng, giảm thiểu sai sót khi deploy.

1.5 Giao tiếp bất đồng bộ qua hệ thống trao đổi tin nhắn

1.5.1 Khái niệm hệ thống trao đổi tin nhắn

Hệ thống trao đổi tin nhắn (message queue) là một phương thức giao tiếp giữa các thành phần phần mềm thông qua việc gửi và nhận các tin nhắn được lưu trữ tạm thời trong hàng đợi. Mô hình này cho phép các bên gửi và nhận hoạt động bất đồng bộ - bên gửi không cần chờ bên nhận xử lý xong. Điều này giúp giảm thời gian chờ, tăng khả năng chịu tải và tách biệt hoàn toàn các thành phần với nhau.

1.5.2 Apache Kafka

Apache Kafka là một hệ thống trao đổi tin nhắn phân tán, được thiết kế để xử lý luồng dữ liệu lớn với throughput cao, độ trễ thấp và khả năng tái sử dụng dữ liệu. Kafka tổ chức tin nhắn theo các topic (chủ đề), mỗi topic được chia thành nhiều partition (phân vùng) để có thể mở rộng song song. Các nhóm consumer (người tiêu thụ) đọc tin nhắn từ topic, cho phép nhiều tác vụ xử lý cùng lúc [10], [11].

Trong kiến trúc vi dịch vụ, Kafka thường được dùng làm hàng đợi sự kiện nối các dịch vụ xử lý bất đồng bộ: dịch vụ tạo sự kiện (producer) publish tin vào topic sau khi hoàn tất một tác vụ, dịch vụ xử lý (consumer) nhận tin và thực thi. Cách làm này đảm bảo:

- Producer và consumer là hai quá trình độc lập; bên gửi không phải chờ bên nhận xử lý xong.
- Nếu consumer bị quá tải hoặc tạm dừng, các tin nhắn vẫn được lưu bền trong Kafka và sẽ được xử lý khi consumer phục hồi.

- Khả năng mở rộng: nhiều consumer có thể cùng đọc từ một topic để xử lý song song [10], [11].

1.6 Lưu trữ đối tượng

1.6.1 Khái niệm lưu trữ đối tượng

Lưu trữ đối tượng (object storage) là phương thức lưu trữ dữ liệu phi cấu trúc (file, ảnh, video, kết quả chấm...) dưới dạng các đối tượng gồm nội dung và metadata (thông tin mô tả), thay vì lưu theo cấu trúc thư mục như filesystem truyền thống. Object storage có khả năng mở rộng gần như vô hạn và phù hợp cho việc lưu trữ khối lượng lớn file nhỏ như bài nộp của sinh viên [12].

Việc sử dụng một máy chủ lưu trữ đối tượng chuyên dụng (object storage server) thay vì lưu bài nộp trực tiếp vào filesystem của server ứng dụng xuất phát từ ba lý do.

1. Bài nộp của sinh viên là các file zip có kích thước biến động lớn; việc xử lý upload qua server ứng dụng sẽ chiếm dụng tài nguyên (CPU, bộ nhớ, băng thông) của các dịch vụ nghiệp vụ — tách riêng object storage giúp giảm tải cho backend và cho phép scale lưu trữ độc lập.
2. Presigned URL cho phép sinh viên tải lên trực tiếp vào object storage mà không cần request đi qua backend, từ đó tăng tốc độ upload và giảm nguy cơ nghẽn ở server ứng dụng.
3. Object storage cung cấp cơ chế sao lưu, nhân bản và truy cập qua giao thức chuẩn (S3) mà filesystem truyền thống không có, giúp hệ thống đáng tin cậy hơn và dễ phục hồi dữ liệu.

1.6.2 RustFS và presigned URL

RustFS là hệ thống lưu trữ đối tượng mã nguồn mở, tương thích với giao thức S3, được viết bằng ngôn ngữ Rust nhằm tận dụng hiệu năng cao và an toàn bộ nhớ của Rust. Hệ thống lưu trữ các file bài nộp dưới dạng zip.

Tiêu chí lựa chọn RustFS gồm:

1. Tương thích cao với giao thức S3, cho phép sử dụng các thư viện MinIO/AWS S3 đã có sẵn cho Java mà không cần thay đổi code [13].
2. Hỗ trợ webhook sẵn, giúp object storage tự động thông báo cho backend khi file bài nộp được upload xong — sự kiện này dùng làm tín hiệu để bắt đầu quy trình chấm bài.
3. Hiệu năng cao nhờ được viết bằng Rust.
4. Mã nguồn mở, phù hợp với định hướng của đồ án [14].

Một cơ chế quan trọng được áp dụng là presigned URL: server tạo một đường dẫn truy cập tạm thời (kèm chữ ký) cho phép sinh viên upload/download trực tiếp bài nộp vào object storage mà không cần đi qua server ứng dụng [12]. Cơ chế này giúp:

- Giảm tải cho backend (không phải truyền file qua server).
- Tiết kiệm băng thông và tài nguyên của server.
- Tăng tốc độ upload cho sinh viên.

1.7 Đóng gói và điều phối ứng dụng

1.7.1 Container và Docker

Docker là nền tảng container hóa ứng dụng, cho phép đóng gói ứng dụng và tất cả các phụ thuộc (runtime, thư viện, cấu hình) vào một docker image (hình ảnh). Image khi được chạy sẽ tạo thành một docker container - một tiến trình chạy trong môi trường cô lập về file hệ thống, mạng và quá trình [15]. Sự cô lập này rất quan trọng với đề tài, vì nó cho phép thực thi mã nguồn của sinh viên mà không ảnh hưởng đến môi trường hệ thống.

Docker Compose là công cụ định nghĩa và chạy các ứng dụng đa container thông qua một file cấu hình, cho phép khởi động đồng thời các container liên quan (ví dụ: container bài làm của sinh viên cùng các dịch vụ phụ trợ).

1.7.2 Kubernetes (k3s)

Kubernetes (thường gọi tắt là K8s) là hệ thống điều phối container (container orchestration), tự động hóa việc triển khai, mở rộng và quản lý các container trên một cụm máy chủ [16]. k3s là phiên bản Kubernetes nhẹ, được tối ưu cho môi trường edge và các cụm nhỏ [17] - phù hợp với hạ tầng hiện tại của hệ thống.

Trong kiến trúc vi dịch vụ, mỗi dịch vụ được đóng gói thành một Docker image và được Kubernetes điều phối, giúp tự động khởi động lại (restart) khi có sự cố, cân bằng tải và mở rộng số lượng bản sao theo nhu cầu.

1.7.3 Helm và ArgoCD

Helm là công cụ quản lý gói (package manager) cho Kubernetes, cho phép đóng gói cấu hình deployment của một dịch vụ thành một chart (bao gồm template Kubernetes và các giá trị cấu hình). Việc sử dụng Helm giúp quản lý phiên bản và tái sử dụng cấu hình cho nhiều môi trường (local, staging, production) [18].

ArgoCD là công cụ triển khai theo mô hình GitOps: toàn bộ cấu hình hệ thống được lưu trữ trong Git, và ArgoCD tự động đồng bộ hóa trạng thái của cluster với Git. Khi cấu hình thay đổi, ArgoCD phát hiện và áp dụng thay đổi một cách liên

tục, đồng thời tự động phát hiện và sửa lỗi nếu cluster bị lệch khỏi cấu hình (self-healing) [19]. Quy trình này giúp việc triển khai trở nên minh bạch, có thể kiểm soát và dễ dàng rollback. Cả Helm và ArgoCD đều là các giải pháp mã nguồn mở, giúp hệ thống có công cụ versioning và GitOps mà không phát sinh chi phí licence.

1.8 Quản lý danh tính và truy cập

1.8.1 Khái niệm IAM

IAM (Identity and Access Management - quản lý danh tính và quyền truy cập) là tập hợp các quy trình và công nghệ để quản lý danh tính số của người dùng và kiểm soát quyền truy cập tài nguyên. IdP (Identity Provider - nhà cung cấp danh tính) là dịch vụ chịu trách nhiệm xác thực danh tính người dùng [20].

1.8.2 OAuth 2.0 — Khái niệm chung về ủy quyền

OAuth 2.0 là một khung ủy quyền (authorization framework) mở, cho phép một ứng dụng bên thứ ba truy cập tài nguyên của người dùng trên server mà không cần biết mật khẩu của người dùng đó [21]. Khác với xác thực (authentication — trả lời câu hỏi "ai là bạn?"), OAuth 2.0 giải quyết bài toán ủy quyền — "bạn được phép làm gì?". Giao thức này định nghĩa bốn vai trò cơ bản:

- Resource Owner: chủ sở hữu tài nguyên, chính là người dùng cuối.
- Client: ứng dụng xin quyền truy cập tài nguyên.
- Resource Server: server chứa tài nguyên được bảo vệ (ở hệ thống, đây là các vi dịch vụ nội bộ).
- Authorization Server: server xác thực người dùng và cấp phát token ủy quyền (ở hệ thống, đây là Keycloak).

OAuth 2.0 định nghĩa một số grant type (phương thức cấp token), mỗi loại phù hợp với một bối cảnh sử dụng:

- Authorization Code Grant — phương thức gồm hai bước: client xin một mã ngắn hạn (authorization code) qua trình duyệt của người dùng, rồi đổi mã đó lấy token tại một endpoint server mà không qua trình duyệt. Đây là grant type an toàn nhất, phù hợp cho các ứng dụng có backend [21].
- Resource Owner Password Credentials Grant (password grant) cho phép client nhận trực tiếp username/password của người dùng; vì mật khẩu đi qua client nên chỉ được dùng cho các client được tin cậy tuyệt đối, không phù hợp cho ứng dụng web công cộng [21].
- Client Credentials Grant dùng cho giao tiếp máy-máy (machine-to-machine), khi client tự xác thực bằng secret của mình mà không cần người dùng [21].

Bên cạnh đó, OAuth 2.0 định nghĩa hai loại token:

- Access token — token ngắn hạn dùng khi gọi API để truy cập tài nguyên [21].
- Refresh token — token dài hạn hơn, dùng để lấy một access token mới mà không cần người dùng đăng nhập lại [21].

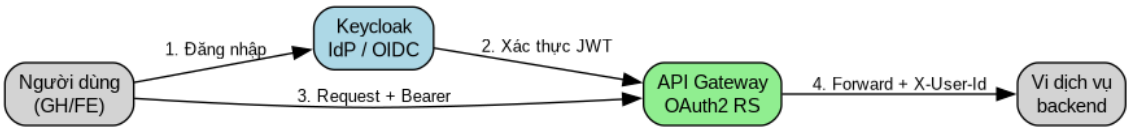
Lý do chọn: Hệ thống lựa chọn Authorization Code Grant vì nó không để token đi qua trình duyệt (chỉ authorization code ngắn hạn), giảm thiểu rủi ro đánh cắp token, đồng thời phù hợp với vai trò của API Gateway làm OAuth2 Resource Server.

1.8.3 OpenID Connect và JWT

OpenID Connect (OIDC) là một lớp xác thực dựa trên OAuth2, cho phép client xác minh danh tính người dùng dựa trên xác thực được thực hiện bởi IdP [22]. Quá trình xác thực kết quả là việc phát hành JWT - một dạng token mã hóa, chứa các thông tin về người dùng (danh tính, vai trò...) và được ký số bởi IdP [22], [23]. JWT cho phép các dịch vụ xác minh danh tính mà không cần gọi lại IdP cho mỗi request.

Trong kiến trúc vi dịch vụ, IdP (ví dụ Keycloak [20]) xác thực người dùng và phát hành JWT. Cổng API (Gateway), với vai trò là OAuth2 Resource Server, xác thực chữ ký của JWT trước khi chuyển request tới các vi dịch vụ. Gateway trích xuất thông tin người dùng (danh tính, vai trò) từ JWT và chuyển tiếp dưới dạng header cho các dịch vụ phía sau. Cách thiết kế này đảm bảo mọi request từ client đều mang JWT hợp lệ, và các service nội bộ không cần trực tiếp xử lý đăng nhập.

Hình 1.1 vẽ lại luồng xác thực JWT qua API Gateway.



Hình 1.1: Luồng xác thực JWT qua API Gateway.

1.8.4 Xác thực trên trình duyệt — Authorization Code Flow và PKCE

Luồng ở mục 1.8.3 mô tả mô hình xác thực dùng JWT qua gateway. Phần này trình bày cách triển khai luồng đó trên client trình duyệt - nơi có những ràng buộc bảo mật đặc thù.

Vì ứng dụng frontend là một SPA (Single Page Application) chạy 100% trên trình duyệt, nó là một client công cộng (public client) — không thể lưu trữ một bí mật (client secret) an toàn trong code JavaScript, vì bất kỳ bí mật nào cũng có thể bị đọc được. Do đó, hệ thống không dùng password grant (yêu cầu client giữ mật khẩu người dùng) mà dùng Authorization Code Grant kết hợp với PKCE (Proof

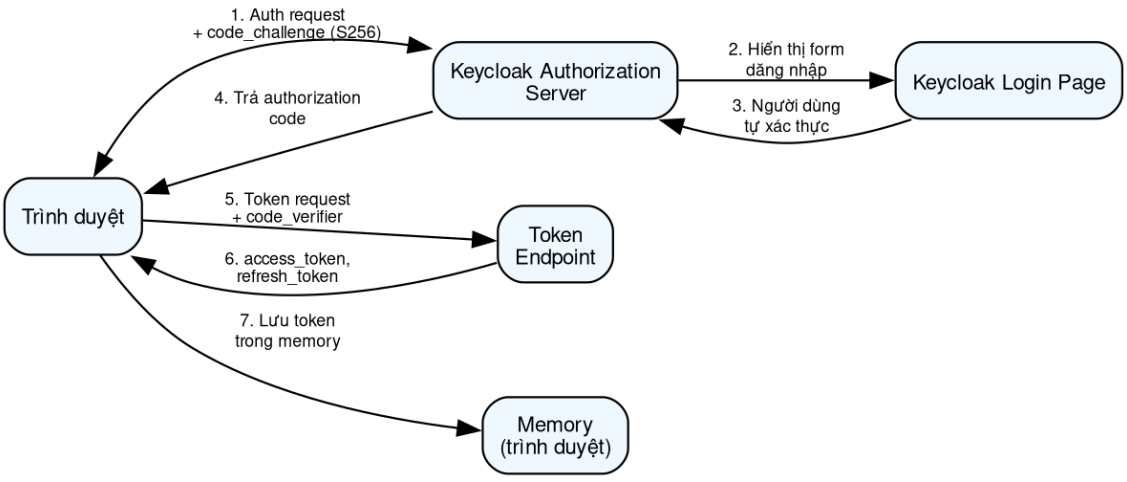
Key for Code Exchange, chuẩn hóa trong RFC 7636 và bắt buộc đối với client công cộng theo RFC 8252) [24], [25].

PKCE là cơ chế bảo vệ authorization code: thay vì dùng client secret, trình duyệt tự sinh một chuỗi ngẫu nhiên entropy cao gọi là code_verifier, tính giá trị băm (hash) của nó theo phương thức S256 (tức giá trị Base64URL-ENCODE(SHA-256(code_verifier))) và gửi kèm code_challenge này vào yêu cầu xin mã xác thực. Sau khi Keycloak trả về authorization code, trình duyệt đổi code thành token bằng cách gửi lại code_verifier thật; Keycloak tính lại giá trị băm và so khớp với code_challenge đã lưu. Nếu khớp, token được cấp phát [24].

Cơ chế này giải quyết bài toán interception attack: nếu một attacker có thể chặn hoặc đánh cắp authorization code (ví dụ qua URL redirect bị giả mạo), chúng vẫn không thể đổi code thành token vì không biết code_verifier — mã chỉ vô dụng nếu không đi kèm chuỗi bí mật sinh tại chỗ [24]. Đồng thời, vì mật khẩu người dùng được nhập trực tiếp trên form do Keycloak quản lý (hosted login page) và không bao giờ đi qua ứng dụng frontend, rủi ro lộ mật khẩu tại client cũng được loại bỏ.

Lý do chọn: Authorization Code Flow + PKCE là phương án an toàn nhất có thể áp dụng cho SPA public client; cơ chế đã được chuẩn hóa (RFC) và được hỗ trợ rộng rãi bởi các thư viện như keycloak-js nên không cần tự triển khai; việc giữ token trong bộ nhớ (memory-only) giúp phiên làm việc chỉ tồn tại trong vòng đời tab, hạn chế thiệt hại nếu trình duyệt bị xâm nhập.

Hình 1.2 minh họa luồng PKCE.



Hình 1.2: Luồng xác thực trên trình duyệt với Authorization Code Flow và PKCE.

1.9 Giao diện người dùng phía trước

1.9.1 React và TypeScript

React là thư viện JavaScript mã nguồn mở do Meta phát triển, dùng để xây dựng giao diện người dùng dựa trên mô hình component (thành phần) độc lập, có thể tái sử dụng. React sử dụng mô hình declarative: nhà phát triển mô tả giao diện mong muốn ở mỗi trạng thái, và React tự động cập nhật DOM [26]. TypeScript là ngôn ngữ lập trình mở rộng JavaScript với hệ thống kiểu tĩnh (static typing), giúp phát hiện lỗi cú pháp và lỗi kiểu dữ liệu ngay trong quá trình phát triển, giảm thiểu lỗi runtime và tăng độ tin cậy cho codebase lớn [27].

1.9.2 Các thư viện hỗ trợ

- Vite: công cụ build (build tool) hiện đại, tận dụng ESM native để cho phép khởi động server phát triển gần như tức thì và build nhanh [28].
- Ant Design: bộ thành phần giao diện (UI library) phong phú, mang lại giao diện chuyên nghiệp, nhất quán cho ứng dụng [29].
- React Router: thư viện định tuyến (routing) cho ứng dụng React, quản lý chuyển đổi giữa các trang mà không cần tải lại trang (SPA) [30].
- TanStack Query: thư viện quản lý dữ liệu từ server (remote data), tự động quản lý cache, làm tươi dữ liệu nền và xử lý lỗi [31].
- keycloak-js: thư viện JavaScript chính thức của Keycloak, tích hợp luồng đăng nhập OIDC vào frontend [20].
- i18next: thư viện quốc tế hóa, cho phép ứng dụng hỗ trợ nhiều ngôn ngữ (tiếng Việt, tiếng Anh) [32].

1.10 Kết luận chương

Chương này đã trình bày các cơ sở lý thuyết và công nghệ được lựa chọn: kiến trúc vi dịch vụ với mẫu DB-per-service; nền tảng Java cùng Spring Boot; lưu trữ quan hệ với ORM; trao đổi tin nhắn bất đồng bộ qua Kafka; lưu trữ đối tượng tương thích S3; đóng gói và điều phối bằng Docker và Kubernetes; quản lý danh tính với OAuth 2.0, OIDC và PKCE; và giao diện React. Các công nghệ được chọn đều là giải pháp mã nguồn mở, ổn định, có cộng đồng hỗ trợ lớn.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Phân tích hệ thống

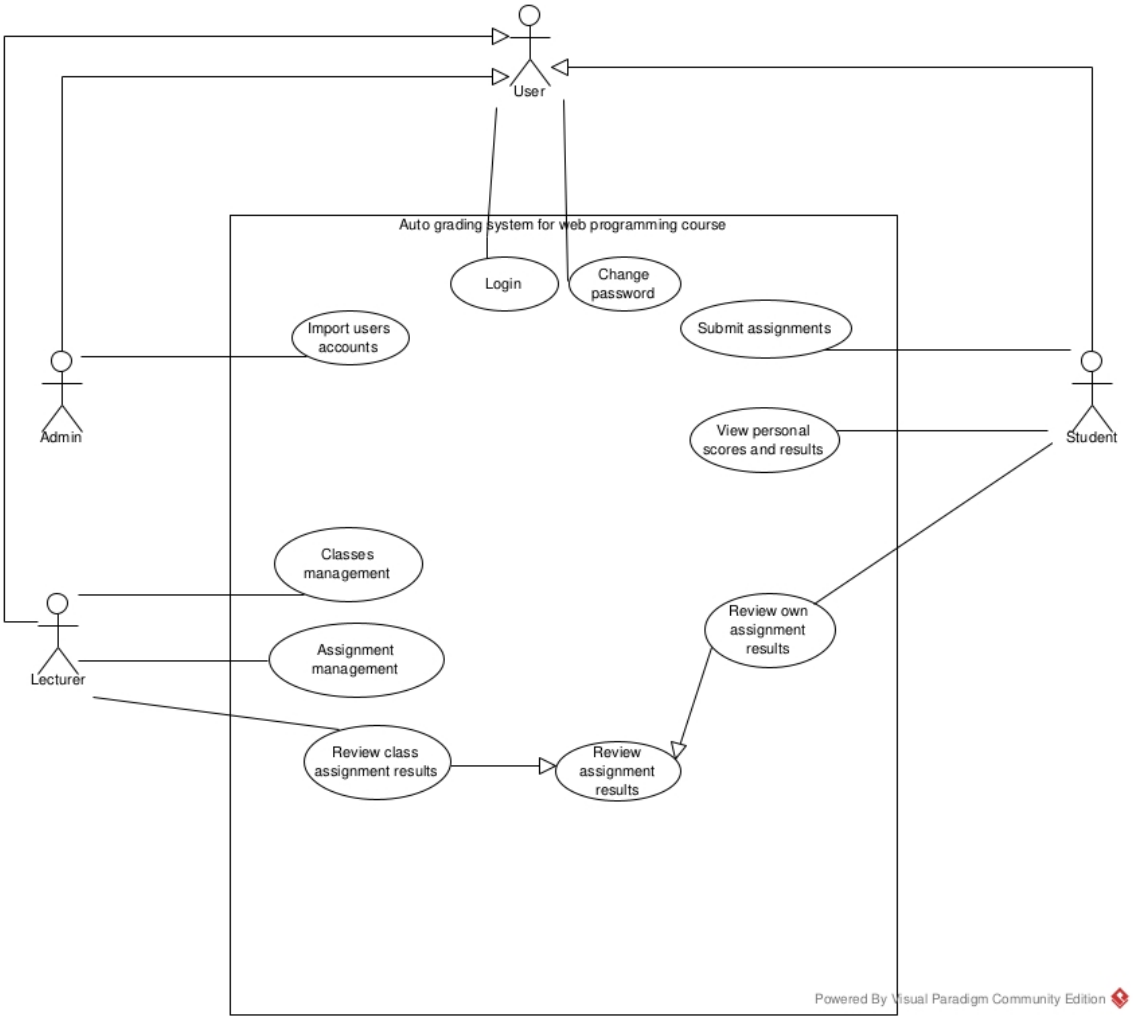
2.1.1 Xác định các tác nhân trong hệ thống

Hệ thống chấm bài tự động có bốn tác nhân, trong đó User là tác nhân trừu tượng còn Quản trị viên, Giảng viên và Sinh viên là các tác nhân cụ thể kế thừa từ User:

- Người dùng (User): tác nhân trừu tượng đại diện cho mọi tài khoản truy cập hệ thống; các tác nhân còn lại đều kế thừa User nên đều thực hiện được hai usecase chung là đăng nhập và đổi mật khẩu.
- Quản trị viên (Admin): quản trị tài khoản ở mức hệ thống; là tác nhân duy nhất được phép nhập danh sách tài khoản người dùng từ tệp CSV để cấp tài khoản ban đầu cho giảng viên và sinh viên.
- Giảng viên (Lecturer): quản lý lớp học của mình gồm tạo lớp, nhập danh sách sinh viên, cấu hình thành phần điểm và xem bảng điểm; quản lý bài tập kèm các bước chấm tự động; xem kết quả chấm bài của cả lớp.
- Sinh viên (Student): tham gia các lớp đã được ghi danh, nộp bài thực hành để hệ thống chấm điểm tự động và xem điểm, kết quả chấm của bản thân.

2.1.2 Biểu đồ ca chức năng (usecase) tổng quan của hệ thống

Hình 2.1 mô tả các ca chức năng tổng quan của hệ thống và các tác nhân tương tác với chúng.



Hình 2.1: Biểu đồ usecase tổng quan của hệ thống.

Mô tả các usecase của hệ thống:

- Đăng nhập: Usecase này cho phép người dùng (quản trị viên, giảng viên và sinh viên) truy cập vào hệ thống bằng tài khoản đã có để thực hiện được các chức năng theo đúng quyền quy định của mình.
- Đổi mật khẩu: Usecase này cho phép người dùng đã đăng nhập thay đổi mật khẩu tài khoản của mình để tiếp tục sử dụng hệ thống một cách an toàn với thông tin đăng nhập mới.
- Nhập danh sách tài khoản: Usecase này cho phép quản trị viên tạo hàng loạt tài khoản cho giảng viên và sinh viên từ tệp CSV để cấp tài khoản ban đầu mà không phải tạo thủ công từng tài khoản. Mỗi tài khoản mới nhận một mật khẩu tạm và bắt buộc đổi mật khẩu ở lần đăng nhập đầu tiên.
- Quản lý lớp học: Usecase này cho phép giảng viên tạo và lưu trữ lớp học của mình, nhập danh sách sinh viên vào lớp, cấu hình các thành phần điểm và xem bảng điểm của lớp để quản lý toàn bộ hoạt động của một lớp học trong một

học kỳ.

- Quản lý bài tập: Usecase này cho phép giảng viên tạo, sửa, công bố và xoá bài tập thực hành trong lớp của mình, kèm theo các bước chấm tự động để hệ thống chấm bài nộp của sinh viên theo đúng yêu cầu đề ra.
- Xem kết quả chấm của cả lớp: Usecase này cho phép giảng viên xem kết quả chấm bài tập của toàn bộ sinh viên trong lớp, bao gồm điểm tổng hợp của từng sinh viên, điểm từng bài nộp và chi tiết từng bước chấm, để theo dõi và đánh giá tình hình học tập của lớp.
- Xem chi tiết kết quả chấm một bài tập: Usecase này là bước chung khi xem chi tiết kết quả chấm của một bài tập: giảng viên đi tới usecase này từ Xem kết quả chấm của cả lớp, còn sinh viên đi tới từ Xem kết quả chấm bài tập của bản thân; phạm vi kết quả nhìn thấy phụ thuộc vào tác nhân gọi nó.
- Nộp bài tập: Usecase này cho phép sinh viên nộp bài thực hành của mình dưới dạng tệp nén để hệ thống tải lên và chấm điểm tự động, đồng thời theo dõi trạng thái chấm và xem lại kết quả của các lần nộp.
- Xem điểm và kết quả cá nhân: Usecase này cho phép sinh viên xem điểm thành phần, điểm tổng kết, điểm chữ và điểm hệ 4 của mình trong mỗi lớp đã ghi danh để tự đánh giá kết quả học tập của bản thân.
- Xem kết quả chấm bài tập của bản thân: Usecase này cho phép sinh viên xem điểm và chi tiết từng bước chấm của các bài tập mình đã nộp, giúp hiểu rõ vì sao bài làm đạt hay chưa đạt yêu cầu của đề bài.

2.1.3 Kịch bản các chức năng

a, Kịch bản các chức năng chung

Bảng 2.1 mô tả kịch bản của usecase Đăng nhập.

Tên mục	Nội dung mô tả
Tên kịch bản	Đăng nhập
Tác nhân	Người dùng (quản trị viên, giảng viên và sinh viên)
Tiền điều kiện	Người dùng đã được cấp tài khoản trên hệ thống (do quản trị viên tạo hoặc nhập từ tệp CSV).
Hậu điều kiện	Người dùng có phiên làm việc hợp lệ; các chức năng tiếp theo được phân quyền theo vai trò.
Kịch bản chính	1. Người dùng truy cập hệ thống và chọn chức năng đăng nhập 2. Hệ thống hiển thị trang đăng nhập. 3. Người dùng nhập tên đăng nhập và mật khẩu và nhấn đăng nhập 4. Nếu tài khoản còn yêu cầu đổi mật khẩu bắt buộc (tài khoản cấp mới bằng mật khẩu tạm), hệ thống yêu cầu người dùng đổi mật khẩu trước khi tiếp tục. 5. Người dùng nhập mật khẩu mới và xác nhận mật khẩu mới và nhấn vào nút xác nhận 6. Hệ thống xác nhận đăng nhập thành công và thiết lập phiên làm việc cho người dùng, mở giao diện tương ứng với vai trò của người dùng (quản trị viên, giảng viên hoặc sinh viên).
Kịch bản ngoại lệ	• Tên đăng nhập hoặc mật khẩu không đúng: trang đăng nhập báo lỗi và cho phép nhập lại. • Nhập sai mật khẩu quá số lần quy định: tài khoản bị tạm khoá để chống dò mật khẩu. • Phiên làm việc hết hạn: hệ thống yêu cầu người dùng đăng nhập lại.

Bảng 2.1: Kịch bản chức năng Đăng nhập.

Bảng 2.2 mô tả kịch bản của usecase Đổi mật khẩu.

Tên mục	Nội dung mô tả
Tên kịch bản	Đổi mật khẩu
Tác nhân	Người dùng đã đăng nhập (quản trị viên, giảng viên và sinh viên)
Tiền điều kiện	Người dùng đang có phiên làm việc hợp lệ.
Hậu điều kiện	Mật khẩu mới được áp dụng cho lần đăng nhập sau; phiên làm việc hiện tại không bị kết thúc nên người dùng tiếp tục làm việc bình thường.
Kịch bản chính	1. Người dùng mở menu tài khoản và chọn chức năng Đổi mật khẩu. 2. Hệ thống hiển thị hộp thoại gồm ba trường: mật khẩu hiện tại, mật khẩu mới và xác nhận mật khẩu mới. 3. Người dùng nhập thông tin; giao diện kiểm tra ngay độ dài tối thiểu và sự trùng khớp giữa mật khẩu mới với phần xác nhận trước khi gửi yêu cầu. 4. Hệ thống xác minh mật khẩu hiện tại, sau đó đặt mật khẩu mới cho tài khoản. 5. Hệ thống thông báo đổi mật khẩu thành công và đóng hộp thoại.
Kịch bản ngoại lệ	• Mật khẩu hiện tại không đúng: hệ thống báo lỗi và yêu cầu nhập lại. • Mật khẩu mới quá ngắn hoặc xác nhận không khớp: giao diện chặn và không gửi yêu cầu. • Hệ thống tạm thời không xử lý được yêu cầu: báo lỗi và yêu cầu người dùng thử lại sau.

Bảng 2.2: Kịch bản chức năng Đổi mật khẩu.

b, Kịch bản các chức năng của quản trị viên

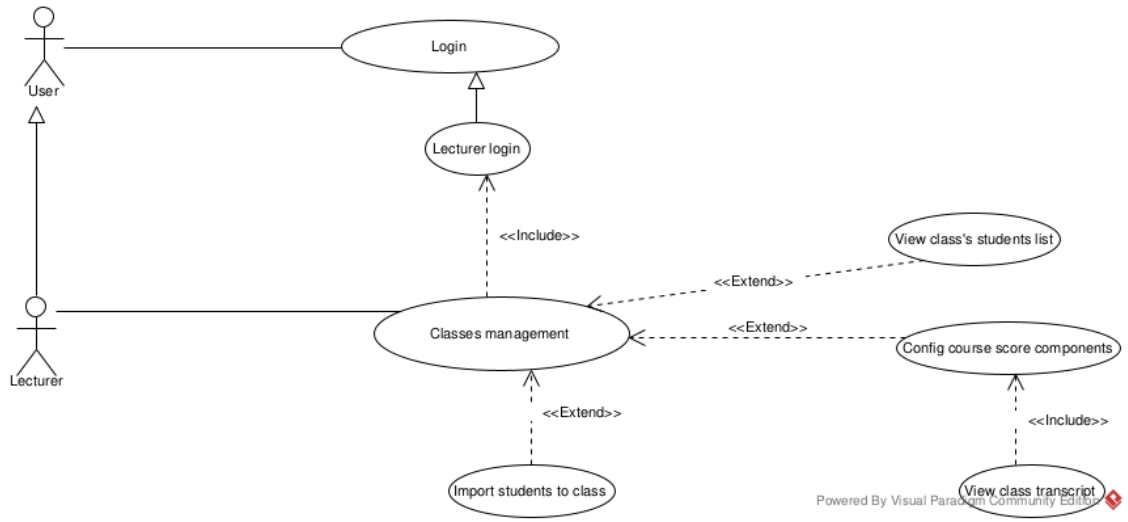
Bảng 2.3 mô tả kịch bản của usecase Nhập danh sách tài khoản.

Tên mục	Nội dung mô tả
Tên kịch bản	Nhập danh sách tài khoản
Tác nhân	Quản trị viên
Tiền điều kiện	Quản trị viên đã đăng nhập bằng tài khoản có vai trò quản trị cũng như có một file CSV theo mẫu với các thông tin người dùng đã có từ trước
Hậu điều kiện	Các tài khoản mới sẵn sàng đăng nhập bằng mật khẩu tạm; người dùng bắt buộc đổi mật khẩu ở lần đăng nhập đầu tiên.
Kịch bản chính	1. Tại giao diện trang chủ dành cho quản trị viên, quản trị viên chọn chức năng nhập tài khoản 2. Hệ thống hiển thị giao diện nhập danh sách tài khoản gồm nút nhập tài khoản 3. Quản trị viên nhấn nút nhập tài khoản và chọn tệp CSV gồm các cột: tên đăng nhập, họ tên, email và vai trò (giảng viên hoặc sinh viên) và nhấn nút gửi 4. Hệ thống trả về kết quả nhập tài khoản gồm số tài khoản đã tạo theo từng vai trò, số dòng bị bỏ qua và danh sách dòng lỗi kèm lý do.
Kịch bản ngoại lệ	• Tệp vượt quá giới hạn kích thước hoặc số dòng: hệ thống từ chối cả tệp và báo lỗi. • Dòng thiếu tên đăng nhập hoặc email, hoặc vai trò không hợp lệ: dòng đó bị bỏ qua và được ghi vào báo cáo kèm lý do, các dòng khác vẫn được xử lý. • Tên đăng nhập hoặc email đã tồn tại: dòng đó bị bỏ qua; nhập lại cùng tệp vẫn an toàn.

Bảng 2.3: Kịch bản chức năng Nhập danh sách tài khoản.

c, Kịch bản các chức năng của giảng viên

Hình 2.2 mô tả chi tiết usecase Quản lý lớp học của giảng viên.



Hình 2.2: Biểu đồ usecase chi tiết Quản lý lớp học của giảng viên.

Kịch bản của chức năng được trình bày trong Bảng 2.4 và Bảng 2.5.

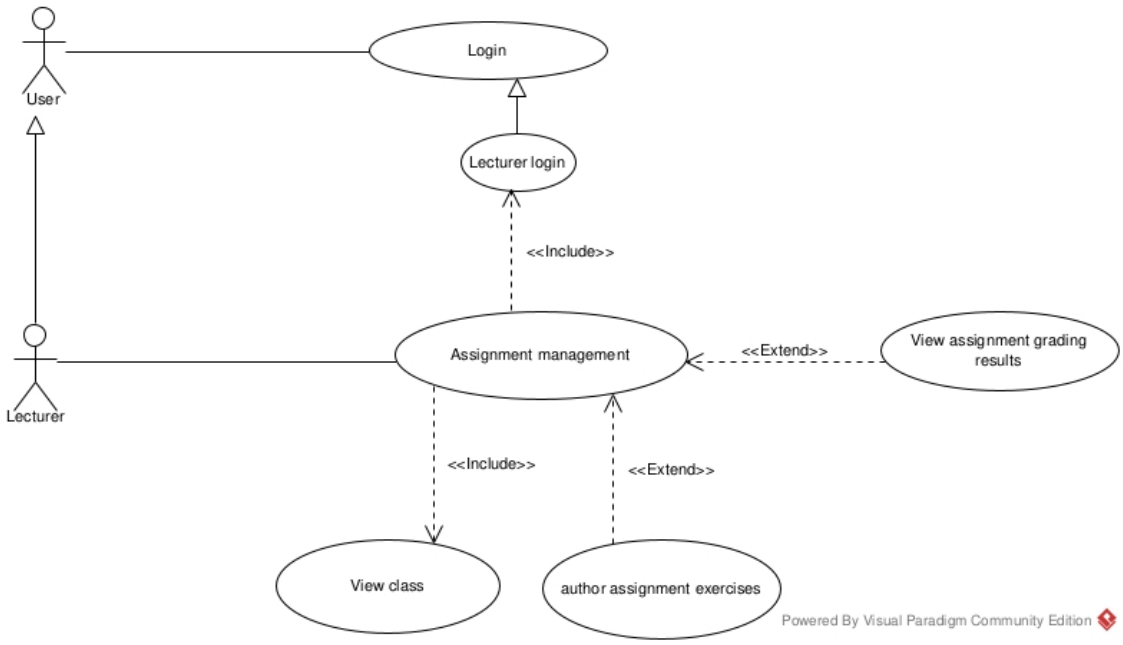
Tên mục	Nội dung mô tả
Tên kịch bản	Quản lý lớp học
Tác nhân	Giảng viên
Tiền điều kiện	Giảng viên đã đăng nhập bằng tài khoản có vai trò giảng viên.
Hậu điều kiện	Lớp học, danh sách sinh viên, cấu trúc điểm và điểm số được lưu trong hệ thống; lớp đã lưu trữ chỉ còn xem và không thể mở lại.
Kịch bản chính	1. Tại giao diện trang chủ dành cho giảng viên, giảng viên nhấn vào nút Vào lớp của tôi. 2. Hệ thống hiển thị giao diện quản lý lớp học gồm danh sách lớp của giảng viên kèm trạng thái, có tìm kiếm theo tên lớp, học kỳ và bộ lọc theo trạng thái. 3. Giảng viên nhấn vào nút tạo lớp mới. 4. Hệ thống hiển thị giao diện tạo lớp mới với các trường thông tin như tên lớp và học kỳ. 5. Giảng viên nhập thông tin lớp và nhấn nút tạo. 6. Hệ thống tạo lớp ở trạng thái đang hoạt động. 7. Giảng viên nhấn nút Vào lớp. 8. Hệ thống hiển thị giao diện quản lý lớp của giảng viên gồm các phần danh sách lớp, nhập danh sách sinh viên, thành phần điểm và bảng điểm của lớp.

Bảng 2.4: Kịch bản chức năng Quản lý lớp học.

Tên mục	Nội dung mô tả
Kịch bản chính (tiếp)	9. Giảng viên nhấn vào phần nhập danh sách sinh viên lớp. 10. Hệ thống hiển thị giao diện nhập danh sách sinh viên lớp gồm bảng kết quả nhập và nút nhập danh sách sinh viên. 11. Giảng viên nhấn nút nhập danh sách sinh viên và chọn tệp CSV tương ứng. 12. Hệ thống báo số sinh viên được thêm và số dòng bị bỏ qua. 13. Giảng viên nhấn vào phần thành phần điểm trên giao diện quản lý chi tiết lớp học 14. Hệ thống hiển thị giao diện thành phần điểm của lớp với danh sách các thành phần điểm và nút cấu hình 15. Giảng viên nhấn nút cấu hình điểm thành phần của lớp 16. Hệ thống hiển thị hộp thoại thành phần điểm, với danh sách các loại điểm thành phần và trọng số để giảng viên nhập 17. Giảng viên cấu hình các thành phần điểm của lớp (chuyên cần, bài tập, thi cuối kỳ) cùng trọng số tương ứng. 18. Hệ thống kiểm tra các ràng buộc về trọng số và lưu cấu trúc điểm. 19. Giảng viên nhập điểm thủ công cho từng sinh viên. 20. Hệ thống lấy điểm thành phần bài tập từ kết quả chấm bài nộp và tính điểm tổng kết, điểm chữ, điểm hệ 4. 21. Giảng viên xem bảng điểm của cả lớp hoặc lưu trữ lớp khi kết thúc học kỳ.
Kịch bản ngoại lệ	• Tên lớp trùng trong cùng học kỳ: hệ thống báo lỗi và không tạo lớp. • Tệp CSV sai định dạng: hệ thống báo lỗi. • Cấu trúc điểm thiếu thành phần thi cuối kỳ, trọng số thành phần này thấp hơn mức tối thiểu hoặc tổng trọng số không bằng 1: hệ thống từ chối lưu. • Sinh viên không thuộc danh sách lớp: hệ thống báo lỗi khi nhập điểm. • Một thành phần điểm chưa có điểm: điểm tổng kết, điểm chữ và điểm hệ 4 để trống trên bảng điểm.

Bảng 2.5: Kịch bản chức năng Quản lý lớp học (tiếp).

Hình 2.3 mô tả chi tiết usecase Quản lý bài tập của giảng viên.



Hình 2.3: Biểu đồ usecase chi tiết Quản lý bài tập của giảng viên.

Kịch bản của chức năng được trình bày trong Bảng 2.6 và Bảng 2.7.

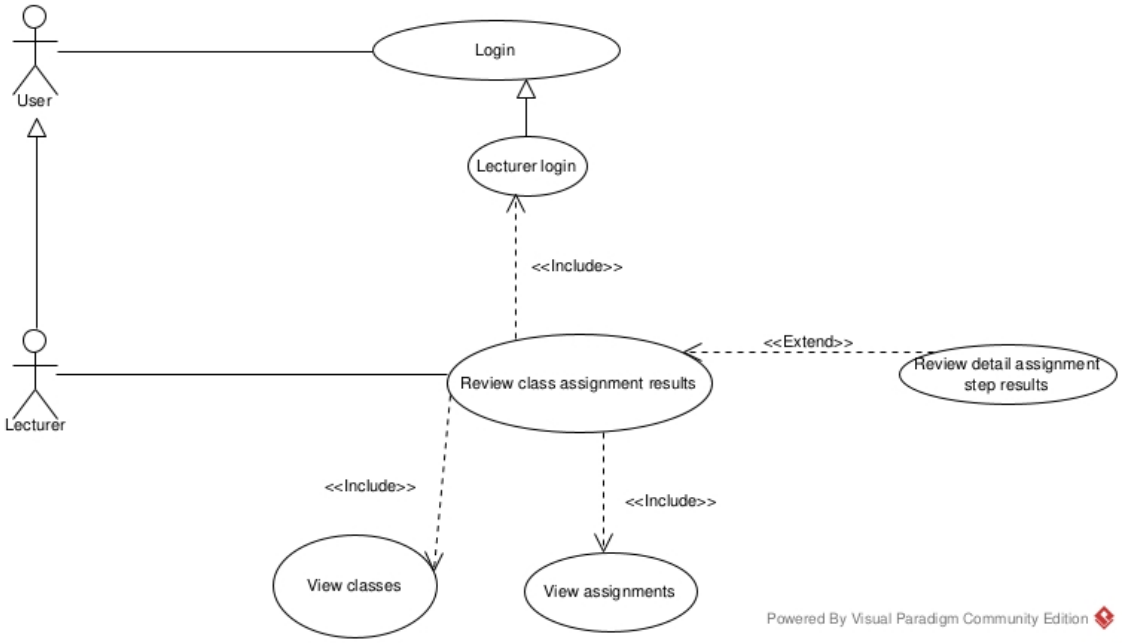
Tên mục	Nội dung mô tả
Tên kịch bản	Quản lý bài tập
Tác nhân	Giảng viên
Tiền điều kiện	Giảng viên đã đăng nhập; lớp được chọn thuộc giảng viên.
Hậu điều kiện	Bài tập cùng các bước chấm được lưu; bài tập đã công bố xuất hiện trong danh sách bài tập của sinh viên đã ghi danh.
Kịch bản chính	1. Giảng viên mở danh sách bài tập của lớp cần quản lý. 2. Hệ thống hiển thị danh sách bài tập kèm bộ lọc theo trạng thái công bố và ô tìm kiếm theo tiêu đề. 3. Giảng viên nhấn nút tạo bài tập mới. 4. Hệ thống hiển thị biểu mẫu gồm tiêu đề và cấu hình chấm (chiến lược chấm, thời gian chờ và giới hạn tài nguyên khi chấm). 5. Giảng viên nhập tiêu đề và cấu hình chấm cho bài tập. 6. Hệ thống lưu bài tập ở trạng thái chưa công bố. 7. Giảng viên soạn các bước chấm tự động cho bài tập: gửi yêu cầu kiểm thử tới ứng dụng của sinh viên, kiểm tra dữ liệu và trích xuất biến để dùng cho các bước sau; mỗi bộ bước như vậy tương ứng với một câu của bài tập. 8. Hệ thống thông báo tạo bài tập thành công

Bảng 2.6: Kịch bản chức năng Quản lý bài tập.

Tên mục	Nội dung mô tả
Kịch bản chính (tiếp)	8. Hệ thống kiểm tra tính hợp lệ của các bước chấm. 9. Giảng viên chọn môi trường chạy cần thiết cho bài tập. 10. Hệ thống lưu các câu và môi trường chạy vào bài tập. 11. Giảng viên công bố bài tập. 12. Hệ thống chuyển bài tập sang trạng thái đã công bố và hiển thị bài tập cho sinh viên đã ghi danh. 13. Giảng viên xem kết quả chấm, sửa thông tin hoặc xoá bài tập khi không còn sử dụng. 14. Hệ thống áp dụng thay đổi tương ứng lên bài tập.
Kịch bản ngoại lệ	• Tiêu đề bài tập trùng trong cùng lớp: hệ thống báo lỗi và không tạo bài tập. • Lớp không thuộc giảng viên: hệ thống báo không tìm thấy. • Cấu hình chấm thiếu thông tin bắt buộc hoặc bước chấm trùng thứ tự: hệ thống báo lỗi kèm vị trí vi phạm. • Bài tập đã xoá: không còn xuất hiện với cả giảng viên và sinh viên.

Bảng 2.7: Kịch bản chức năng Quản lý bài tập (tiếp).

Hình 2.4 mô tả chi tiết usecase Xem kết quả chấm của cả lớp dành cho giảng viên.



Hình 2.4: Biểu đồ usecase chi tiết Xem kết quả chấm của cả lớp dành cho giảng viên.

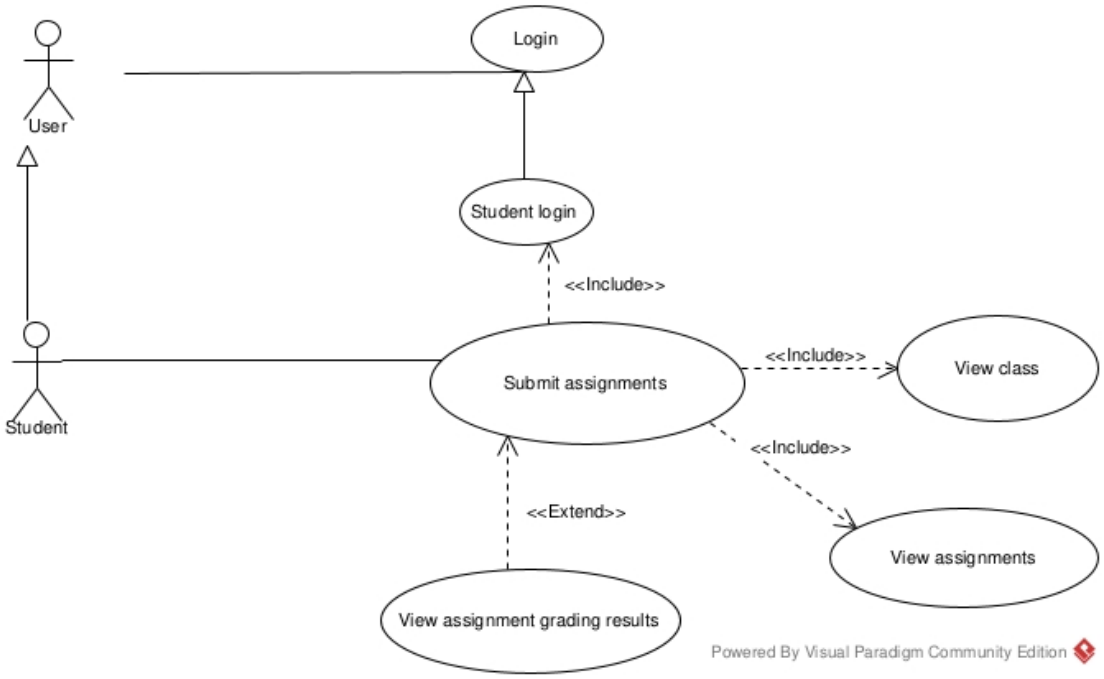
Kịch bản của chức năng được trình bày trong Bảng 2.8.

Tên mục	Nội dung mô tả
Tên kịch bản	Xem kết quả chấm của cả lớp
Tác nhân	Giảng viên
Tiền điều kiện	Giảng viên sở hữu lớp; bài tập đã có ít nhất một bài nộp được chấm.
Hậu điều kiện	Không thay đổi dữ liệu; kết quả chỉ dùng để theo dõi và đánh giá tình hình học tập của lớp.
Kịch bản chính	1. Giảng viên chọn lớp và bài tập cần xem kết quả. 2. Hệ thống hiển thị điểm tổng hợp của từng sinh viên trong lớp cùng điểm từng bài nộp theo từng câu. 3. Giảng viên lọc theo mã sinh viên. 4. Hệ thống thu hẹp danh sách kết quả theo mã sinh viên đã chọn. 5. Giảng viên mở chi tiết một bài nộp. 6. Hệ thống hiển thị kết quả từng bước chấm gồm yêu cầu đã gửi, kết quả nhận về và nhận xét của từng bước. 7. Giảng viên chọn xem danh sách toàn bộ bài nộp của bài tập. 8. Hệ thống hiển thị danh sách bài nộp kèm trạng thái chấm của từng bài.
Kịch bản ngoại lệ	• Bài tập không thuộc giảng viên: hệ thống báo không tìm thấy. • Mã sinh viên không có trong lớp: hệ thống trả về danh sách rỗng. • Bài tập chưa có bài nộp nào: danh sách kết quả rỗng.

Bảng 2.8: Kịch bản chức năng Xem kết quả chấm của cả lớp.

d, Kịch bản các chức năng của sinh viên

Hình 2.5 mô tả chi tiết usecase Nộp bài tập của sinh viên.



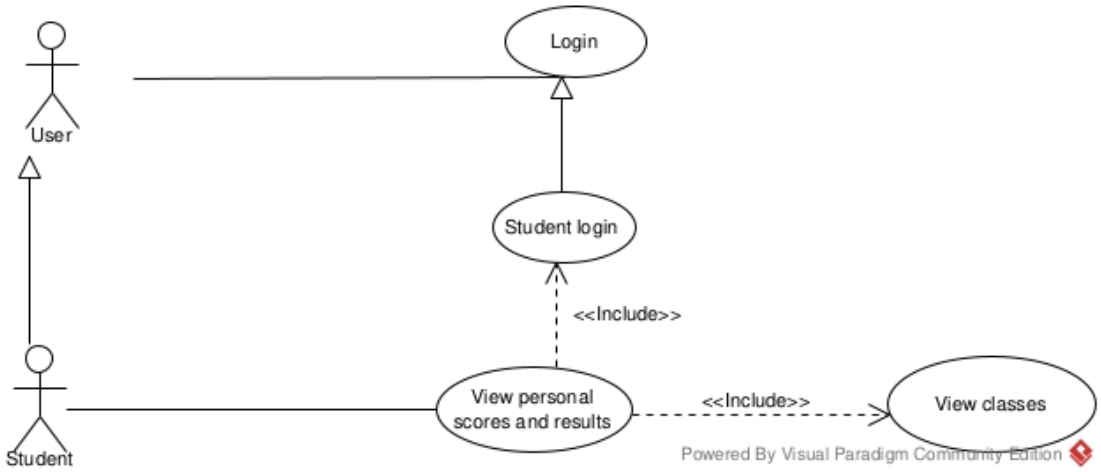
Hình 2.5: Biểu đồ usecase chi tiết Nộp bài tập của sinh viên.

Kịch bản của chức năng được trình bày trong Bảng 2.9.

Tên mục	Nội dung mô tả
Tên kịch bản	Nộp bài tập
Tác nhân	Sinh viên
Tiền điều kiện	Sinh viên đã đăng nhập; đã được ghi danh vào lớp; bài tập đã được công bố.
Hậu điều kiện	Bài nộp được lưu cùng kết quả chấm cho câu đã nộp; nếu chấm thất bại, bài nộp giữ trạng thái thất bại kèm nhật ký để sinh viên xem lại và nộp lại.
Kịch bản chính	1. Sinh viên chọn lớp và bài tập cần nộp. 2. Hệ thống hiển thị đề bài dưới dạng các bước yêu cầu, ẩn phần cấu hình chấm nội bộ của giảng viên. 3. Sinh viên chọn câu cần nộp hoặc nộp cho toàn bộ bài tập. 4. Hệ thống hiển thị yêu cầu nộp tệp nén bài làm. 5. Sinh viên tải tệp nén bài làm lên hệ thống. 6. Hệ thống tiếp nhận tệp, tự động chấm bài trong môi trường cách ly theo các bước đã soạn và tổng hợp điểm. 7. Sinh viên theo dõi trạng thái chấm. 8. Hệ thống hiển thị điểm và chi tiết từng bước chấm khi quá trình chấm hoàn tất.
Kịch bản ngoại lệ	• Sinh viên chưa ghi danh hoặc bài tập chưa công bố: hệ thống báo không tìm thấy. • Tệp nộp sai định dạng hoặc tải lên thất bại: hệ thống báo lỗi và sinh viên thực hiện lại. • Quá trình chấm lỗi (ứng dụng không khởi động được, hết thời gian chạy hoặc bước chấm sai cấu hình): bài nộp chuyển trạng thái thất bại kèm nhật ký lỗi.

Bảng 2.9: Kịch bản chức năng Nộp bài tập.

Hình 2.6 mô tả chi tiết usecase Xem điểm và kết quả cá nhân của sinh viên.



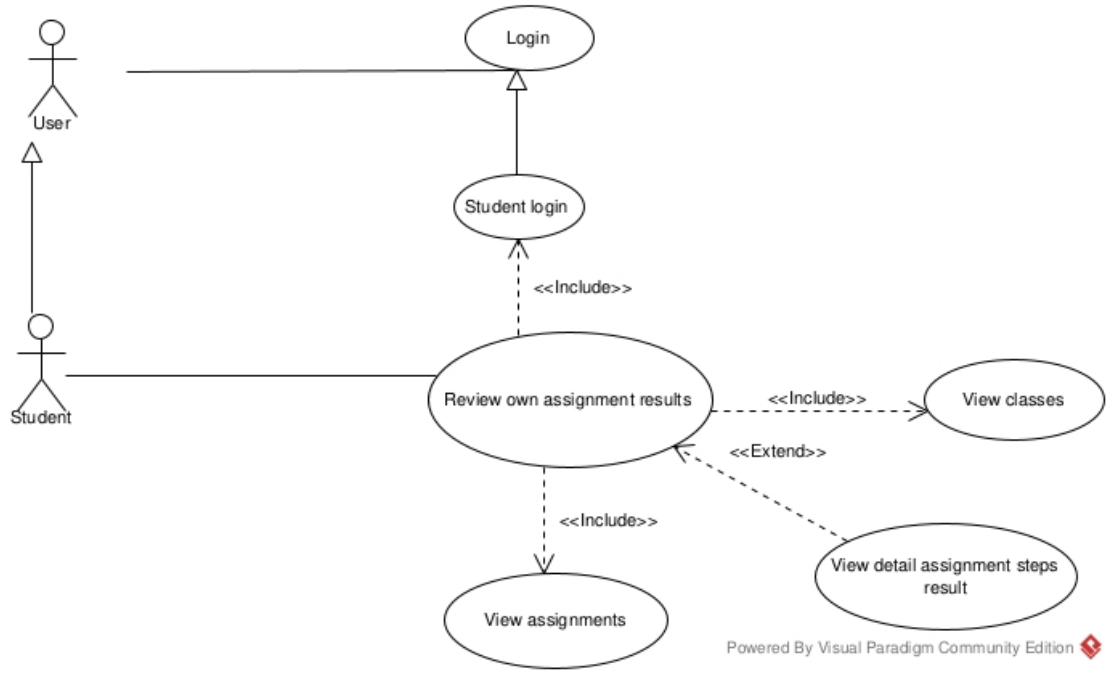
Hình 2.6: Biểu đồ usecase chi tiết Xem điểm và kết quả cá nhân của sinh viên.

Kịch bản của chức năng được trình bày trong Bảng 2.10.

Tên mục	Nội dung mô tả
Tên kịch bản	Xem điểm và kết quả cá nhân
Tác nhân	Sinh viên
Tiền điều kiện	Sinh viên đã đăng nhập và được ghi danh trong lớp.
Hậu điều kiện	Không thay đổi dữ liệu.
Kịch bản chính	1. Sinh viên chọn lớp đã ghi danh. 2. Hệ thống hiển thị điểm từng thành phần kèm trọng số và, khi đã đủ điểm các thành phần, điểm tổng kết, điểm chữ và điểm hệ 4. 3. Sinh viên chọn xem danh sách sinh viên cùng lớp. 4. Hệ thống hiển thị danh sách sinh viên cùng lớp, chỉ gồm mã sinh viên và họ tên.
Kịch bản ngoại lệ	• Sinh viên không ghi danh trong lớp: hệ thống báo không tìm thấy lớp. • Còn thiếu điểm thành phần: điểm tổng kết, điểm chữ và điểm hệ 4 để trống; sinh viên xem lại sau khi giảng viên nhập đủ điểm. • Một thành phần điểm bị điểm 0: điểm chữ xếp F bất kể điểm tổng kết.

Bảng 2.10: Kịch bản chức năng Xem điểm và kết quả cá nhân.

Hình 2.7 mô tả chi tiết usecase Xem kết quả chấm bài tập của bản thân dành cho sinh viên.



Hình 2.7: Biểu đồ usecase chi tiết Xem kết quả chấm bài tập của bản thân dành cho sinh viên.

Kịch bản của chức năng được trình bày trong Bảng 2.11.

Tên mục	Nội dung mô tả
Tên kịch bản	Xem kết quả chấm bài tập của bản thân
Tác nhân	Sinh viên
Tiền điều kiện	Sinh viên đã đăng nhập; sinh viên đã nộp bài và bài tập đã được chấm
Hậu điều kiện	Sinh viên xem được kết quả chấm bài thành công
Kịch bản chính	1. Sinh viên chọn lớp và bài tập. 2. Hệ thống hiển thị danh sách bài nộp của sinh viên trong bài tập. 3. Sinh viên chọn bài nộp của mình. 4. Hệ thống hiển thị điểm, trạng thái chấm, nhật ký chấm và kết quả từng bước. 5. Sinh viên mở chi tiết một bước chấm. 6. Hệ thống hiển thị yêu cầu đã gửi, kết quả nhận về và nhận xét của bước đó, với thông tin nội bộ nhạy cảm được ẩn.
Kịch bản ngoại lệ	• Bài nộp không thuộc sinh viên: hệ thống báo không tìm thấy. • Bài nộp chưa chấm xong: hệ thống hiển thị trạng thái đang chấm; sinh viên xem lại sau.

Bảng 2.11: Kịch bản chức năng Xem kết quả chấm bài tập của bản thân.

2.1.4 Biểu đồ lớp toàn hệ thống

a, Xác định các lớp của hệ thống

Áp dụng kỹ thuật trích danh từ trên các use case và kịch bản đã mô tả ở các mục trước, các danh từ thu được gồm: tài khoản, người dùng, vai trò, quản trị viên, giảng viên, sinh viên, lớp học, học kỳ, danh sách sinh viên, mã sinh viên, họ tên, email, bài tập, câu, bước chấm, cấu hình chấm, môi trường chạy, thành phần điểm, trọng số, điểm thành phần, điểm tổng kết, điểm chữ, điểm hệ 4, bảng điểm, bài nộp, tệp nén, trạng thái chấm, kết quả chấm, kết quả từng bước, nhật ký chấm, giải thích của trợ lý AI, gợi ý và từ khoá học tập.

Loại bỏ các danh từ nằm ngoài phạm vi mục đích của hệ thống, các danh từ trùng lặp và các danh từ chỉ là thuộc tính của lớp khác (trọng số, trạng thái, mã sinh viên, họ tên, email, thời gian chờ, giới hạn tài nguyên), ta thu được các lớp thực thể sau:

Người dùng (User), Vai trò (Role), Liên kết người dùng - vai trò (UserRole), Lớp học (CourseClass), Sinh viên thuộc lớp (ClassStudent), Bài tập (Assignment), Câu (TestPlan), Bước chấm (TestStep), Thành phần điểm (ScoreComponent), Điểm thành phần (StudentScore), Bài nộp (Submission), Công việc chấm (GradingJob), Phiên chấm (GradingSaga), Bước của phiên chấm (GradingSagaStep), Nhật ký chấm (GradingLog), Kết quả chấm (Result), Kết quả từng bước (StepResult), Giải thích của trợ lý AI (AiExplanation).

Hai lớp Người dùng (User) và Vai trò (Role) biểu diễn tác nhân và quyền truy cập của hệ thống; lớp UserRole biểu diễn liên kết nhiều-nhiều giữa hai lớp này.

Trong mô hình toàn hệ thống, kết quả thực thi từng bước và kết quả chấm từng bước được mô hình hóa chung thành lớp StepResult; bản ghi thực thi riêng của dịch vụ thực thi được trình bày ở mục 2.1.6.

b, Bước 4: Xét quan hệ số lượng giữa các lớp

Xác định quan hệ số lượng giữa các lớp dựa trên số lượng đối tượng có thể tham gia quan hệ ở mỗi phía. Với quan hệ 1 - n, một đối tượng ở phía 1 có thể liên kết với nhiều đối tượng ở phía n, nhưng mỗi đối tượng ở phía n chỉ liên kết với đúng một đối tượng ở phía 1. Các quan hệ 1 - n trong hệ thống được xác định như sau:

- Một giảng viên có thể sở hữu nhiều lớp học, nhưng mỗi lớp học chỉ thuộc về đúng một giảng viên. Vì vậy quan hệ giữa User và CourseClass là 1 - n.
- Một lớp học có nhiều sinh viên, nhưng mỗi bản ghi sinh viên trong lớp chỉ thuộc về đúng một lớp. Vì vậy quan hệ giữa CourseClass và ClassStudent là 1 - n.
- Một lớp học có nhiều bài tập, nhưng mỗi bài tập chỉ thuộc về đúng một lớp. Vì

vậy quan hệ giữa CourseClass và Assignment là 1 - n.

- Một lớp học có nhiều thành phần điểm, nhưng mỗi thành phần điểm chỉ thuộc về đúng một lớp. Vì vậy quan hệ giữa CourseClass và ScoreComponent là 1 - n.
- Một bài tập có nhiều câu, nhưng mỗi câu chỉ thuộc về đúng một bài tập. Vì vậy quan hệ giữa Assignment và TestPlan là 1 - n.
- Một câu có nhiều bước chấm, nhưng mỗi bước chấm chỉ thuộc về đúng một câu. Vì vậy quan hệ giữa TestPlan và TestStep là 1 - n.
- Một bài tập có thể nhận nhiều bài nộp từ nhiều sinh viên và nhiều lần nộp, nhưng mỗi bài nộp chỉ thuộc về đúng một bài tập. Vì vậy quan hệ giữa Assignment và Submission là 1 - n.
- Một sinh viên trong lớp có thể nộp nhiều bài tập và có thể nộp lại cùng một bài tập nhiều lần, nhưng mỗi bài nộp chỉ thuộc về đúng một sinh viên trong lớp. Vì vậy quan hệ giữa ClassStudent và Submission là 1 - n.
- Mỗi bài nộp chỉ sinh ra đúng một công việc chấm, và mỗi công việc chấm chỉ gắn với đúng một bài nộp. Vì vậy quan hệ giữa Submission và GradingJob là 1 - 1.
- Một công việc chấm có thể có nhiều phiên chấm do mỗi lần chấm lại tạo một phiên mới, nhưng mỗi phiên chấm chỉ thuộc về đúng một công việc chấm. Vì vậy quan hệ giữa GradingJob và GradingSaga là 1 - n.
- Một phiên chấm có nhiều bước, nhưng mỗi bước chỉ thuộc về đúng một phiên chấm. Vì vậy quan hệ giữa GradingSaga và GradingSagaStep là 1 - n.
- Một công việc chấm có nhiều nhật ký, nhưng mỗi nhật ký chỉ thuộc về đúng một công việc chấm. Vì vậy quan hệ giữa GradingJob và GradingLog là 1 - n.
- Một bài nộp có thể sinh ra nhiều kết quả chấm, bao gồm kết quả theo từng câu và kết quả toàn bài, nhưng mỗi kết quả chỉ thuộc về đúng một bài nộp. Vì vậy quan hệ giữa Submission và Result là 1 - n.
- Một kết quả chấm có nhiều kết quả bước, nhưng mỗi kết quả bước chỉ thuộc về đúng một kết quả chấm. Vì vậy quan hệ giữa Result và StepResult là 1 - n.
- Một bài nộp có thể có nhiều giải thích của trợ lý AI theo các danh mục khác nhau, nhưng trong mỗi danh mục chỉ có tối đa một giải thích đang hiệu lực. Vì vậy quan hệ tổng quát giữa Submission và AiExplanation là 1 - n, còn theo từng danh mục là 1 - 0..1.

Bên cạnh các quan hệ 1 - n, hệ thống còn có các quan hệ n - n cần được tách thành các quan hệ 1 - n thông qua lớp trung gian. Các quan hệ n - n được xác định như sau:

- Một người dùng có thể giữ nhiều vai trò và một vai trò có thể được gán cho nhiều người dùng, do đó quan hệ giữa User và Role là n - n.

- Một sinh viên trong lớp có điểm ở nhiều thành phần điểm và một thành phần điểm áp dụng cho nhiều sinh viên, do đó quan hệ giữa ClassStudent và ScoreComponent là n - n.
- Một bài tập được giao cho nhiều sinh viên và một sinh viên nhận nhiều bài tập, do đó quan hệ giữa Assignment và ClassStudent là n - n. Quan hệ này được biểu diễn thông qua lớp Submission.

Việc tách các quan hệ n - n được thực hiện bằng cách xác định một lớp trung gian sao cho mỗi phía của quan hệ ban đầu có quan hệ 1 - n với lớp trung gian. Sau đó, cần kiểm tra xem lớp trung gian có thỏa mãn điều kiện của lớp kết hợp hay không.

- Với quan hệ giữa User và Role, ta thêm lớp UserRole. Một người dùng có nhiều bản ghi UserRole và một vai trò cũng có nhiều bản ghi UserRole, nhưng mỗi cặp User-Role chỉ có duy nhất một bản ghi UserRole. Do đó UserRole là lớp kết hợp, và mỗi bản ghi chỉ cần giữ userId và roleId.
- Với quan hệ giữa ClassStudent và ScoreComponent, ta thêm lớp StudentScore. Một sinh viên có nhiều bản ghi StudentScore và một thành phần điểm cũng có nhiều bản ghi StudentScore, nhưng mỗi cặp ClassStudent-ScoreComponent chỉ có duy nhất một bản ghi StudentScore. Do đó StudentScore là lớp kết hợp, và mỗi bản ghi giữ điểm của sinh viên ở thành phần điểm tương ứng.
- Với quan hệ giữa Assignment và ClassStudent, ta xét lớp Submission làm lớp trung gian. Một bài tập có nhiều bài nộp và một sinh viên cũng có nhiều bài nộp, nhưng mỗi bài nộp chỉ thuộc về đúng một bài tập và đúng một sinh viên. Để kiểm tra Submission có phải lớp kết hợp hay không, ta xét câu hỏi: một cặp Assignment-ClassStudent có duy nhất một Submission không? Câu trả lời là không, vì sinh viên có thể nộp lại cùng một bài tập nhiều lần. Mỗi lần nộp tạo ra một bản ghi Submission mới, và chỉ một bản ghi được đánh dấu latest để biểu diễn bài nộp hiện tại. Vì không duy nhất trên mỗi cặp Assignment-ClassStudent, Submission không phải là lớp kết hợp. Submission là lớp thường đóng vai trò lớp trung gian, có định danh riêng và có các thuộc tính như tên tệp, đường dẫn tệp, trạng thái chấm và cờ latest.

Như vậy, chỉ UserRole và StudentScore là hai lớp kết hợp trong mô hình. Submission không phải lớp kết hợp vì một cặp Assignment-ClassStudent có thể có nhiều lần nộp.

c, Bước 5: Bổ sung quan hệ đối tượng giữa các lớp

Sau khi xác định quan hệ số lượng, Bước 5 bổ sung quan hệ đối tượng giữa các lớp. Với mỗi quan hệ 1 - n, ta xét xem đối tượng ở phía n có phải là thành phần của

đối tượng ở phía 1 hay không. Nếu có, quan hệ được biểu diễn bằng composition; nếu đối tượng ở phía n tồn tại độc lập, quan hệ được biểu diễn bằng aggregation hoặc association.

Các quan hệ thành phần trong hệ thống gồm:

- CourseClass và Assignment là quan hệ thành phần vì bài tập chỉ tồn tại trong phạm vi một lớp học và không còn ý nghĩa nếu lớp học bị xóa.
- CourseClass và ScoreComponent là quan hệ thành phần vì thành phần điểm chỉ có ý nghĩa trong một lớp học cụ thể.
- Assignment và TestPlan là quan hệ thành phần vì câu chỉ tồn tại trong một bài tập.
- TestPlan và TestStep là quan hệ thành phần vì bước chấm chỉ tồn tại trong một câu.
- Assignment và Submission là quan hệ thành phần vì bài nộp chỉ tồn tại trong phạm vi một bài tập.
- Submission và Result là quan hệ thành phần vì kết quả chấm chỉ tồn tại khi có bài nộp tương ứng.
- Submission và AiExplanation là quan hệ thành phần vì giải thích của trợ lý AI gắn với một bài nộp cụ thể.
- GradingJob và GradingSaga là quan hệ thành phần vì phiên chấm thuộc một công việc chấm.
- GradingSaga và GradingSagaStep là quan hệ thành phần vì bước chấm thuộc một phiên chấm.
- GradingJob và GradingLog là quan hệ thành phần vì nhật ký chấm thuộc một công việc chấm.
- Result và StepResult là quan hệ thành phần vì kết quả bước thuộc một kết quả chấm.

Quan hệ aggregation trong hệ thống gồm:

- CourseClass và ClassStudent là quan hệ aggregation vì sinh viên thuộc một lớp học nhưng không phải là thành phần bị xóa theo lớp. Thông tin sinh viên vẫn có ý nghĩa độc lập với lớp học.

Các quan hệ liên kết trong hệ thống gồm:

- User và CourseClass là quan hệ liên kết vì giảng viên sở hữu lớp học nhưng không phải là thành phần của lớp.
- ClassStudent và Submission là quan hệ liên kết vì bài nộp gắn với sinh viên nhưng không phải là thành phần của sinh viên.
- Submission và GradingJob là quan hệ liên kết 1 - 1 vì công việc chấm là bản ghi

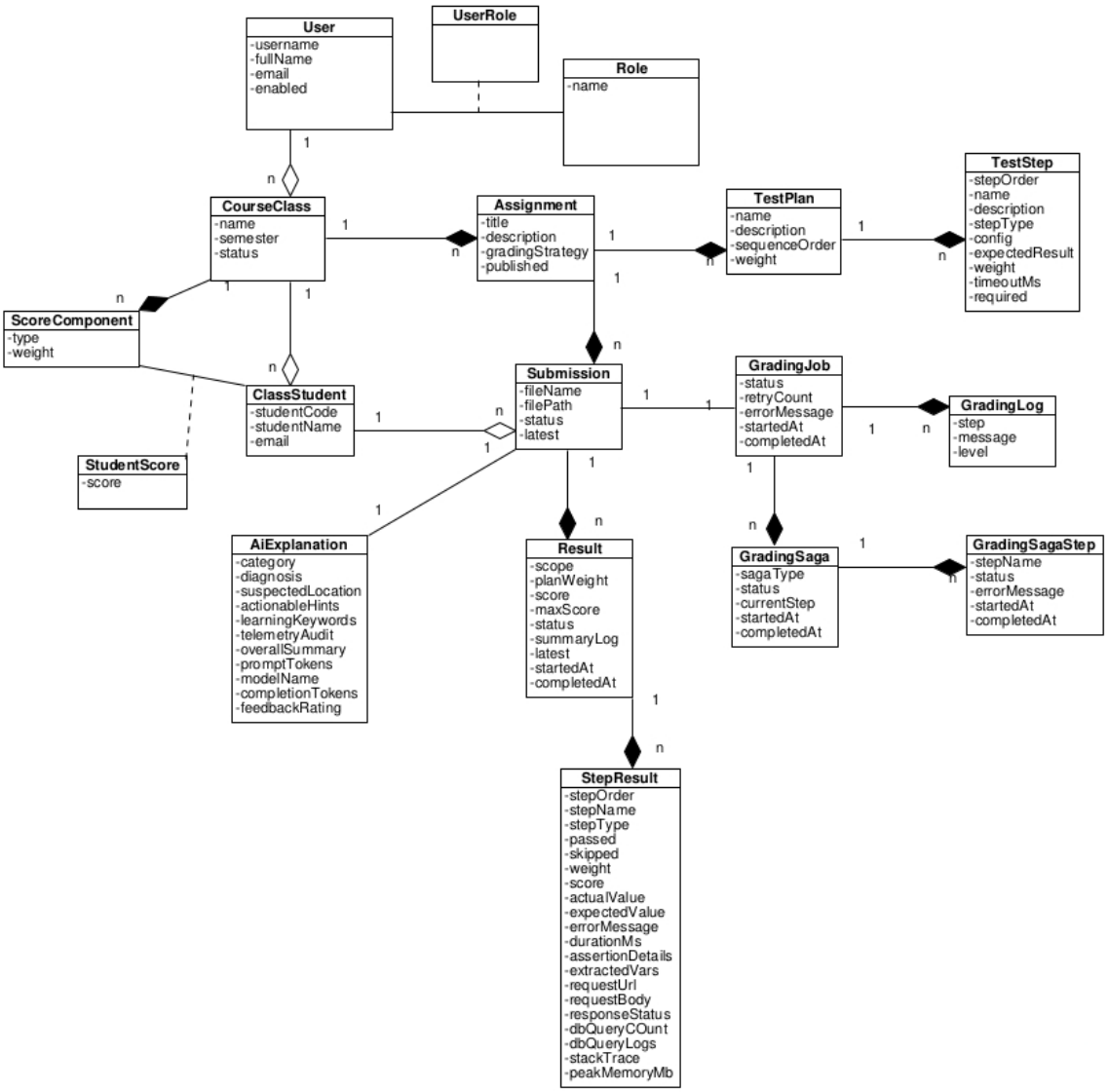
xử lý riêng, không phải thành phần của bài nộp.

Hai lớp kết hợp trong mô hình gồm:

- UserRole là lớp kết hợp giữa User và Role, dùng để biểu diễn quan hệ n - n kèm khóa userId và roleId.
- StudentScore là lớp kết hợp giữa ClassStudent và ScoreComponent, dùng để biểu diễn quan hệ n - n kèm điểm của sinh viên.

Như vậy, mô hình chỉ còn hai lớp kết hợp là UserRole và StudentScore. ClassStudent là lớp thường vì có định danh riêng và các thuộc tính studentCode, studentName, email. Submission cũng là lớp thường vì có định danh riêng, có nhiều bản ghi trên một cặp Assignment-ClassStudent và có thuộc tính tệp nộp, trạng thái chấm, cờ latest.

d, Biểu đồ lớp thực thể pha phân tích



Hình 2.8: Biểu đồ lớp thực thể pha phân tích.

2.1.5 Phân rã hệ thống theo miền chức năng

Dựa trên kết quả phân tích nghiệp vụ ở các mục trước, hệ thống được chia thành các miền chức năng nhằm xác định ranh giới nghiệp vụ và phạm vi xử lý của từng nhóm chức năng. Việc phân rã này giúp làm rõ cấu trúc logic của hệ thống, đồng thời tạo tiền đề để chuyển đổi các miền chức năng thành các dịch vụ độc lập trong kiến trúc vi dịch vụ ở giai đoạn thiết kế.

Quá trình phân rã được thực hiện dựa trên các tiêu chí:

- Tính độc lập nghiệp vụ.
- Khả năng triển khai và mở rộng riêng biệt.
- Mức độ liên kết giữa các chức năng và mức độ phụ thuộc dữ liệu.
- Ranh giới tự nhiên của các thực thể trong mô hình domain.

Hệ thống được phân rã thành các miền chức năng sau:

STT	Miền chức năng	Mô tả
1	Xác thực và phân quyền	Quản lý tài khoản, đăng nhập, đổi mật khẩu, cấp và kiểm tra vai trò.
2	Quản lý lớp học và điểm	Quản lý lớp học, nhập danh sách sinh viên, cấu hình thành phần điểm, nhập điểm thủ công và tính điểm tổng kết.
3	Nộp bài	Nhận tệp bài nộp của sinh viên, lưu trữ tệp và chuyển yêu cầu chấm.
4	Thực thi chấm	Tải bài nộp, thực thi các bước chấm tự động và tổng hợp kết quả.
5	Kết quả chấm	Lưu kết quả chấm theo câu và kết quả từng bước; cung cấp dữ liệu cho bảng điểm.
6	Trợ lý AI	Sinh giải thích và gợi ý học tập cho bài nộp dựa trên kết quả chấm.

Bảng 2.12: Phân rã hệ thống thành các miền chức năng.

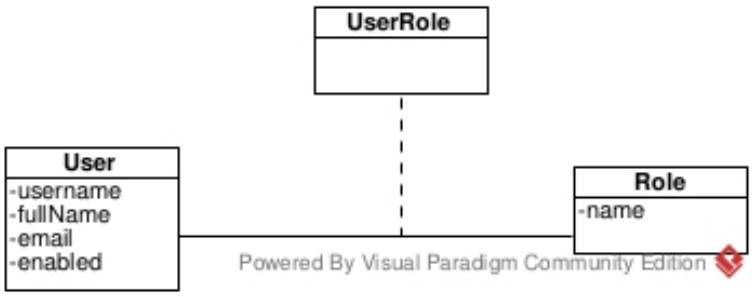
2.1.6 Biểu đồ lớp chi tiết từng dịch vụ

Sau khi phân rã, mỗi miền chức năng được thiết kế thành một dịch vụ độc lập với cơ sở dữ liệu riêng. Các mục dưới đây trình bày biểu đồ lớp chi tiết của từng dịch vụ.

Ở mức dịch vụ, kết quả bước chấm được lưu ở hai dạng: GradingStepResult là bản ghi thực thi của dịch vụ thực thi chấm, còn StepResult là bản đã chấm do dịch vụ kết quả chấm lưu; cả hai tương ứng với lớp StepResult trong mô hình toàn hệ thống ở mục 2.1.4.

a, Biểu đồ lớp pha phân tích miền Xác thực và phân quyền

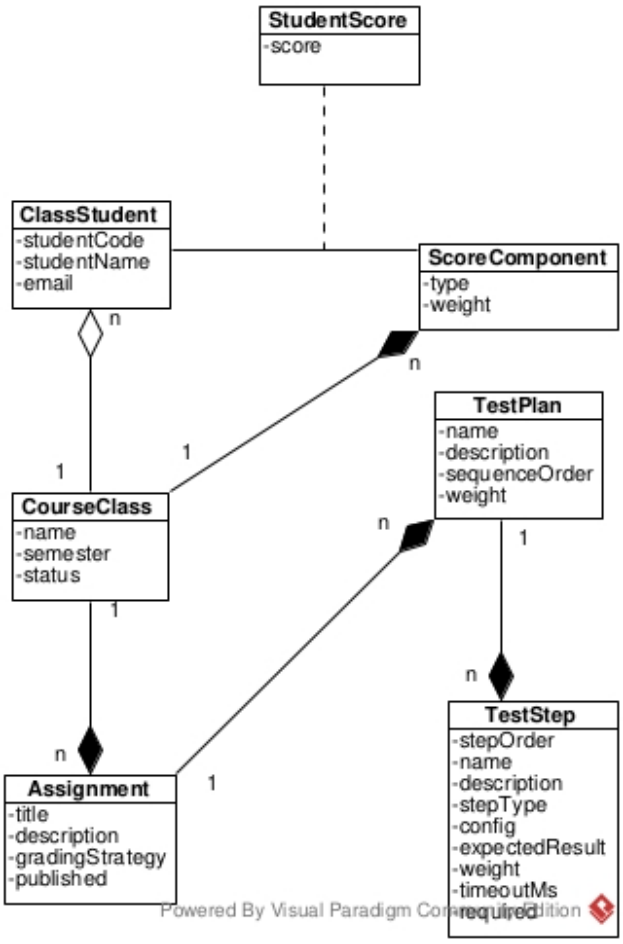
Miền Xác thực và phân quyền gồm Người dùng (User), Vai trò (Role) và lớp kết hợp Liên kết người dùng - vai trò (UserRole). Lớp UserRole biểu diễn quan hệ nhiều-nhiều giữa người dùng và vai trò, trong đó mỗi bản ghi giữ userId và roleId.



Hình 2.9: Biểu đồ lớp pha phân tích miền Xác thực và phân quyền.

b, Biểu đồ lớp pha phân tích dịch vụ quản lý lớp học và điểm

Các lớp thuộc dịch vụ quản lý lớp học và điểm gồm Lớp học (CourseClass), Sinh viên thuộc lớp (ClassStudent), Bài tập (Assignment), Câu (TestPlan), Bước chấm (TestStep), Thành phần điểm (ScoreComponent) và Điểm thành phần (StudentScore).



Hình 2.10: Biểu đồ lớp pha phân tích dịch vụ quản lý lớp học và điểm.

c, Biểu đồ lớp pha phân tích dịch vụ nộp bài

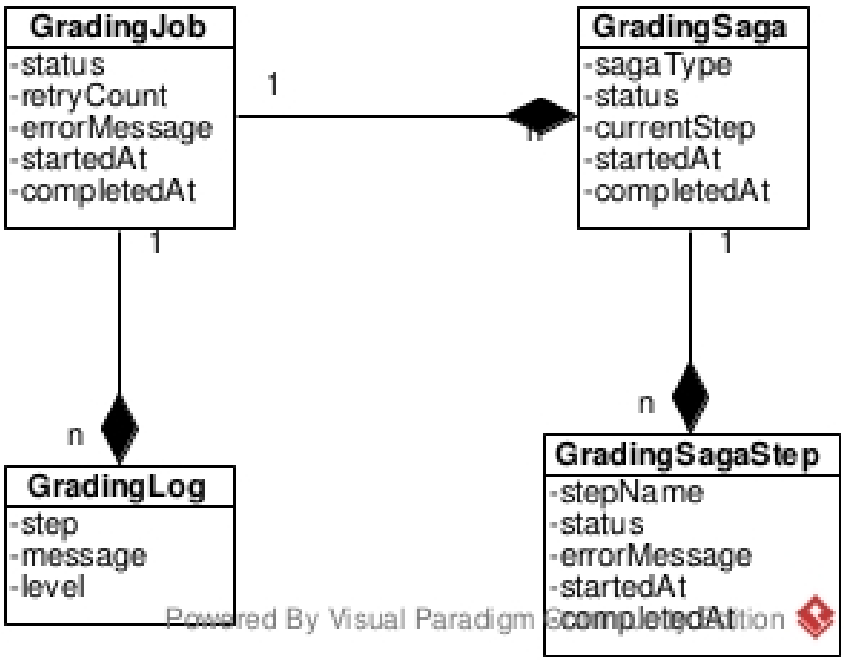
Các lớp thuộc dịch vụ nộp bài gồm Bài nộp (Submission) và thông tin tệp nộp, trạng thái chấm của bài nộp.



Hình 2.11: Biểu đồ lớp pha phân tích dịch vụ nộp bài.

d, Biểu đồ lớp pha phân tích dịch vụ thực thi chấm

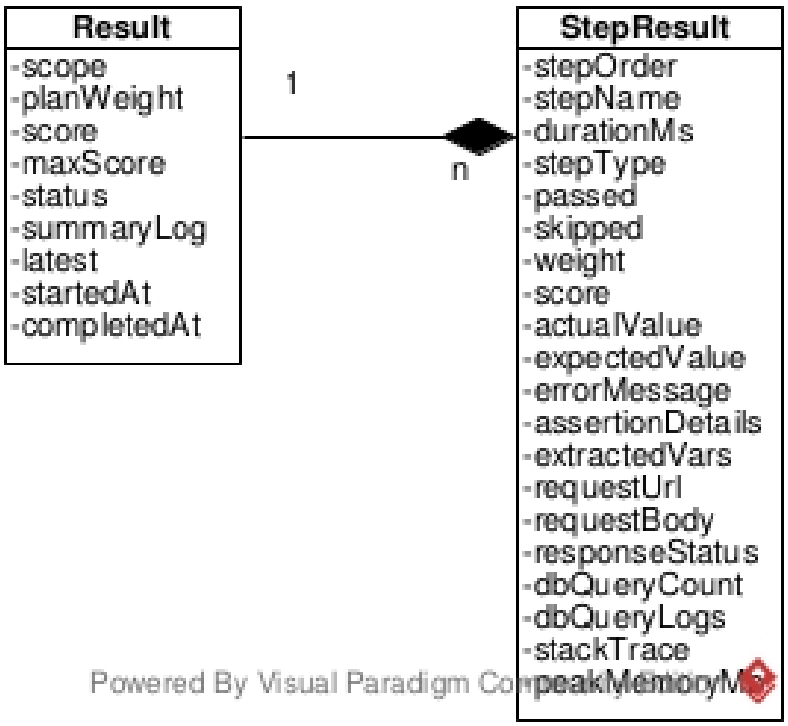
Các lớp thuộc dịch vụ thực thi chấm gồm Công việc chấm (GradingJob), Phiên chấm (GradingSaga), Bước của phiên chấm (GradingSagaStep), Kết quả bước chấm (GradingStepResult) và Nhật ký chấm (GradingLog).



Hình 2.12: Biểu đồ lớp pha phân tích dịch vụ thực thi chấm.

e, Biểu đồ lớp pha phân tích dịch vụ kết quả chấm

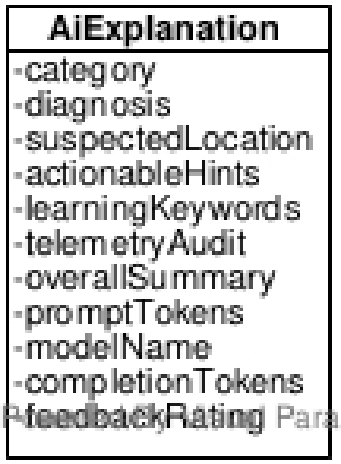
Các lớp thuộc dịch vụ kết quả chấm gồm Kết quả chấm (Result) và Kết quả từng bước (StepResult).



Hình 2.13: Biểu đồ lớp pha phân tích dịch vụ kết quả chấm.

f, Biểu đồ lớp pha phân tích dịch vụ trợ lý AI

Các lớp thuộc dịch vụ trợ lý AI gồm Giải thích của trợ lý AI (AiExplanation) cùng thông tin bài nộp và danh mục được giải thích.

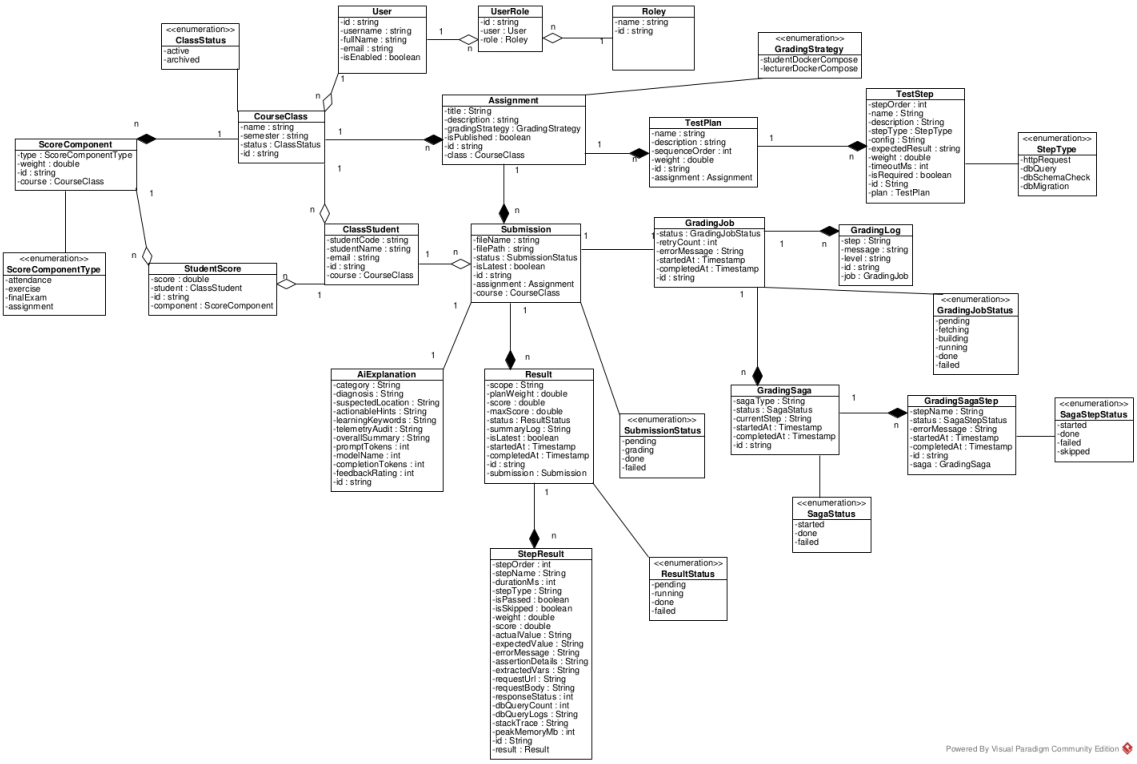


Hình 2.14: Biểu đồ lớp pha phân tích dịch vụ trợ lý AI.

2.2 Thiết kế hệ thống

2.2.1 Biểu đồ lớp thiết kế chi tiết từng dịch vụ

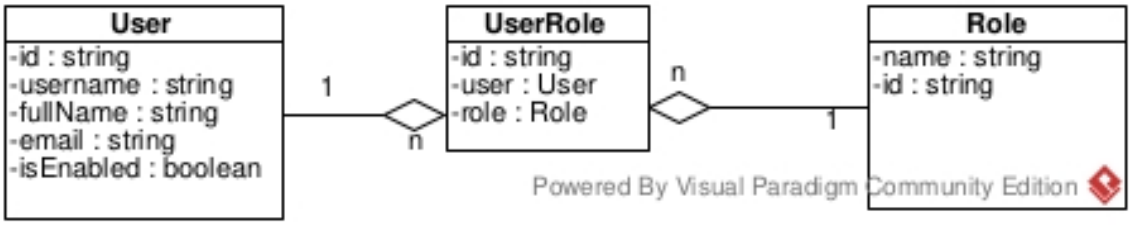
Sau khi phân rã hệ thống thành các miền chức năng, mỗi miền được thiết kế thành một dịch vụ độc lập với cơ sở dữ liệu riêng. Dưới đây là biểu đồ lớp thiết kế chi tiết của từng dịch vụ.



Hình 2.15: Biểu đồ lớp thiết kế toàn hệ thống.

a, Dịch vụ Xác thực và phân quyền

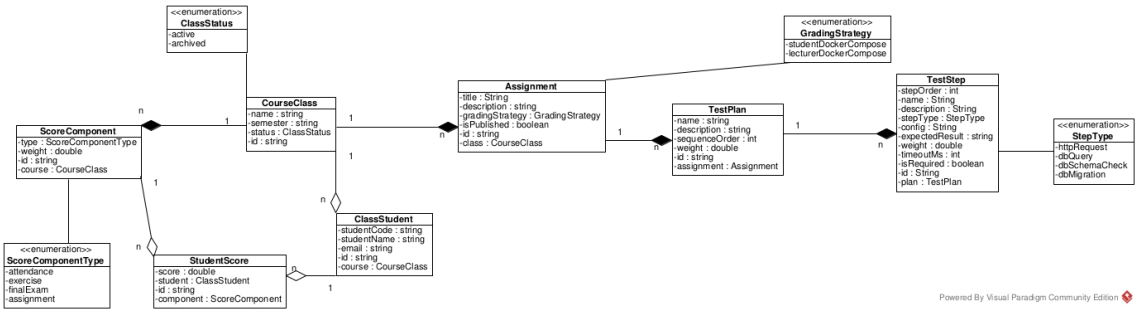
Dịch vụ xác thực và phân quyền quản lý tài khoản, đăng nhập, đổi mật khẩu, cấp và kiểm tra vai trò của người dùng trong hệ thống.



Hình 2.16: Biểu đồ lớp thiết kế dịch vụ Xác thực và phân quyền.

b, Dịch vụ Quản lý lớp học và điểm

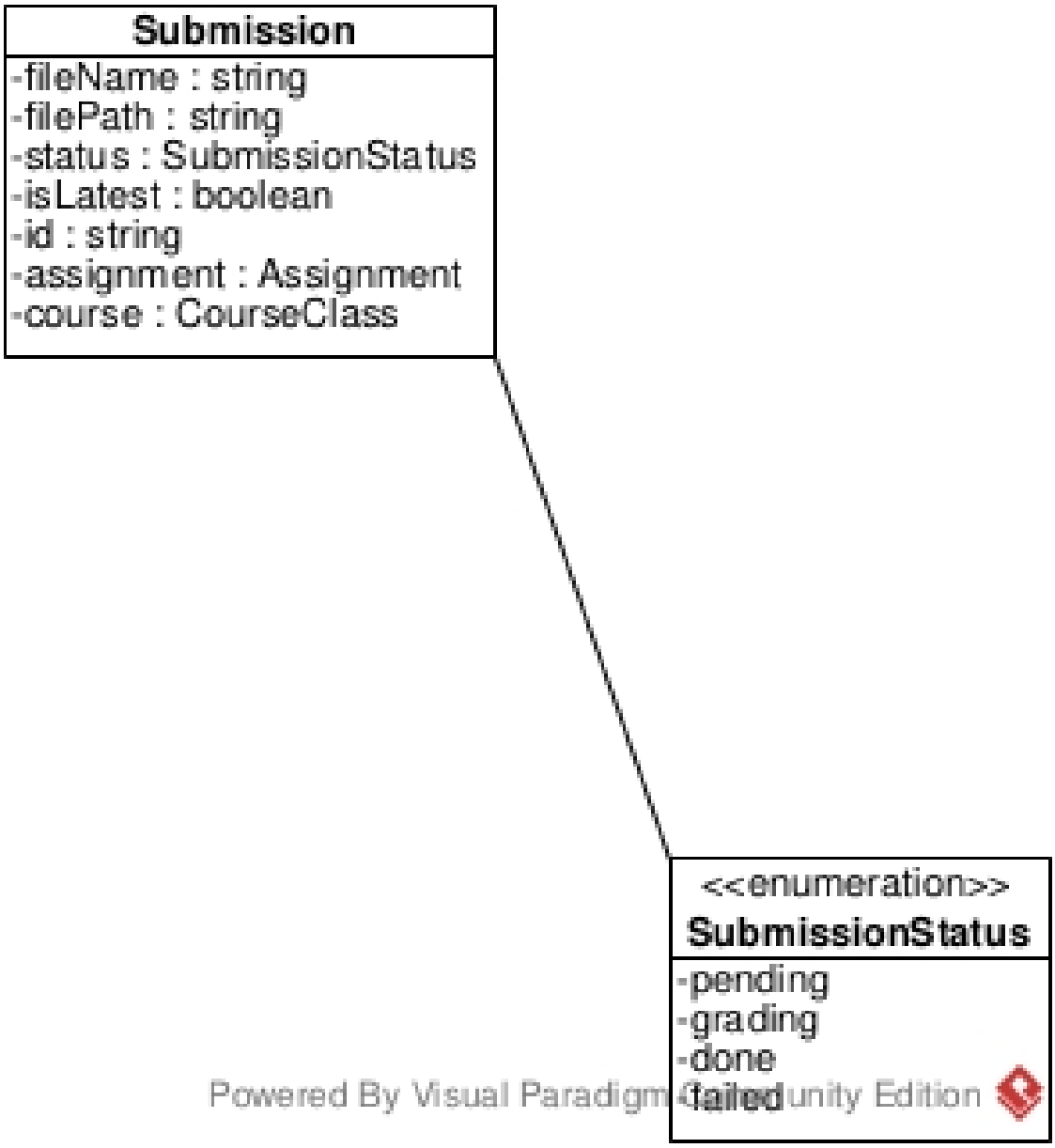
Dịch vụ quản lý lớp học và điểm quản lý lớp học, danh sách sinh viên, cấu hình thành phần điểm và điểm số của từng sinh viên.



Hình 2.17: Biểu đồ lớp thiết kế dịch vụ Quản lý lớp học và điểm.

c, Dịch vụ Nộp bài

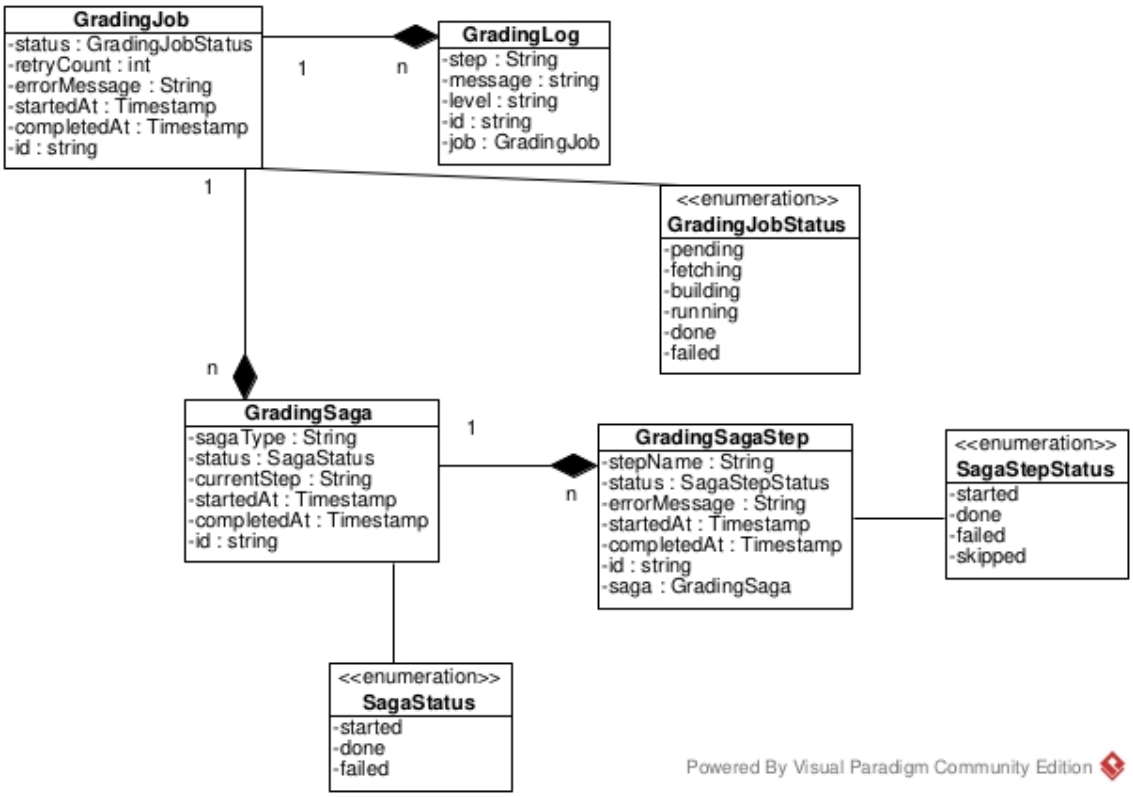
Dịch vụ nộp bài quản lý bài nộp của sinh viên và điểm chấm thủ công.



Hình 2.18: Biểu đồ lớp thiết kế dịch vụ Nộp bài.

d, Dịch vụ Thực thi chấm

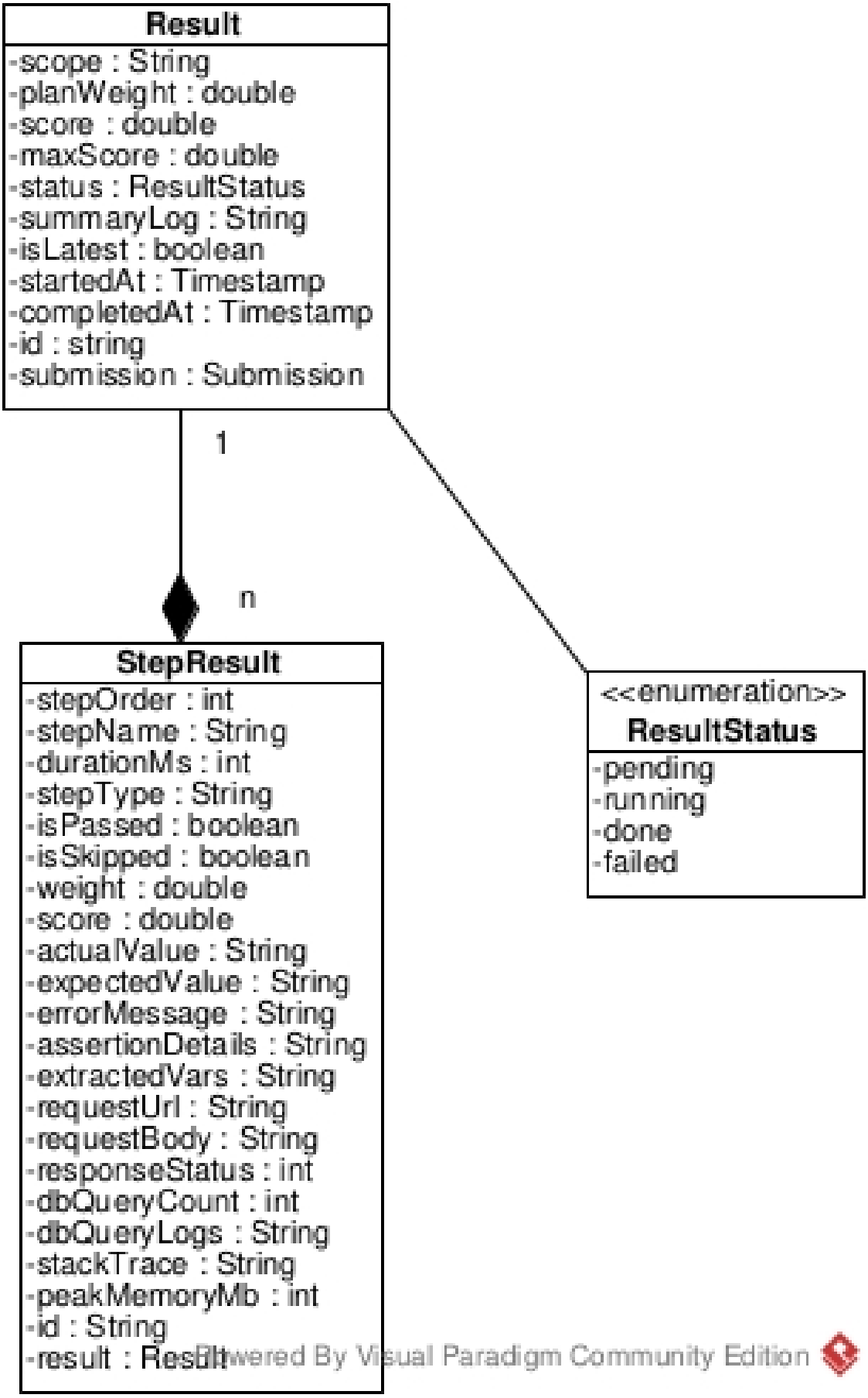
Dịch vụ thực thi chấm quản lý công việc chấm, phiên chấm, bước chấm và nhật ký thực thi.



Hình 2.19: Biểu đồ lớp thiết kế dịch vụ Thực thi chấm.

e, Dịch vụ Kết quả chấm

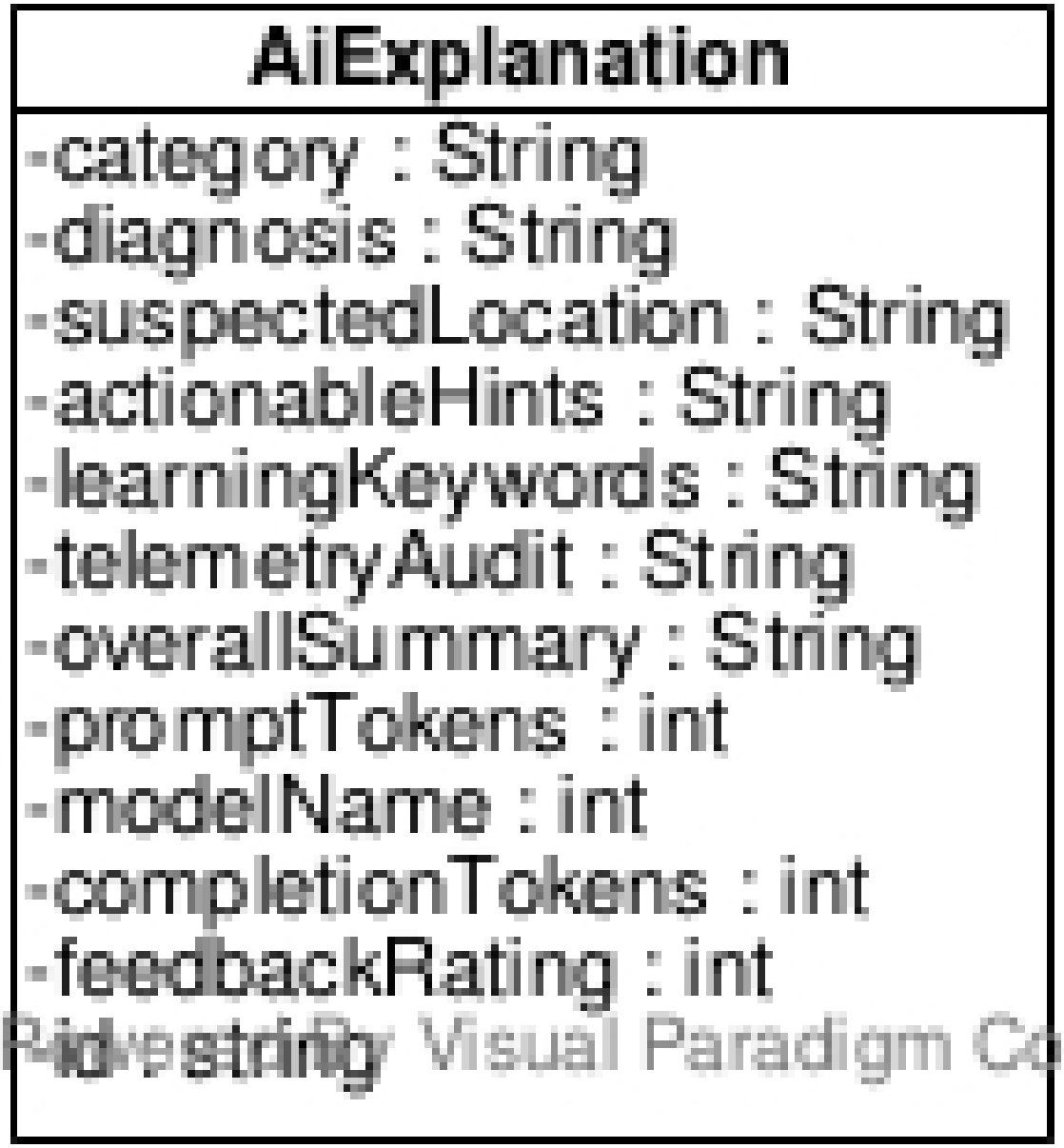
Dịch vụ kết quả chấm lưu kết quả chấm theo từng bài nộp và kết quả từng bước chấm.



Hình 2.20: Biểu đồ lớp thiết kế dịch vụ Kết quả chấm.

f, Dịch vụ Trợ lý AI

Dịch vụ trợ lý AI quản lý hội thoại, giải thích và gợi ý học tập cho bài nộp.



Hình 2.21: Biểu đồ lớp thiết kế dịch vụ Trợ lý AI.

2.2.2 Thiết kế cơ sở dữ liệu theo dịch vụ

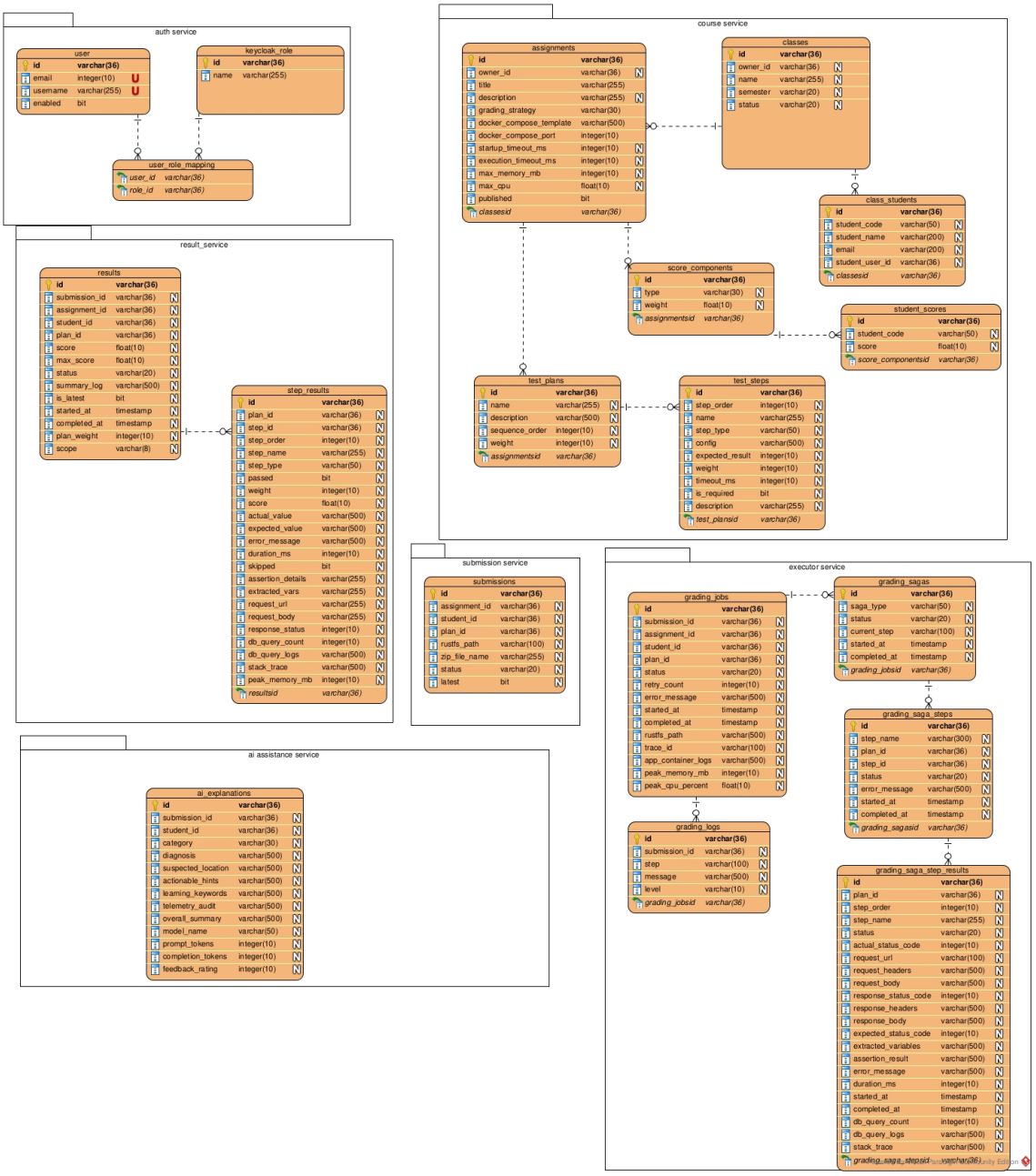
Sau khi phân rã hệ thống thành các miền chức năng, mỗi miền được thiết kế thành một dịch vụ độc lập với cơ sở dữ liệu riêng. Mỗi dịch vụ quản lý độc lập các bảng thuộc miền nghiệp vụ của mình, không truy cập trực tiếp cơ sở dữ liệu của dịch vụ khác.

- Dịch vụ xác thực và phân quyền: quản lý tài khoản, đăng nhập, đổi mật khẩu, cấp và kiểm tra vai trò của người dùng trong hệ thống.
- Dịch vụ quản lý lớp học và điểm: bao gồm các bảng classes, class_students,

score_components, student_scores, để quản lý lớp học, danh sách sinh viên, cấu hình thành phần điểm và điểm số của từng sinh viên.

- Dịch vụ nộp bài: bao gồm các bảng submissions, manual_scores, để quản lý bài nộp của sinh viên và điểm chấm thủ công.
- Dịch vụ thực thi chấm: bao gồm các bảng grading_jobs, grading_logs, grading_sagas, grading_saga_steps, grading_step_results, để quản lý công việc chấm, phiên chấm, bước chấm và nhật ký thực thi.
- Dịch vụ kết quả chấm: bao gồm các bảng results, step_results, để lưu kết quả chấm theo từng bài nộp và kết quả từng bước chấm.
- Dịch vụ trợ lý AI: bao gồm các bảng ai_conversation, ai_message, ai_explanations, ai_task, ai_run, để quản lý hội thoại, giải thích và gợi ý học tập cho bài nộp.

Biểu đồ 2.22 minh họa quan hệ giữa các thực thể chính của hệ thống.



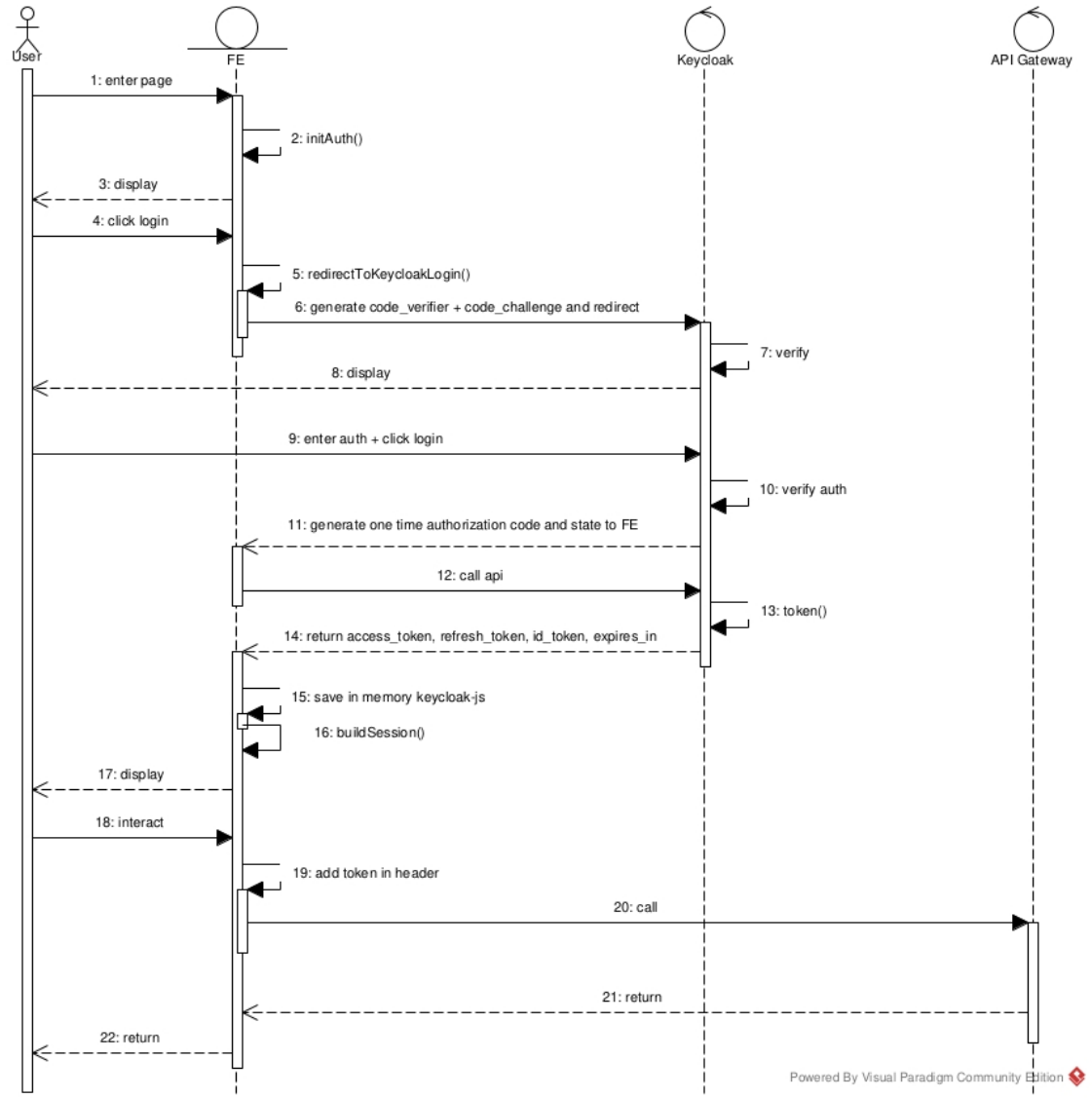
Hình 2.22: Biểu đồ quan hệ thực thể (ERD) của hệ thống.

2.2.3 Sơ đồ tuần tự các chức năng

Dưới đây là sơ đồ tuần tự mô tả luồng xử lý nghiệp vụ của từng chức năng chính trong hệ thống.

a, Đăng nhập

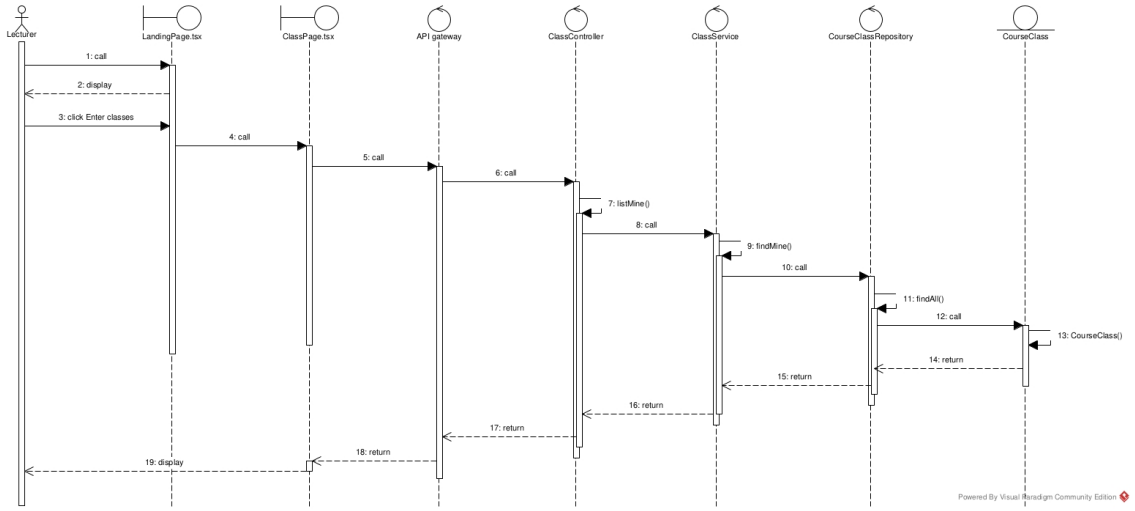
Hình 2.23 mô tả luồng xử lý khi người dùng đăng nhập vào hệ thống.



Hình 2.23: Sơ đồ tuần tự chức năng Đăng nhập.

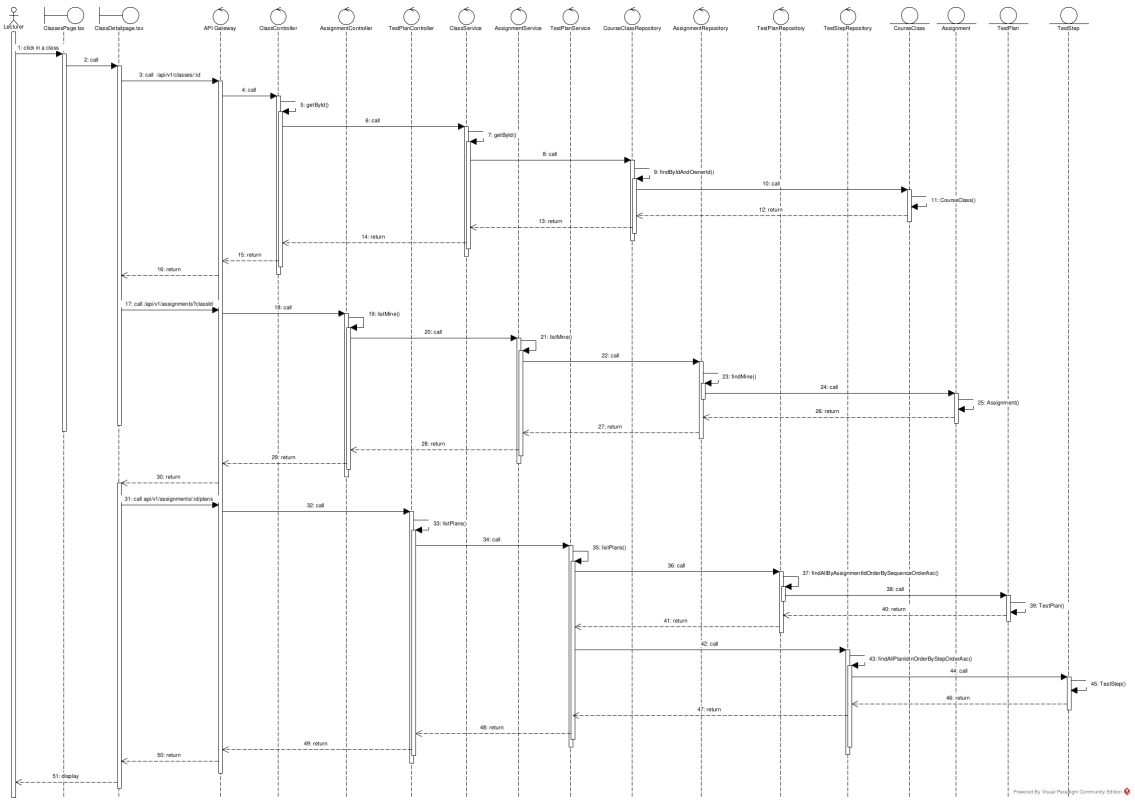
b, Quản lý lớp học

Hình 2.24 mô tả luồng xử lý khi giảng viên xem danh sách lớp học của mình.



Hình 2.24: Sơ đồ tuần tự chức năng Xem lớp.

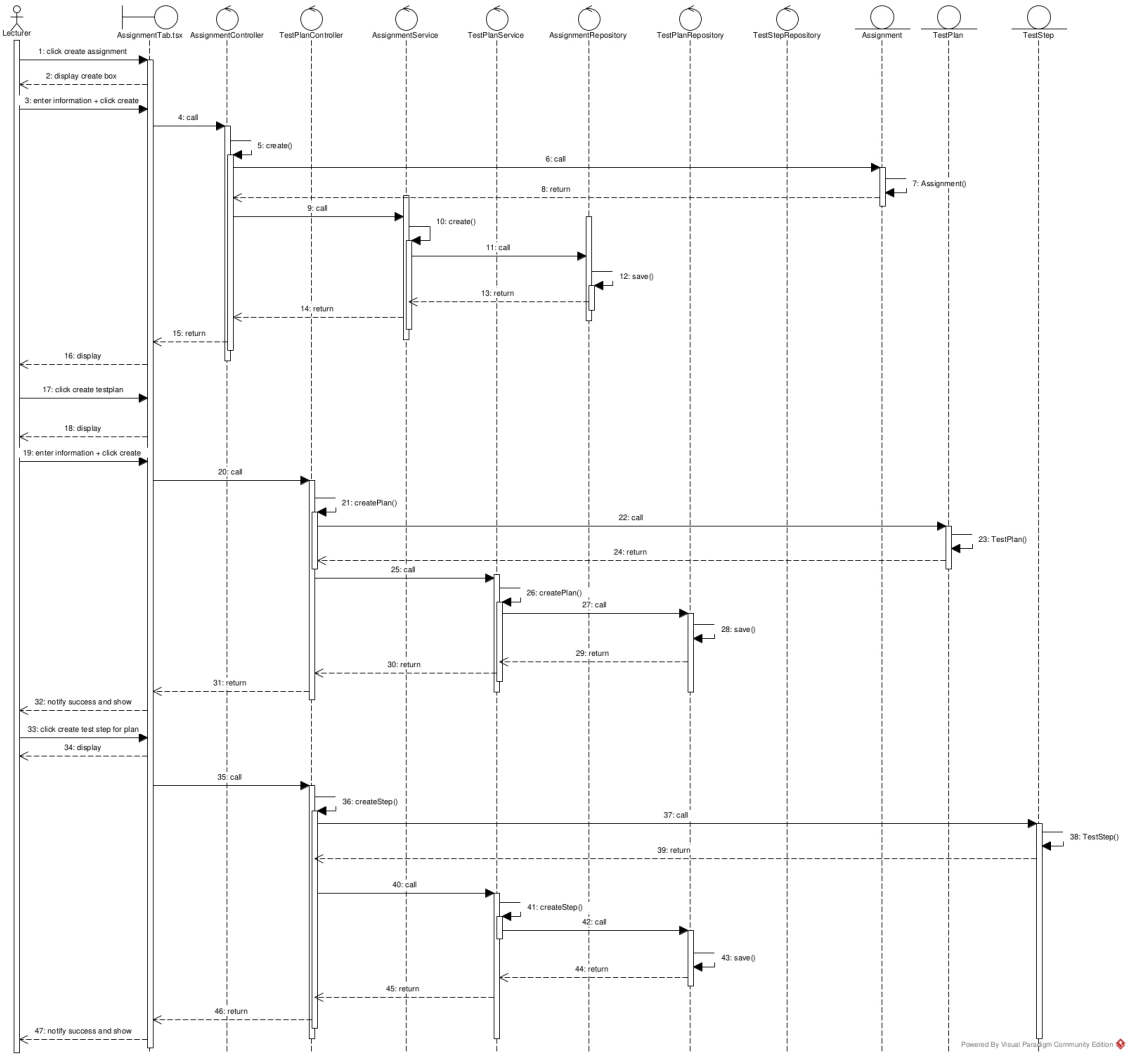
Hình 2.25 mô tả luồng xử lý khi giảng viên xem chi tiết một lớp học.



Hình 2.25: Sơ đồ tuần tự chức năng Xem chi tiết lớp.

c, Quản lý bài tập

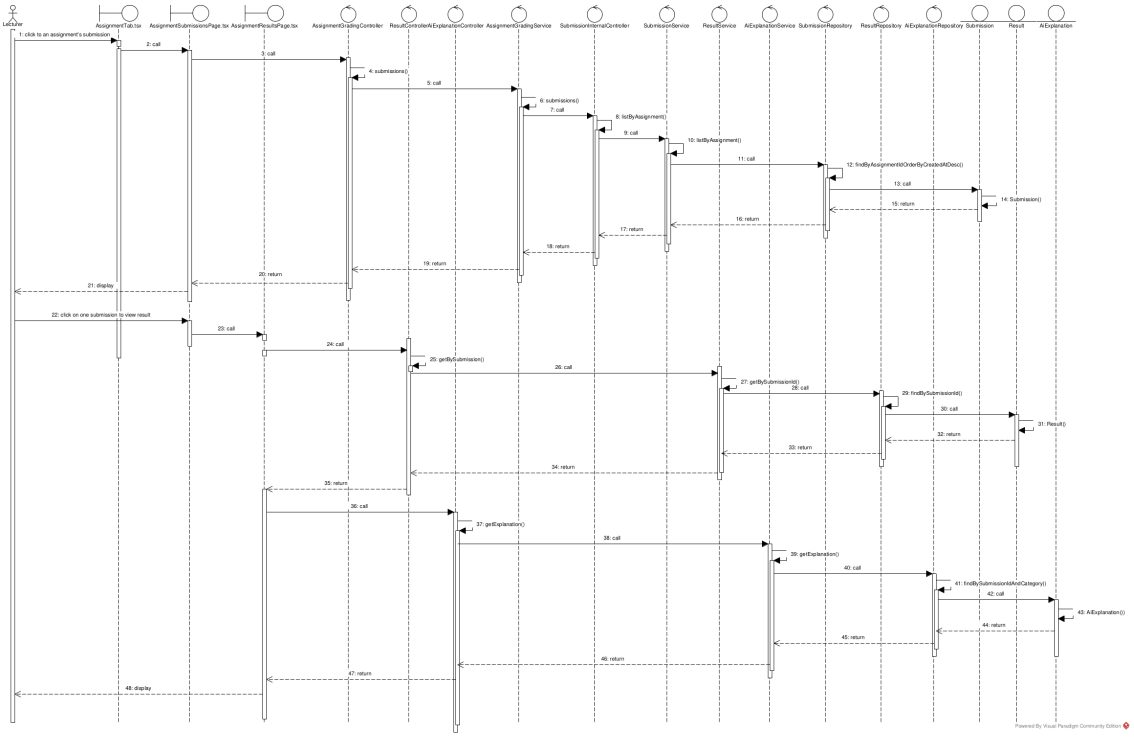
Hình 2.26 mô tả luồng xử lý khi giảng viên quản lý bài tập trong lớp.



Hình 2.26: Sơ đồ tuần tự chức năng Quản lý bài tập.

d, Xem kết quả chấm của cả lớp

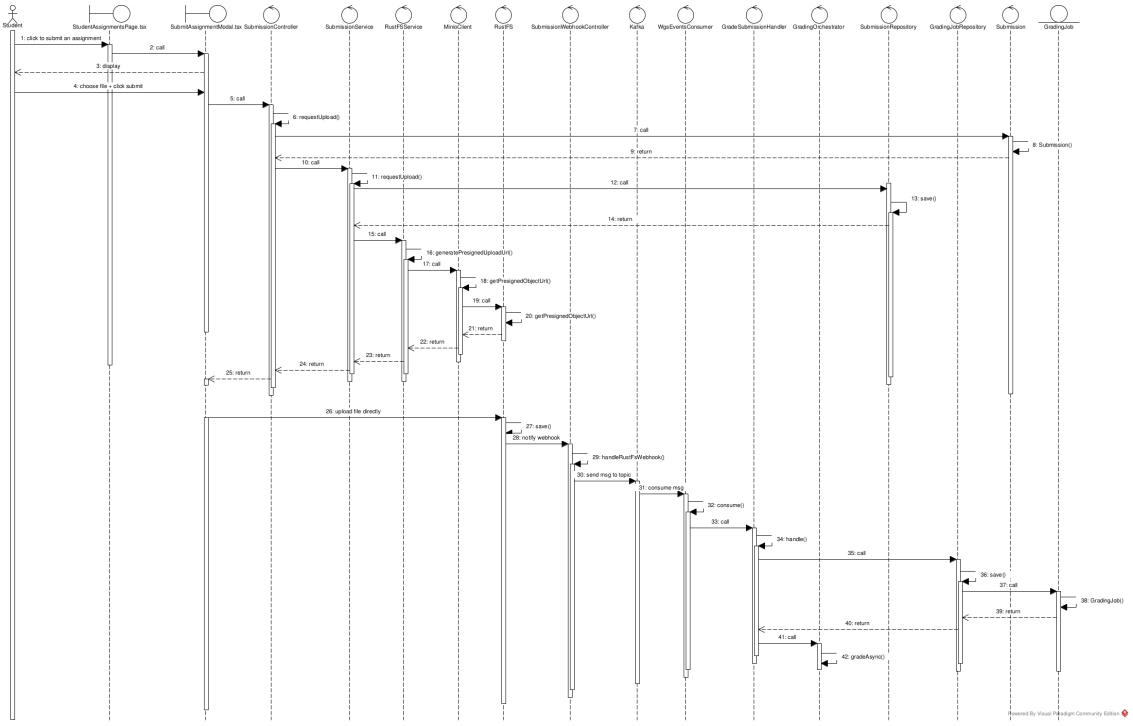
Hình 2.27 mô tả luồng xử lý khi giảng viên xem kết quả chấm bài tập của toàn bộ sinh viên trong lớp.



Hình 2.27: Sơ đồ tuần tự chức năng Xem kết quả chấm của cả lớp.

e, Nộp bài tập

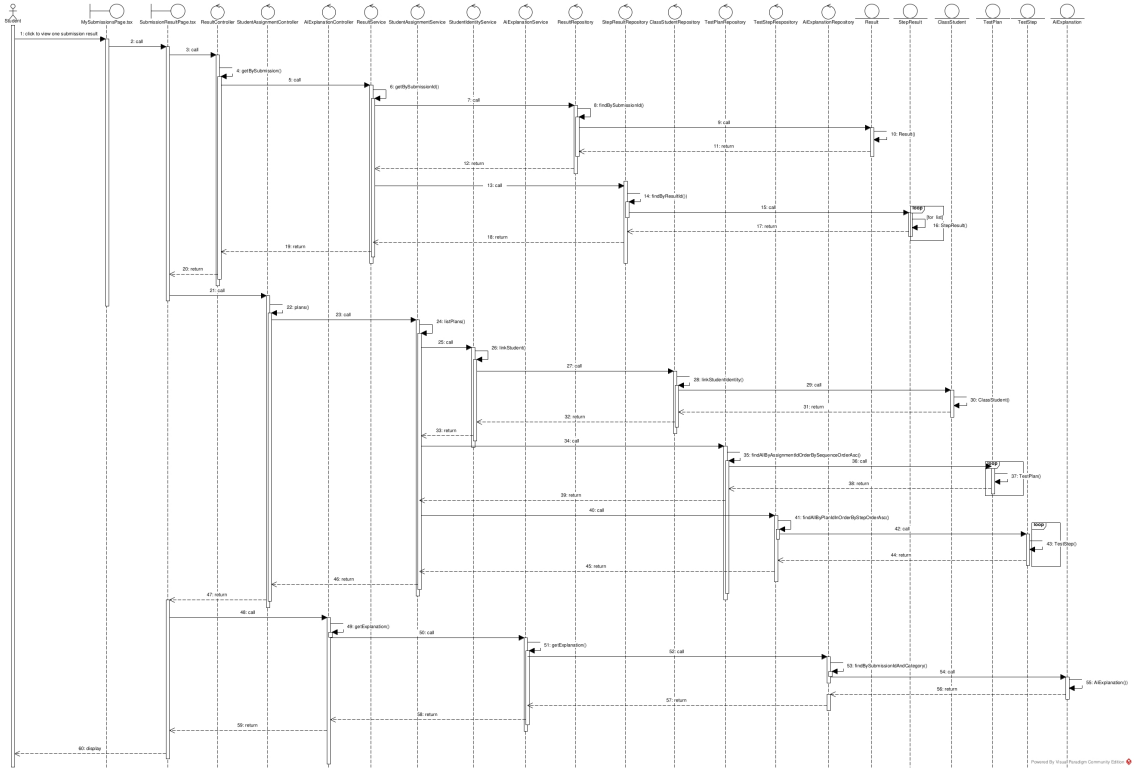
Hình 2.28 mô tả luồng xử lý khi sinh viên nộp bài tập thực hành.



Hình 2.28: Sơ đồ tuần tự chức năng Nộp bài tập.

f, Xem kết quả chấm bài tập của bản thân

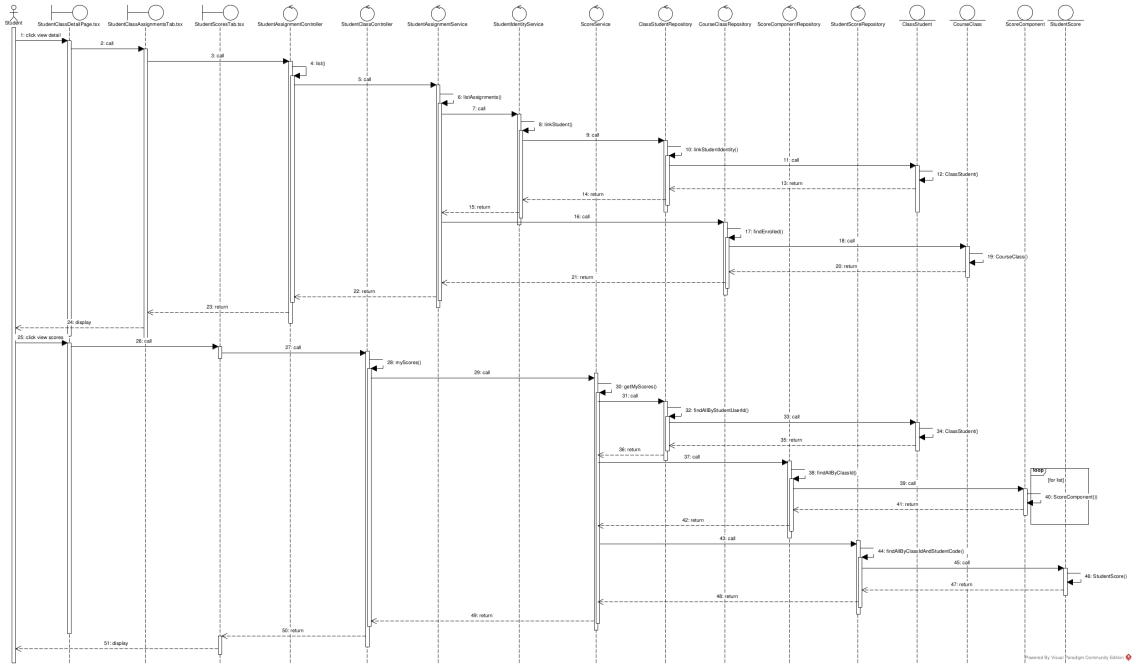
Hình 2.29 mô tả luồng xử lý khi sinh viên xem kết quả chấm của bài nộp mình đã gửi.



Hình 2.29: Sơ đồ tuần tự chức năng Xem kết quả chấm bài tập của bản thân.

g, Xem điểm và kết quả cá nhân

Hình 2.30 mô tả luồng xử lý khi sinh viên xem điểm thành phần, điểm tổng kết và bảng điểm của mình trong lớp.



Hình 2.30: Sơ đồ tuần tự chức năng Xem điểm và kết quả cá nhân.

2.2.4 Kiến trúc hệ thống

Hình 2.31 mô tả kiến trúc tổng thể của hệ thống. Hệ thống được tổ chức theo mô hình Client - Server: phía client đảm nhiệm tương tác với người dùng, phía server xử lý nghiệp vụ, quản lý dữ liệu và cung cấp dịch vụ. Cách tổ chức này tách biệt giao diện khỏi xử lý nghiệp vụ, thuận lợi cho phát triển, triển khai và mở rộng từng phần độc lập.

Hình 2.31: Kiến trúc tổng thể của hệ thống.

Phía client, giao diện được xây dựng bằng React và TypeScript, dùng Vite và bộ thành phần Ant Design, chạy trên trình duyệt của người dùng. Client gửi các yêu cầu HTTPS đến API Gateway - điểm vào duy nhất của backend. Gateway xác thực JWT do Keycloak cấp, kiểm tra vai trò rồi chuyển tiếp yêu cầu đến dịch vụ đích.

Phía backend, ngoài API Gateway, hệ thống gồm năm dịch vụ nghiệp vụ độc lập là Quản lý lớp học và điểm, Nộp bài, Thực thi chấm, Kết quả chấm và Trợ lý AI; miền Xác thực và phân quyền do Keycloak đảm nhiệm. Mỗi dịch vụ quản lý một cơ sở dữ liệu riêng và không truy cập trực tiếp dữ liệu của dịch vụ khác. Cách phân rã này giới hạn phạm vi ảnh hưởng khi một dịch vụ gặp sự cố, đồng thời cho phép build, triển khai và mở rộng từng dịch vụ mà không ảnh hưởng đến phần còn lại.

Các dịch vụ giao tiếp với nhau qua hai cơ chế. Giao tiếp đồng bộ dùng REST thông qua OpenFeign cho các yêu cầu cần phản hồi ngay, chẳng hạn dịch vụ Thực

thi chấm lấy cấu hình bài tập từ dịch vụ Quản lý lớp học và điểm, cập nhật trạng thái bài nộp hoặc ghi kết quả sang dịch vụ Kết quả chấm. Giao tiếp bất đồng bộ dùng Kafka với một topic dùng chung `wgs-events`: mỗi message mang envelope gồm action, traceId và payload, được khóa theo submissionId để giữ thứ tự xử lý trên từng bài nộp. Khi bài nộp hoàn tất, dịch vụ Nộp bài phát sự kiện `GRADE_SUBMISSION`; dịch vụ Thực thi chấm tiêu thụ sự kiện, thực thi các bước chấm rồi ghi kết quả. Nhờ đó, quá trình chấm không chặn thao tác của người dùng và vẫn tiếp tục được khi một dịch vụ tạm thời quá tải.

Tệp bài nộp được lưu trong RustFS - hệ thống lưu trữ đối tượng tương thích S3 - thay vì lưu trong cơ sở dữ liệu. Client tải tệp trực tiếp lên RustFS bằng presigned URL do dịch vụ Nộp bài cấp; sau khi tải xong, RustFS gọi webhook báo hoàn tất để dịch vụ Nộp bài cập nhật trạng thái và phát sự kiện chấm. Cách này giảm tải cho dịch vụ và tránh truyền tệp lớn qua gateway.

Dịch vụ Trợ lý AI gọi mô hình ngôn ngữ lớn qua giao diện tương thích OpenAI; địa chỉ và tên mô hình được cấu hình bằng biến môi trường nên có thể thay nhà cung cấp mà không sửa mã nguồn. Hội thoại, giải thích và gợi ý học tập được lưu trong cơ sở dữ liệu riêng của dịch vụ.

Về xác thực, Keycloak đóng vai trò Authorization Server. Người dùng đăng nhập trên giao diện do Keycloak quản lý theo luồng Authorization Code kèm PKCE; access token dạng JWT được gửi kèm mỗi yêu cầu. API Gateway kiểm tra chữ ký và thời hạn token, trích xuất danh tính và vai trò rồi chuyển tiếp trong các header nội bộ. Các dịch vụ phía sau không được giao diện gọi trực tiếp; ngoài JWT, gateway còn gửi kèm một mã bí mật dùng chung để dịch vụ nội bộ từ chối các yêu cầu không đi qua gateway.

Đối với tính nhất quán, mỗi bài nộp có một bản ghi công việc trong bảng `grading_jobs`; tiến trình chấm nhiều bước được ghi nhận trong `grading_sagas` và `grading_saga_steps` để có thể theo dõi, chạy lại hoặc đặt lại khi một bước thất bại. Cơ chế này không thay thế giao dịch phân tán, nhưng bảo đảm mỗi bài nộp đều có trạng thái cuối cùng rõ ràng và không bị kẹt khi một bước chấm gặp lỗi.

Toàn bộ dịch vụ backend được đóng gói thành container và triển khai trên cụm Kubernetes (k3s) theo mô hình GitOps; phần này được trình bày ở chương 3.

CHƯƠNG 3. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Chương này trình bày cách hệ thống được đóng gói, triển khai trên cụm Kubernetes và vận hành với các dịch vụ hạ tầng dùng chung; sau đó trình bày các tiêu chí, kịch bản đánh giá và những hạn chế còn lại. Nội dung được tổ chức thành ba phần chính: triển khai hệ thống (mục 3.1), đánh giá hệ thống (mục 3.2) và hướng phát triển (mục 3.3).

3.1 Triển khai hệ thống

Mục này trình bày môi trường triển khai, cách đóng gói dịch vụ, quy trình đưa mã nguồn lên cụm và cơ chế bảo đảm nhất quán khi chấm bài.

3.1.1 Môi trường và kiến trúc triển khai

Hệ thống được triển khai theo mô hình lai. Giao diện web chạy trên Vercel, các dịch vụ backend chạy trong cụm k3s [17], còn PostgreSQL và Kafka được thuê dưới dạng dịch vụ quản lý. Cách chia này giữ phần tính toán trong cụm để chủ động cấu hình, đồng thời chuyển việc sao lưu, vá lỗi và mở rộng dung lượng của tầng dữ liệu cho nhà cung cấp.

Luồng truy cập từ Internet đi qua Cloudflare Zero Trust Tunnel [33]. Cloudflare kết thúc kết nối TLS, chuyển tiếp yêu cầu vào cụm qua một kết nối do phía trong khởi tạo; nhờ đó không cần mở cổng vào trực tiếp trên firewall của máy chủ. Trong cụm, Traefik nhận yêu cầu và định tuyến theo tên miền tới Service tương ứng, sau đó Service chuyển tiếp tới các Pod. Hình 3.1 mô tả kiến trúc triển khai tổng thể.

Hình 3.1: Kiến trúc triển khai của hệ thống trên cụm k3s.

Bảng 3.1 liệt kê các thành phần chính, cổng, cơ sở dữ liệu và điểm truy cập. Các tên miền trong bảng dùng chung hậu tố vucongtuanduong.dpdns.org; cột điểm truy cập chỉ ghi phần tên miền đứng trước hậu tố này.

Thành phần	Cổng	Cơ sở dữ liệu	Bản sao	Điểm truy cập
Giao diện web	Không	Không	Không	Vercel, web-dev1-fe
API Gateway	8080	Không	1	web-dev1-api
Dịch vụ quản lý lớp học và điểm	8081	assignment_db	1	web-dev1-course
Dịch vụ nộp bài	8082	submission_db	1	web-dev1-submission
Dịch vụ thực thi chấm	8083	executor_db	2	web-dev1-grading
Dịch vụ kết quả chấm	8084	result_db	1	web-dev1-result
Dịch vụ trợ lý AI	8086	ai_db	1	web-dev1-ai
Keycloak	8080	Cơ sở dữ liệu quản lý	1	web-dev1-keycloak
RustFS	9000, 9001	PVC 10Gi	1	web-dev1-rustfs-api
Kafka UI	8080	Không	1	web-dev1-kafka-ui

Bảng 3.1: Các thành phần chính của môi trường triển khai.

Hệ thống dùng hai namespace chính: web-grading chứa ứng dụng và argocd chứa ArgoCD. Các Service đều thuộc kiểu ClusterIP; truy cập từ ngoài đi qua Ingress với ingressClassName là traefik. Mỗi tên miền trỏ tới một Service, còn Service chọn Pod theo nhãn ứng dụng. Cách tổ chức này giữ địa chỉ nội bộ ổn định khi Pod được tạo lại.

Giao diện web được triển khai trên Vercel [34]. Tệp vercel.json khai báo rewrite mọi yêu cầu /api/:path* sang API Gateway, nhờ đó trình duyệt chỉ gọi cùng một origin và không phát sinh yêu cầu CORS cho các API nghiệp vụ. Giao diện cũng dùng rewrite toàn bộ đường dẫn còn lại về index.html để React Router xử lý điều hướng phía client. Keycloak được cấu hình tên miền công khai và danh sách redirect URI tương ứng với miền của giao diện.

Phạm vi báo cáo không trình bày hệ thống giám sát tập trung; nội dung này được nêu ở mục 3.3 như một hướng phát triển.

3.1.2 Đóng gói và cấu hình triển khai

Mỗi dịch vụ backend có một Dockerfile riêng theo mẫu build nhiều tầng [15]. Tầng thứ nhất dùng image eclipse-temurin:21-jdk-alpine, sao chép Maven wrapper, tệp pom.xml và mã nguồn rồi chạy mvn clean package để tạo tệp JAR. Tầng thứ hai dùng image eclipse-temurin:21-jre, chỉ chứa JRE và tệp JAR, sau đó chạy bằng lệnh java -jar. Cách đóng gói này giữ image chạy nhỏ hơn image build, không chứa mã nguồn và công cụ build, đồng thời bảo đảm mọi môi trường chạy cùng một phiên bản Java 21.

Toàn bộ cấu hình được truyền qua biến môi trường thay vì ghi vào image. Các giá trị nhạy cảm như thông tin kết nối cơ sở dữ liệu, khóa Kafka và khóa RustFS

được lấy từ Secret của Kubernetes; các giá trị không nhạy cảm như tên cơ sở dữ liệu, cổng dịch vụ và mức tài nguyên được khai báo trong tệp values của Helm. Nhờ vậy, cùng một image có thể chạy ở nhiều môi trường mà không cần build lại.

Dịch vụ thực thi chấm là trường hợp đặc biệt vì cần Docker để dựng môi trường chấm của bài nộp. Mỗi Pod của dịch vụ này gồm hai container: container ứng dụng và container docker:27-dind chạy Docker daemon riêng. Container ứng dụng đặt biến DOCKER_HOST trỏ tới tcp://localhost:2375; container DinD chạy ở chế độ privileged, tắt TLS nội bộ và dùng một volume emptyDir chung tại /var/lib/docker. Mỗi Pod có Docker daemon riêng nên các bài chấm không dùng chung daemon; volume là tạm thời nên dữ liệu Docker không tồn tại sau khi Pod bị thay thế. Dịch vụ này chạy hai bản sao để tăng khả năng xử lý đồng thời, trong khi các dịch vụ còn lại chạy một bản sao.

Cấu hình triển khai được đóng gói bằng Helm [18]. Kho cấu hình chứa một chart cho mỗi dịch vụ, gồm Deployment và Service; các Ingress được quản lý riêng trong kho triển khai. Tệp values-stg.yaml khai báo image, tag, profile Spring, tên cơ sở dữ liệu, cổng, tài nguyên và số bản sao. Tài nguyên mặc định của mỗi container là 200m CPU và 256Mi bộ nhớ cho yêu cầu, 500m CPU và 512Mi cho giới hạn; container DinD có yêu cầu và giới hạn riêng. Tên tài nguyên được sinh từ tên release, mà ArgoCD đặt theo tên Application, nên các Service có tiền tố grading- ghép với tên dịch vụ.

Phần lớn chart dùng endpoint /actuator/health cho readiness và liveness, với thời gian chờ khởi động lần lượt 30 giây và 60 giây. Riêng dịch vụ trợ lý AI dùng ba probe riêng biệt trỏ tới các nhóm /actuator/health/readiness và /actuator/health/liveness. Lý do là endpoint /actuator/health gốc chạy cả chỉ báo cơ sở dữ liệu; khi compute của PostgreSQL được đánh thức từ trạng thái tạm dừng, chỉ báo này có thể giữ probe lâu hơn thời gian chờ mặc định và khiến Pod bị khởi động lại. Các nhóm health chỉ kiểm tra trạng thái nội bộ của ứng dụng nên trả lời nhanh và không phụ thuộc cơ sở dữ liệu.

3.1.3 Tích hợp liên tục với GitHub Actions

Mã nguồn backend nằm trong một kho mono-repo gồm sáu dịch vụ. Quy trình tích hợp liên tục dùng GitHub Actions [35] với hai workflow: workflow thứ nhất build và kiểm thử, workflow thứ hai phát hành image và cập nhật cấu hình.

Workflow build chạy khi có push lên các nhánh main, develop hoặc nhánh có tên chứa feat, và khi có pull request vào main hoặc develop. Bước phát hiện thay đổi dùng dorny/paths-filter để xác định dịch vụ nào bị ảnh hưởng, sau đó tạo ma trận build chỉ gồm các dịch vụ đó. Mỗi job trong ma trận dùng JDK 21 Temurin,

bật cache Maven, chạy mvn clean package -DskipTests để đóng gói nhanh, rồi chạy kiểm thử bằng Maven. Các lớp kiểm thử có tên kết thúc bằng ApplicationTests bị loại trừ vì chúng cần hạ tầng bên ngoài mới khởi động được; phần kiểm thử còn lại bao gồm các kiểm thử dùng Testcontainers cho PostgreSQL và MySQL. Danh sách dịch vụ thay đổi được lưu thành artifact changed-services.json để workflow phát hành sử dụng lại.

Workflow phát hành chạy sau khi workflow build hoàn tất trên nhánh main và chỉ tiếp tục khi kết quả thành công. Workflow tải artifact chứa danh sách dịch vụ, xác định phiên bản kế tiếp từ các tag có dạng <tên dịch vụ>-vX.Y.Z, tăng số patch và tạo GitHub Release kèm tệp JAR. Sau đó workflow đăng nhập Docker Hub và build, push image với hai tag: tag phiên bản và tag latest. Job cuối cùng clone kho cấu hình, thay tag image trong values-stg.yaml của từng dịch vụ, commit và push thay đổi. Hình 3.2 mô tả luồng từ mã nguồn đến cụm triển khai.

Hình 3.2: Quy trình CI/CD từ mã nguồn đến cụm triển khai.

Giai đoạn	Công việc chính	Kết quả
Phát hiện thay đổi	Lọc đường dẫn, tạo ma trận dịch vụ	Danh sách dịch vụ và artifact
Build và kiểm thử	JDK 21, Maven, đóng gói JAR, chạy kiểm thử	JAR và kết quả kiểm thử
Phát hành	Tính phiên bản, tạo GitHub Release, build và push image	Image theo phiên bản và latest
Cập nhật cấu hình	Clone kho cấu hình, thay tag, commit, push	Commit cấu hình mới
Đồng bộ	ArgoCD phát hiện thay đổi trong kho cấu hình	Pod dùng image mới

Bảng 3.2: Các giai đoạn của quy trình CI/CD.

Cách tách build và publish giúp một commit chỉ tạo ra một bộ image tương ứng, đồng thời giữ lịch sử phát hành trong GitHub Release. Việc cập nhật kho cấu hình bằng một commit riêng cũng tạo ra điểm kiểm tra rõ ràng: nếu image mới gây lỗi, có thể quay lại commit cấu hình trước đó mà không cần sửa mã nguồn.

3.1.4 Triển khai liên tục với ArgoCD

ArgoCD được cài trong namespace argocd từ manifest chính thức [19]. Máy chủ ArgoCD chạy sau Traefik và Cloudflare, với TLS được kết thúc ở Cloudflare; Service của ArgoCD giữ kiểu ClusterIP và chỉ Ingress mới mở đường truy cập. Cách cấu hình này tránh phải mở thêm cổng và giữ toàn bộ giao diện quản trị trong cùng một luồng truy cập đã có.

Kho cấu hình chứa một chart Helm cho mỗi dịch vụ và một tệp services.env liệt kê sáu dịch vụ cần triển khai. Script sinh Application đọc danh sách này, thay tên dịch vụ vào một template chung và tạo một Application cho mỗi dịch vụ. Mỗi Application trỏ tới kho cấu hình, theo dõi nhánh main, dùng chart tại thư mục của dịch vụ, nạp tệp values-stg.yaml, triển khai vào namespace web-grading và bật đồng bộ tự động với tùy chọn prune và selfHeal. Tùy chọn CreateNamespace=false buộc namespace phải được tạo trước, tránh việc ArgoCD tự tạo namespace ngoài ý muốn.

Luồng triển khai bắt đầu khi workflow phát hành cập nhật tag image trong kho cấu hình. ArgoCD phát hiện commit mới, render chart với values tương ứng rồi so sánh với trạng thái thực tế trong cụm; nếu có khác biệt, ArgoCD áp dụng thay đổi để đưa cụm về đúng mô tả trong Git. Cơ chế selfHeal tự sửa các thay đổi thủ công trên cụm, còn prune xóa tài nguyên không còn trong chart. Khi cần quay lại phiên bản trước, có thể đảo commit cấu hình hoặc chỉnh tag về phiên bản cũ; ArgoCD sẽ đồng bộ lại theo Git thay vì phải thao tác trực tiếp trên cụm.

Thứ tự khởi tạo môi trường được giữ cố định: tạo namespace và Secret, triển khai RustFS, áp dụng Ingress, cài ArgoCD, sau đó tạo các Application. Các bước này được đóng gói trong các script riêng để có thể chạy lại khi dựng môi trường mới. Cách làm này phù hợp với mô hình GitOps: Git là nguồn mô tả trạng thái mong muốn, còn cụm chỉ là nơi thực thi trạng thái đó.

3.1.5 Hạ tầng dùng chung

Mỗi dịch vụ có một cơ sở dữ liệu PostgreSQL riêng trên nền tảng quản lý Neon [36]; tên từng cơ sở dữ liệu được liệt kê trong Bảng 3.1. Dịch vụ API Gateway không cần cơ sở dữ liệu. Các dịch vụ có cơ sở dữ liệu kết nối qua JDBC, thông tin kết nối được lấy từ Secret db-secret. Mỗi dịch vụ có cơ sở dữ liệu chạy Flyway khi khởi động để áp dụng migration, còn Hibernate được đặt ở chế độ validate để chỉ kiểm tra schema, không tự sửa cấu trúc bảng. Cách này giữ migration trong mã nguồn và tránh hai phiên bản dịch vụ cùng tự thay đổi schema.

Kafka được thuê trên Aiven [37]. Gói miễn phí giới hạn số topic nên hệ thống dùng một topic chung là wgs-events cho toàn bộ sự kiện. Mỗi message có envelope gồm action, traceId và payload; producer gửi với khóa là mã bài nộp để các sự kiện của cùng một bài nộp đi vào cùng partition và giữ thứ tự. Dịch vụ nộp bài là nơi phát sự kiện GRADE_SUBMISSION; dịch vụ thực thi chấm tiêu thụ với consumer group executor-group, đặt số bản ghi tối đa mỗi lần lấy bằng 1, bật auto-commit và dùng chế độ ack theo từng bản ghi. Kết nối dùng SASL_SSL với cơ chế SCRAM-SHA-256; chứng chỉ CA được đóng gói trong Secret và mount vào container.

RustFS là hệ thống lưu trữ đối tượng tương thích S3 [12], [13], chạy trong cụm với một PVC dung lượng 10Gi và chế độ truy cập ReadWriteOnce. RustFS mở hai cổng: 9000 cho API S3 và 9001 cho giao diện quản trị. Dịch vụ nộp bài cấp presigned URL để sinh viên tải tệp trực tiếp lên RustFS, sau đó RustFS gọi webhook báo hoàn tất tới dịch vụ nộp bài; dịch vụ này cập nhật trạng thái bài nộp và phát sự kiện chấm. Cách làm này tránh truyền tệp lớn qua gateway và giảm tải cho dịch vụ nộp bài.

Keycloak phiên bản 26.0 chạy trong cụm, dùng PostgreSQL quản lý làm kho dữ liệu và được cấu hình tên miền công khai, chế độ proxy edge và hostname strict. Realm ptit-wgs chứa các client cho giao diện web, dịch vụ người dùng và luồng kiểm tra mật khẩu; client web-grading-fe là public client dùng PKCE với phương thức S256, còn các client máy-máy dùng client credentials. Keycloak cấp JWT để API Gateway kiểm tra chữ ký, thời hạn và vai trò trước khi chuyển tiếp yêu cầu.

Ngoài các thành phần trên, Kafka UI dùng để xem topic, partition và tiến độ consumer trong quá trình vận hành. Dịch vụ trợ lý AI dùng thêm Secret ai-credentials để giữ khóa API, tên mô hình và địa chỉ nhà cung cấp mô hình. Các Secret được tạo từ tệp môi trường cục bộ và không được commit vào kho mã nguồn. Bảng 3.3 tóm tắt các Secret chính.

Secret	Nội dung	Mục đích
db-secret	Máy chủ, cổng, tên đăng nhập và mật khẩu PostgreSQL	Kết nối cơ sở dữ liệu của từng dịch vụ
kafka-aiven-credentials	Địa chỉ bootstrap, tài khoản, mật khẩu và chứng chỉ CA	Kết nối Kafka qua SASL_SSL
rustfs-secret	Endpoint nội bộ, endpoint công khai, access key và secret key	Truy cập RustFS và tạo presigned URL
gateway-trust	Khóa bí mật dùng chung	Xác thực yêu cầu nội bộ gửi từ gateway
keycloak-db	Thông tin kết nối cơ sở dữ liệu và tài khoản quản trị Keycloak	Khởi tạo và kết nối Keycloak
ai-credentials	Khóa API, tên mô hình và địa chỉ nhà cung cấp mô hình	Gọi nhà cung cấp mô hình cho trợ lý AI
app-config	Mức log của ứng dụng	Điều chỉnh mức ghi log khi vận hành

Bảng 3.3: Các Secret cấu hình cho cụm triển khai.

Khóa gateway-trust được sinh một lần và tái sử dụng giữa các lần triển khai. Việc tạo khóa mới mà không khởi động lại toàn bộ dịch vụ sẽ khiến các Pod đang chạy giữ khóa cũ và từ chối yêu cầu, nên script tạo Secret ưu tiên đọc giá trị đang có trong cụm thay vì sinh lại.

3.1.6 Nhất quán dữ liệu trong quy trình chấm bài với Grading Saga

Chấm bài không phải là một giao dịch ngắn. Một bài nộp phải trải qua nhiều bước: lấy cấu hình bài tập, tải tệp từ RustFS, dựng môi trường Docker, chạy từng bước kiểm thử, tính điểm và ghi kết quả sang dịch vụ kết quả. Các tài nguyên này nằm ở nhiều hệ thống khác nhau, gồm cơ sở dữ liệu của dịch vụ thực thi, RustFS, Docker daemon và ba dịch vụ khác. Không thể bọc toàn bộ quy trình trong một giao dịch ACID. Khi một bước lỗi, hệ thống cần biết bài nộp đang ở bước nào, đã làm gì, có thể chấm lại hay không và làm sao tránh chấm trùng khi Kafka giao lại sự kiện. Đồ án áp dụng mẫu Saga [1] để theo dõi tiến trình chấm, gọi là Grading Saga, nhằm giải quyết các yêu cầu này.

Mỗi bài nộp có một bản ghi trong bảng grading_jobs. Bảng này lưu mã bài nộp, mã bài tập, mã sinh viên, mã kế hoạch chấm, đường dẫn tệp trong RustFS, trạng thái, số lần chạy lại, thông báo lỗi và các mốc thời gian. Trên cột mã bài nộp có một chỉ mục duy nhất, bảo đảm một bài nộp chỉ có một công việc chấm. Mỗi lần chạy chấm được ghi thành một bản ghi trong bảng saga và nhiều bản ghi bước; mỗi bước lưu tên bước, mã plan, mã bước, trạng thái, lỗi và thời điểm kết thúc. Một job có thể có nhiều lần chạy, nên có thể có nhiều bản ghi saga tương ứng. Kết quả từng bước được lưu trong grading_step_results, còn nhật ký dạng văn bản được lưu trong grading_logs.

Bảng 3.4 mô tả các trạng thái của một công việc chấm. Trạng thái của job là nguồn thông tin chính về tiến độ; trạng thái của saga và từng bước là dấu vết chi tiết để điều tra khi có lỗi.

Trạng thái	Ý nghĩa
PENDING	Sự kiện đã được tiếp nhận, công việc đang chờ worker chấm.
FETCHING	Đang lấy cấu hình bài tập và kế hoạch chấm từ dịch vụ quản lý lớp học và điểm.
BUILDING	Đang tải tệp bài nộp, chuẩn bị thư mục làm việc và dựng môi trường Docker.
RUNNING	Môi trường đã chạy, đang thực thi các bước chấm và tính điểm.
DONE	Chấm xong, kết quả đã được ghi nhận.
FAILED	Quá trình chấm dừng do lỗi; thông báo lỗi được lưu lại.

Bảng 3.4: Các trạng thái của một công việc chấm bài.

Các bước chính của một phiên chấm gồm:

- FETCH_CONFIG: gọi dịch vụ quản lý lớp học và điểm để lấy thông tin bài tập, kế hoạch chấm và các bước cần thực thi.
- DOWNLOAD_ARTIFACT: tải tệp nộp từ RustFS, giải nén và chuẩn bị thư

mục làm việc.

- BOOT_COMPOSE: vá tệp Docker Compose với cổng, tài nguyên và thông tin cơ sở dữ liệu cần thiết, sau đó dựng môi trường bằng Docker daemon trong Pod.
- RUN_STEPS: chạy các plan và từng bước theo thứ tự. Mỗi bước được ghi thành một bản ghi STEP: kèm tên bước. Nếu một bước bắt buộc thất bại, các bước còn lại của plan được đánh dấu SKIPPED thay vì tiếp tục chạy.
- SCORE_REPORT: tính điểm từ kết quả các bước, lưu trạng thái DONE, gửi kết quả sang dịch vụ kết quả và cập nhật trạng thái bài nộp.

Mỗi bước có thể ở một trong bốn trạng thái STARTED, DONE, FAILED hoặc SKIPPED. Saga tổng hợp có ba trạng thái STARTED, DONE và FAILED. Khi một bước không bắt buộc thất bại, quá trình chấm vẫn tiếp tục và điểm của bước đó được tính là không đạt; khi bước bắt buộc thất bại, plan hiện tại dừng lại nhưng các plan khác vẫn tiếp tục được xử lý.

Tính nhất quán được bảo vệ ở ba điểm. Thứ nhất, chỉ mục duy nhất trên cột mã bài nộp biến việc chấm trùng thành một lỗi ràng buộc cơ sở dữ liệu thay vì tạo thêm công việc. Khi nhận một sự kiện trùng, dịch vụ thực thi tìm công việc cũ; nếu công việc cũ đang ở trạng thái FAILED, dịch vụ đặt lại công việc và chấm lại, còn nếu không thì bỏ qua sự kiện. Thứ hai, khi hàng đợi xử lý bị đầy, công việc vẫn ở trạng thái PENDING thay vì bị mất; cơ chế quét công việc tồn đọng sẽ nhặt lại sau. Thứ ba, kết quả được gửi sang dịch vụ kết quả tối đa ba lần với thời gian chờ tăng dần; lỗi xác thực từ phía dịch vụ kết quả không được thử lại vì đó là lỗi dữ liệu, không phải lỗi tạm thời.

Cơ chế đặt lại công việc từ chối các công việc đã DONE. Với công việc chưa hoàn tất, hệ thống xóa kết quả từng bước, xóa các bước của saga, đưa saga về STARTED, đưa job về PENDING, xóa thông báo lỗi và tăng số lần chạy lại. Cơ chế quét công việc tồn đọng chạy định kỳ mỗi 5 phút, tìm các công việc ở trạng thái PENDING, FETCHING hoặc BUILDING đã quá 30 phút và chưa vượt quá ba lần chạy lại, rồi đưa chúng trở lại hàng đợi. Các công việc đang RUNNING không bị quét tự động vì một công việc đang chạy thật và một công việc bị treo do Pod chết có thể trông giống nhau nếu chỉ dựa vào thời gian; việc chấm lại một công việc đang chạy sẽ gây chấm trùng. Vì vậy, trường hợp Pod chết giữa lúc RUNNING cần được xử lý thủ công.

Cần phân biệt Grading Saga với một giao dịch phân tán có bù trừ tự động. Ở đây, saga chủ yếu đóng vai trò theo dõi và hỗ trợ phục hồi; các tài nguyên tạm như

container, cổng và thư mục làm việc được dọn trong khối finally khi tiến trình còn sống, nhưng không có hành động bù trừ tự động nếu Pod chết đột ngột. Việc ghi saga được thực hiện theo hướng best-effort: nếu ghi saga thất bại, quá trình chấm vẫn tiếp tục và lỗi chỉ được ghi log. Do đó, bảng job mới là nguồn sự thật về trạng thái chấm, còn saga là dấu vết phục vụ điều tra. Nếu việc gửi kết quả thất bại sau ba lần thử, job vẫn có thể ở trạng thái DONE trong khi kết quả chưa có ở dịch vụ kết quả; trường hợp này cần được đối soát và là một hạn chế được nêu ở mục 3.2.

3.1.7 Tổng hợp kết quả triển khai

Cấu hình triển khai hiện tại gồm sáu dịch vụ backend, Keycloak, RustFS và Kafka UI trong namespace web-grading, cùng sáu Application ArgoCD trong namespace argocd. Ngoài ra còn một Ingress controller Traefik, một tunnel Cloudflare và giao diện web trên Vercel. Các image backend được lấy từ Docker Hub theo tag phiên bản do workflow phát hành tạo ra, còn cấu hình cụm được lấy từ kho cấu hình theo mô hình GitOps.

Về luồng vận hành, một thay đổi mã nguồn đi qua các bước: chạy kiểm thử, tạo image, cập nhật tag trong kho cấu hình, ArgoCD đồng bộ và thay Pod. Mỗi bước đều để lại dấu vết trong GitHub Actions, GitHub Release, lịch sử commit của kho cấu hình và lịch sử đồng bộ của ArgoCD. Nhờ đó, khi một phiên bản gặp lỗi có thể xác định được image nào đang chạy, cấu hình nào đã tạo ra nó và commit nào cần đảo lại.

Mục 3.2 trình bày các tiêu chí và kịch bản đánh giá hệ thống dựa trên môi trường triển khai này.

3.2 Đánh giá hệ thống

Mục này đánh giá hệ thống ở hai góc: đánh giá định tính kiến trúc triển khai và xây dựng các kịch bản kiểm thử để thu thập dữ liệu định lượng. Các kịch bản xác định dữ liệu cần đo và cách đánh giá; số liệu thực tế được ghi nhận khi chạy trên cụm thử nghiệm.

3.2.1 Tiêu chí và phương pháp đánh giá

Hệ thống được đánh giá theo năm nhóm tiêu chí:

- Chức năng: các usecase chính hoạt động đúng theo đặc tả, phân quyền đúng vai trò và xử lý được các trường hợp lỗi đã nêu trong kịch bản.
- Hiệu năng: thời gian phản hồi của các API đọc và ghi, số yêu cầu xử lý được trong một đơn vị thời gian, thời gian chấm một bài và mức sử dụng tài nguyên của Pod.
- Độ tin cậy: khả năng phục hồi khi Pod bị xóa, khi Kafka giao lại sự kiện, khi

dịch vụ kết quả tạm thời không phản hồi và khi cấu hình cụm bị thay đổi ngoài Git.

- Bảo mật: kiểm soát truy cập theo vai trò, bảo vệ các API nội bộ bằng khóa gateway, thời hạn của presigned URL và cách lưu trữ Secret.
- Khả năng bảo trì: mức độ tự động của quy trình build, kiểm thử, phát hành, triển khai và khả năng quay lại phiên bản trước.

Phương pháp đánh giá kết hợp kiểm thử chức năng theo kịch bản, kiểm thử hiệu năng bằng k6 [38] và kiểm thử độ tin cậy bằng cách tác động vào cụm. Với kiểm thử chức năng, mỗi kịch bản ghi lại điều kiện đầu vào, các bước thực hiện, kết quả mong đợi và trạng thái cuối cùng của bài nộp hoặc công việc chấm. Với kiểm thử hiệu năng, các chỉ số được thu thập gồm thời gian phản hồi trung bình, p50, p95, p99, số yêu cầu mỗi giây, tỷ lệ lỗi và mức sử dụng CPU, bộ nhớ của Pod. Với kiểm thử độ tin cậy, hệ thống được quan sát sau mỗi tác động để xác định công việc chấm có tiếp tục, có bị chấm trùng hay không và trạng thái cuối cùng có rõ ràng hay không.

3.2.2 Kịch bản kiểm thử chức năng

Bảng 3.5 liệt kê các nhóm kịch bản chức năng. Các kịch bản sẽ được chạy theo thứ tự từ xác thực, quản lý lớp học, quản lý bài tập, nộp bài, chấm bài đến xem kết quả và trợ lý AI. Với mỗi nhóm, cột kết quả mong đợi mô tả trạng thái mà hệ thống phải đạt được; kết quả thực tế sẽ được đối chiếu và ghi nhận sau khi chạy.

Dữ liệu đầu vào cho mỗi nhóm sẽ được chuẩn bị theo một bộ mẫu chung: tài khoản thuộc các vai trò khác nhau, lớp có sẵn sinh viên, bài tập ở trạng thái nháp và đã công bố, cùng một bài nộp mẫu. Với các kịch bản lỗi, dữ liệu được chọn để kích hoạt đúng điều kiện biên như tệp quá lớn, sai định dạng hoặc sự kiện bị gửi lặp.

Nhóm chức năng	Kịch bản	Kết quả mong đợi
Xác thực và phân quyền	Đăng nhập đúng, đăng nhập sai, khóa tài khoản sau nhiều lần sai, đổi mật khẩu bắt buộc	Phiên làm việc hợp lệ; sai thông tin bị từ chối; tài khoản bị khóa đúng quy định; mật khẩu mới được áp dụng
Quản lý lớp học	Tạo lớp, nhập danh sách sinh viên từ CSV, xem danh sách lớp, cấu hình thành phần điểm	Lớp được tạo đúng; sinh viên hợp lệ được thêm; dòng lỗi được báo lại; thành phần điểm lưu đúng
Quản lý bài tập	Tạo bài tập, cấu hình các bước chấm, công bố, sửa và xóa bài tập	Bài tập và các bước được lưu; sinh viên chỉ thấy bài đã công bố; thao tác sai quyền bị từ chối
Nộp bài	Tải tệp qua presigned URL, tệp đúng định dạng, tệp quá lớn, tệp sai định dạng, webhook hoàn tất	Tệp được lưu trong RustFS; bài nộp chuyển trạng thái đúng; tệp lỗi bị từ chối với thông báo rõ ràng
Chấm tự động	Chấm bài hợp lệ, bài thiếu tệp, bài có bước bắt buộc thất bại, bài vượt thời gian, sự kiện chấm bị gửi trùng	Job đi qua đúng các trạng thái; kết quả và lỗi được lưu; không tạo job trùng; job lỗi có thể chấm lại
Kết quả chấm	Xem kết quả của lớp, xem chi tiết một bài, xem kết quả của bản thân	Điểm và chi tiết từng bước đúng với dữ liệu chấm; sinh viên chỉ thấy kết quả của mình
Trợ lý AI	Hỏi đáp về bài nộp, yêu cầu giải thích lỗi, mất kết nối tới nhà cung cấp mô hình	Câu trả lời có ngữ cảnh bài nộp; lỗi nhà cung cấp được báo lại và không làm hỏng dữ liệu hội thoại
Grading Saga	Gửi trùng sự kiện, đặt lại job FAILED, job tồn đọng quá 30 phút, Pod bị xóa khi đang chấm	Không chấm trùng; job được đưa về hàng đợi hoặc yêu cầu xử lý thủ công; trạng thái cuối cùng được ghi nhận

Bảng 3.5: Kịch bản kiểm thử chức năng.

3.2.3 Kịch bản kiểm thử hiệu năng và độ tin cậy

Kiểm thử hiệu năng sẽ được thực hiện bằng k6 với bốn loại kịch bản. Kịch bản smoke chạy một lượng nhỏ yêu cầu để xác nhận hệ thống hoạt động sau khi triển khai. Kịch bản load tăng dần số người dùng ảo để đo thời gian phản hồi và số yêu cầu mỗi giây ở mức tải dự kiến. Kịch bản stress đẩy tải vượt mức dự kiến để tìm điểm bão hòa và quan sát cách hệ thống suy giảm. Kịch bản spike tăng tải đột ngột trong thời gian ngắn để kiểm tra khả năng chịu đỉnh và tốc độ phục hồi.

Các endpoint được chọn để đo gồm đăng nhập qua Keycloak, lấy danh sách

bài tập, nộp bài, theo dõi trạng thái chấm và xem kết quả. Với luồng chấm, phép đo không chỉ là thời gian phản hồi HTTP mà còn là thời gian từ lúc sự kiện GRADE_SUBMISSION được phát đến khi job đạt trạng thái DONE hoặc FAILED. Kịch bản chấm đồng thời nhiều bài được dùng để kiểm tra giới hạn của hai bản sao dịch vụ thực thi và của Docker daemon trong mỗi Pod.

Kiểm thử độ tin cậy gồm các tác động sau: xóa Pod dịch vụ thực thi khi job đang ở trạng thái BUILDING hoặc RUNNING; gửi trùng sự kiện chấm; chặn tạm thời dịch vụ kết quả để quan sát cơ chế thử lại; thay đổi một tài nguyên trên cụm để kiểm tra selfHeal của ArgoCD; và quay lại image trước đó để kiểm tra khả năng rollback. Sau mỗi tác động, hệ thống được kiểm tra theo ba câu hỏi: job có tiếp tục hoặc được đưa lại hàng đợi hay không, có xuất hiện job trùng hay không, và trạng thái cuối cùng trong cơ sở dữ liệu có khớp với kết quả trong dịch vụ kết quả hay không.

3.2.4 Ưu điểm của kiến trúc triển khai

Kiến trúc triển khai có một số ưu điểm rõ ràng. Việc tách mỗi dịch vụ thành một Deployment riêng, một cơ sở dữ liệu riêng và một chart Helm riêng cho phép build, kiểm thử, phát hành và quay lại phiên bản của từng dịch vụ mà không ảnh hưởng đến các dịch vụ khác. Khi dịch vụ kết quả gặp lỗi, luồng nộp bài và chấm bài vẫn có thể tiếp tục ở mức độ nhất định; khi dịch vụ trợ lý AI gặp lỗi, các chức năng quản lý lớp học và chấm bài không bị dừng theo.

Mô hình GitOps tạo ra một chuỗi truy vết đầy đủ từ commit mã nguồn đến image và commit cấu hình. Mọi thay đổi trạng thái mong muốn của cụm đều nằm trong Git, nên có thể xem lại lịch sử, so sánh hai phiên bản và quay lại phiên bản trước bằng một commit. Cơ chế selfHeal và prune giảm các sai lệch do thao tác thủ công, còn việc tách workflow build khỏi workflow phát hành giúp chỉ những commit đã vượt qua kiểm thử mới được tạo image phát hành.

Luồng chấm bài bất đồng bộ giúp thao tác nộp bài không phải chờ kết quả chấm. Việc dùng chỉ mục duy nhất trên cột mã bài nộp, cơ chế đặt lại job và cơ chế quét job tồn đọng tạo thành ba lớp bảo vệ chống chấm trùng và chống kẹt job. Cách ghi saga theo từng bước cũng giúp xác định lỗi nằm ở bước tải tệp, dựng môi trường, chạy bước kiểm thử hay ghi kết quả, thay vì chỉ biết job thất bại.

Việc dùng các dịch vụ quản lý cho PostgreSQL và Kafka giảm công sức vận hành tầng dữ liệu, đồng thời cho phép tập trung vào phần tính toán và nghiệp vụ. Cách đóng gói image nhiều tầng, truyền cấu hình qua biến môi trường và dùng Helm values giúp cùng một image có thể chạy ở nhiều môi trường mà không phải sửa mã nguồn.

3.2.5 Hạn chế và rủi ro

Bên cạnh các ưu điểm, môi trường triển khai còn một số hạn chế cần được xử lý trước khi vận hành chính thức.

Thứ nhất, một số dịch vụ nội bộ như dịch vụ quản lý lớp học và điểm, dịch vụ nộp bài, dịch vụ kết quả chấm, dịch vụ thực thi chấm và dịch vụ trợ lý AI vẫn có Ingress riêng. Phần lớn các dịch vụ này kiểm tra khóa gateway ở tầng ứng dụng; riêng dịch vụ thực thi chấm chưa có kiểm tra này, nên endpoint đặt lại công việc có thể bị gọi trực tiếp nếu tên miền được biết. Việc mở thêm tên miền công khai làm tăng bề mặt tấn công và đi ngược mục tiêu chỉ có một điểm vào duy nhất. Khi vận hành chính thức, nên xóa các Ingress không cần thiết hoặc giới hạn chúng bằng chính sách mạng và Cloudflare Access; riêng dịch vụ thực thi chấm cần được bảo vệ bằng khóa gateway trước khi giữ lại Ingress.

Thứ hai, Keycloak đang chạy ở chế độ start-dev và giao diện quản trị được mở công khai. Chế độ này thuận tiện cho phát triển nhưng không phù hợp với môi trường thật vì giảm các kiểm tra bảo mật và không tối ưu hiệu năng. Cần chuyển sang chế độ vận hành chính thức, đổi tài khoản quản trị mặc định và giới hạn đường dẫn /admin bằng Cloudflare Access hoặc quy tắc tường lửa ứng dụng web (WAF).

Thứ ba, dịch vụ thực thi chấm dùng Docker-in-Docker với container privileged. Cách làm này cần thiết để dựng môi trường Docker của bài nộp, nhưng nếu một bài nộp khai thác được lỗ hổng trong container thì mức độ ảnh hưởng có thể lan sang node. Hướng khắc phục là dùng Docker daemon không đặc quyền, rootless container hoặc một môi trường cách ly mạnh hơn như Kata Containers hoặc gVisor.

Thứ tư, cơ chế ghi kết quả chưa có bảo đảm giao dịch. Kết quả được gửi tối đa ba lần, nhưng nếu cả ba lần đều thất bại thì job vẫn có thể ở trạng thái DONE trong khi dịch vụ kết quả chưa có bản ghi. Ngoài ra, cơ chế quét job tồn đọng không xử lý trạng thái RUNNING; nếu Pod chết giữa lúc chạy, cần đặt lại job thủ công. Hai trường hợp này cần được đối soát định kỳ hoặc xử lý bằng một outbox/lease trong tương lai.

Thứ năm, hệ thống chỉ dùng một topic Kafka cho toàn bộ sự kiện do giới hạn của gói miễn phí. Cách này đơn giản nhưng không tách được thời gian lưu trữ, quyền truy cập và số partition theo từng miền nghiệp vụ; khi lưu lượng tăng, một topic duy nhất có thể trở thành điểm nghẽn. Việc tiêu thụ với số bản ghi tối đa mỗi lần lấy bằng 1 cũng giới hạn tốc độ xử lý của mỗi consumer.

Thứ sáu, RustFS chạy một bản sao với PVC ReadWriteOnce và cặp khóa mặc định đang đặt trực tiếp trong manifest. Một Pod RustFS gặp sự cố sẽ làm gián đoạn

việc tải và đọc tệp bài nộp; dữ liệu cũng chưa có phương án sao lưu tự động. Cần chuyển khóa sang Secret, bật sao lưu và cân nhắc chạy nhiều bản sao hoặc dùng lưu trữ đối tượng bên ngoài.

Thứ bảy, cụm k3s hiện tại chưa có cấu hình sẵn sàng cao và chưa có autoscaling. Số bản sao của dịch vụ thực thi cố định là hai, nên không tự tăng khi hàng đợi dài. Hệ thống giám sát tập trung chưa được trình bày trong phạm vi báo cáo; việc thiếu metrics, log tập trung và cảnh báo trong phạm vi đánh giá này làm chậm quá trình phát hiện sự cố. Đây là nội dung được đặt trong hướng phát triển.

Thứ tám, quy trình CI/CD chưa có bước quét lỗ hổng image, tạo danh mục thành phần phần mềm (SBOM) hoặc ký image; kiểm thử chỉ chạy cho các dịch vụ thay đổi và loại trừ các lớp ApplicationTests. Những khoảng trống này không ảnh hưởng đến chức năng nhưng làm giảm mức độ bảo đảm chất lượng khi số lượng thay đổi tăng.

3.3 Hướng phát triển

Từ các hạn chế trên, hướng phát triển của hệ thống được chia thành các nhóm sau.

Về khả năng quan sát, cần bổ sung hệ thống giám sát tập trung gồm Prometheus và Grafana cho số liệu, Loki cho nhật ký, Tempo cho trace và Pyroscope cho profiling. Các thành phần này sẽ giúp theo dõi độ trễ, tỷ lệ lỗi, mức sử dụng tài nguyên và hành vi của từng job chấm; trên cơ sở đó đặt ra chỉ tiêu dịch vụ và cảnh báo khi có bất thường. Đây là phần chưa được trình bày trong phạm vi báo cáo.

Về độ tin cậy và khả năng mở rộng, cần chuyển cụm k3s lên cấu hình nhiều node hoặc dùng dịch vụ Kubernetes quản lý, bổ sung anti-affinity và ngân sách gián đoạn cho các dịch vụ quan trọng, đồng thời dùng Horizontal Pod Autoscaler hoặc KEDA để tăng số bản sao dịch vụ thực thi theo độ dài hàng đợi. Với dịch vụ thực thi, cần thay cơ chế Docker-in-Docker privileged bằng một môi trường cách ly an toàn hơn, hoặc tách việc chạy bài nộp sang một cụm worker riêng.

Về dữ liệu, cần tách topic Kafka theo từng miền nghiệp vụ, tăng số partition và đặt chính sách lưu trữ phù hợp; bổ sung outbox hoặc cơ chế lease để bảo đảm kết quả chấm được ghi ít nhất một lần và để xử lý job RUNNING bị treo. Với PostgreSQL và RustFS, cần có phương án sao lưu, phục hồi và kiểm thử phục hồi định kỳ.

Về bảo mật, cần chuyển Keycloak sang chế độ vận hành chính thức, giới hạn giao diện quản trị, xóa các Ingress không cần thiết, bổ sung NetworkPolicy giữa các namespace và dịch vụ, đưa Secret sang một hệ thống quản lý tập trung như

External Secrets hoặc Vault, đồng thời thêm bước quét lỗ hổng, tạo danh mục thành phần phần mềm (SBOM) và ký image vào quy trình CI/CD.

Về kiểm thử, cần hoàn thiện bộ kiểm thử đầu-cuối cho tất cả usecase, chạy các kịch bản hiệu năng và độ tin cậy đã nêu, ghi nhận số liệu thực tế và bổ sung kiểm thử phá hoại để đánh giá khả năng phục hồi. Kết quả của các lần chạy này sẽ là cơ sở để đánh giá định lượng thay cho phần đánh giá định tính hiện tại.

3.4 Kết luận chương

Chương này đã trình bày cách hệ thống được đóng gói và triển khai: giao diện trên Vercel, backend trên cụm k3s, PostgreSQL và Kafka dưới dạng dịch vụ quản lý, RustFS và Keycloak trong cụm. Hệ thống dùng Docker để đóng gói, Helm để mô tả tài nguyên, GitHub Actions để build và phát hành, ArgoCD để đồng bộ theo GitOps, và các Secret của Kubernetes để tách cấu hình nhạy cảm khỏi mã nguồn.

Chương cũng đã trình bày cơ chế Grading Saga để theo dõi quy trình chấm nhiều bước, chống chấm trùng bằng chỉ mục duy nhất trên cột mã bài nộp, đặt lại job khi cần và quét job tồn đọng. Các cơ chế này không thay thế một giao dịch phân tán, nhưng bảo đảm mỗi bài nộp có trạng thái cuối cùng rõ ràng và có dấu vết để điều tra khi một bước gặp lỗi.

Phần đánh giá đã xác định các tiêu chí, kịch bản kiểm thử chức năng, hiệu năng và độ tin cậy, đồng thời chỉ ra các hạn chế về bề mặt tấn công, chế độ chạy của Keycloak, mức cách ly của Docker-in-Docker, tính nhất quán khi ghi kết quả, giới hạn của một topic Kafka, cấu hình RustFS và mức độ sẵn sàng của cụm. Các hướng phát triển tương ứng đã được nêu để tiếp tục hoàn thiện hệ thống trước khi đưa vào vận hành chính thức.

TÀI LIỆU THAM KHẢO

[1] S. Newman, *Building Microservices*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[2] Oracle. "Oracle JDK 21 documentation. "[Online]. Available: https://docs.oracle.com/en/java/javase/21/

[3] VMware. "Spring boot reference documentation. "[Online]. Available: https://docs.spring.io/spring-boot/

[4] VMware. "Spring cloud openfeign reference documentation. "[Online]. Available: https://docs.spring.io/spring-cloud-openfeign/docs/current/reference/html/

[5] VMware. "Spring security reference documentation. "[Online]. Available: https://docs.spring.io/spring-security/reference/

[6] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

[7] PostgreSQL Global Development Group. "Postgresql documentation. "[Online]. Available: https://www.postgresql.org/docs/

[8] Red Hat. "Hibernate ORM documentation. "[Online]. Available: https://hibernate.org/orm/documentation/

[9] Redgate. "Redgate flyway documentation. "[Online]. Available: https://documentation.red-gate.com/flyway/

[10] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proceedings of the NetDB Workshop*, Athens, Greece, 2011.

[11] Apache Software Foundation. "Apache kafka documentation. "[Online]. Available: https://kafka.apache.org/documentation/

[12] Amazon Web Services. "Amazon S3 documentation. "[Online]. Available: https://docs.aws.amazon.com/s3/

[13] RustFS. "Rustfs documentation. "[Online]. Available: https://docs.rustfs.com/

[14] RustFS. "Rustfs source code repository. "[Online]. Available: https://github.com/rustfs/rustfs

[15] Docker. "Docker documentation. "[Online]. Available: https://docs.docker.com/

[16] Cloud Native Computing Foundation. “Kubernetes documentation. ”[Online]. Available: https://kubernetes.io/docs/home/

[17] Rancher. “K3s documentation. ”[Online]. Available: https://docs.k3s.io/

[18] Cloud Native Computing Foundation. “Helm documentation. ”[Online]. Available: https://helm.sh/docs/

[19] Cloud Native Computing Foundation. “Argo CD documentation. ”[Online]. Available: https://argo-cd.readthedocs.io/

[20] Cloud Native Computing Foundation. “Keycloak documentation. ”[Online]. Available: https://www.keycloak.org/documentation

[21] D. Hardt, “The OAuth 2.0 authorization framework,” Internet Engineering Task Force (IETF), RFC 6749, Oct. 2012. [Online]. Available: https://www.rfc-editor.org/info/rfc6749

[22] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, “OpenID connect core 1.0,” OpenID Foundation, Tech. Rep., Feb. 2014. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html

[23] M. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” Internet Engineering Task Force (IETF), RFC 7519, May 2015. [Online]. Available: https://www.rfc-editor.org/info/rfc7519

[24] N. Sakimura, J. Bradley, and N. Agarwal, “Proof key for code exchange by OAuth public clients,” Internet Engineering Task Force (IETF), RFC 7636, Sep. 2015. [Online]. Available: https://www.rfc-editor.org/info/rfc7636

[25] W. Denniss and J. Bradley, “OAuth 2.0 for native apps,” Internet Engineering Task Force (IETF), RFC 8252, Oct. 2017. [Online]. Available: https://www.rfc-editor.org/info/rfc8252

[26] Meta. “React documentation. ”[Online]. Available: https://react.dev/

[27] Microsoft. “Typescript documentation. ”[Online]. Available: https://www.typescriptlang.org/docs/

[28] Evan You. “Vite guide. ”[Online]. Available: https://vite.dev/guide/

[29] Ant Group. “Ant design documentation. ”[Online]. Available: https://ant.design/docs/react/introduce

[30] Remix Software. “React router documentation. ”[Online]. Available: https://reactrouter.com/

[31] TanStack. “Tanstack query documentation. ”[Online]. Available: https://tanstack.com/query/latest

[32] i18next. “I18next documentation. ”[Online]. Available: https://www.i18next.com/

[33] Cloudflare. “Cloudflare documentation. ”[Online]. Available: https://developers.cloudflare.com/

[34] Vercel. “Vercel documentation. ”[Online]. Available: https://vercel.com/docs

[35] GitHub. “Github actions documentation. ”[Online]. Available: https://docs.github.com/en/actions

[36] Neon. “Neon documentation. ”[Online]. Available: https://neon.com/docs

[37] Aiven. “Aiven documentation. ”[Online]. Available: https://aiven.io/docs

[38] Grafana Labs. “Grafana k6 documentation. ”[Online]. Available: https://grafana.com/docs/k6/